通用图形处理器

性能分析与调优

Performance Analysis and Tuning for General Purpose Graphics Processing Units (GPGPU)

Hyesoon Kim　Richard Vuduc

Sara Baghsorkhi　Jee Choi

Wen-mei Hwu

中文学习修订版

以 2012 年英文原著为底本整理，非官方授权译本

原著：Morgan & Claypool Publishers，2012

译稿说明

本资料依据 Hyesoon Kim 等人 2012 年出版的英文原著整理，保留其技术背景、术语、公式与参考文献。中文译文仅用于个人学习和研究对照，并非官方授权出版版本。原著中的设备性能与软件说明反映出版时的情况。

摘要

通用图形处理器（GPGPU）已成为共享内存并行处理体系结构中的重要一类，广泛应用于从高端超级计算机到嵌入式移动平台的各类计算系统。与本书出版时期更传统的多核系统相比，GPGPU 具有显著更高的硬件多线程程度（数百个硬件线程上下文，而非几十个）、更宽的向量执行单元（数十路，而非 1～10 路）、更高的峰值内存带宽（每秒数百吉字节，而非数十吉字节），但其缓存和暂存存储器通常更小（不足 1 MB，而非 1～10 MB）。

本书首先从较高层次概述当时的 GPGPU 架构及其编程模型，回顾既有共享内存并行计算平台的基本原理，重点介绍并行算法理论与实践的新近研究成果，并分析它们与 GPGPU 平台之间的联系。本书希望帮助体系结构研究人员从算法角度理解 GPGPU，也提供从高层算法设计到低层指令优化的详细性能分析和调优方法。作为案例，本书采用 n 体粒子仿真的快速多极子方法（FMM），并简要综述 GPU 性能分析工具与技术的最新研究进展（以 2012 年出版时为准）。

关键词： GPGPU；性能分析；性能建模；CUDA

目录

译稿说明 ii

摘要 iii

致谢 vi

第一章 GPU 设计、编程与趋势 1

1.1 GPU 简史 . . . 1
1.2 GPU 系统简要概述 . . . 1
1.2.1 GPU 架构概述 . . . 2
1.3 GPGPU 编程模型：CUDA . . . 4
1.3.1 内核 . . . 4
1.3.2 CUDA 中的线程层次结构 . . . 5
1.3.3 内存层次结构 . . . 6
1.3.4 SIMT 执行 . . . 8
1.3.5 CUDA 语言扩展 . . . 9
1.3.6 向量加法示例 . . . 9
1.3.7 PTX . . . 12
1.3.8 一致性模型和特殊内存操作 . . . 12
1.3.9 IEEE 浮点支持 . . . 12
1.3.10 OpenCL 的执行模型 . . . 12
1.4 GPU 架构 . . . 13
1.4.1 GPU 流水线 . . . 13
1.4.2 处理分支指令 . . . 18
1.4.3 GPU 内存系统 . . . 20
1.5 其他 GPU 架构 . . . 22
1.5.1 Fermi 架构 . . . 22
1.5.2 AMD 架构 . . . 22
1.5.3 集成众核架构（MIC） . . . 22
1.5.4 在同一芯片上集成 CPU 与 GPU . . . 23

第二章 性能原则 24

2.1 理论：算法设计模型概述 . . . 24
2.2 刻画并行性：工作-深度模型 . . . 24
2.3 刻画 I/O 行为：外部内存模型 . . . 28
2.4 并行性与 I/O 效率的综合分析 . . . 32
2.5 抽象与具体度量 . . . 33
2.6 小结 . . . 35

第三章 从原理到实践：分析与调优 36

3.1 计算问题：粒子相互作用 . . . 36
3.2 最优近似：快速多极子方法 . . . 36
3.3 设计并行且 I/O 高效的算法 . . . 38
3.4 基线实现 . . . 40
3.5 设定优化目标 . . . 40

3.5.1 识别候选优化 41
3.5.2 探索优化空间 42
3.5.3 总结 42

第四章 用详细的性能分析指导优化 **44**
4.1 指令级分析与调优 44
4.1.1 执行时间建模 44
4.1.2 将模型应用于 FMM 50
4.1.3 性能优化指南 50
4.2 其他性能建模技术和工具 53
4.2.1 有限的性能可见性 53
4.2.2 工作流图 54
4.2.3 随机内存层次模型 55
4.2.4 屋顶线模型 57
4.2.5 使用 Ocelot 对 CUDA 工作负载进行性能分析与剖析 57
4.2.6 其他 GPGPU 性能建模技术与工具 60

参考文献 **62**

作者传记 **72**

致谢

首先,我们要感谢 Mark Hill 和 Michael Morgan 邀请我们撰写这本综合讲座,并感谢他们的支持。非常感谢 Tor M. Aamodt、Mark Hill 及其威斯康星大学 2012 年秋季 CS 758 课程的学生以及匿名审稿人提供的审阅意见。我们感谢 Aparna Chandramowlishwaran、Kenneth Czechowski、Sunpyo Hong、Hyojong Kim、Nagesh B Lakshmanarayana、Jaekyu Lee、Jieun Lim、Nimit Niagania 和 Jaewoong Sim 的评论以及在收集材料方面的帮助。第 4.2.5 章由 Ocelot 团队的 Andrew Kerr 编写。本工作部分得到了美国国家科学基金会（NSF）的支持，资助编号为 NSF CAREER 奖 0953100、NSF CAREER 奖 1139083、NSF I/UCRC 奖 0934114；NSF TeraGrid 分配 CCR-090024 和 ASC-100019；NSF 与半导体研究公司（SRC）联合奖 0903447 和 1981；美国能源部（DOE）根据奖项 DE-FC02-10ER26006/DE-SC0004915；以及国防高级研究计划局（DARPA）计算机科学研究组计划、桑迪亚国家实验室和 NVIDIA 的资助。本材料中表达的任何意见、发现和结论或建议均为作者的观点，不一定反映 NSF、SRC、DOE、DARPA 或 NVIDIA 的观点。

Hyesoon Kim、Richard Vuduc、Sara Baghsorkhi、Jee Choi 与 Wen-mei Hwu

2012 年 10 月

第一章　GPU 设计、编程与趋势

本书旨在帮助读者理解在通用图形处理器（GPGPU）硬件上编程时出现的关键性能问题。尽管有许多具有类似目标的优秀资源可用，但本书强调算法设计的一般原理以及这些原理如何转化为底层 GPGPU 架构和编程。它包括 GPGPU 架构和编程的简要概述（第 1 章）、高层算法设计理论（第 2 章）、将理论转化为 GPGPU 性能工程实践的案例研究（第 3 章），以及当前 GPGPU 底层性能建模和分析的最新进展综述（第 4 章）。综上所述，我们希望这些材料能为 GPGPU 的性能理解提供一个独特的端到端视角。

1.1　GPU 简史

顾名思义，图形处理器（GPU）最初附在视频卡上，专门设计用于加速图形渲染和显示。这些早期的 GPU 采用固定图形管线，随后经过几代硬件演进为可编程核心。

最早的通用 GPU（GPGPU）编程接口仅限于着色器语言，如 DirectX [80]、OpenGL [74] 和 Cg [64]。因此，程序只能以图形管线操作的形式表达。后来，Brook+ [3] 和 Sh [66] 为 C 语言添加了流扩展，以抽象掉图形硬件细节。GPU 加速器 [88] 通过提供具有聚合逐元素操作的数据并行数组，从流编程又向前迈进了一步。

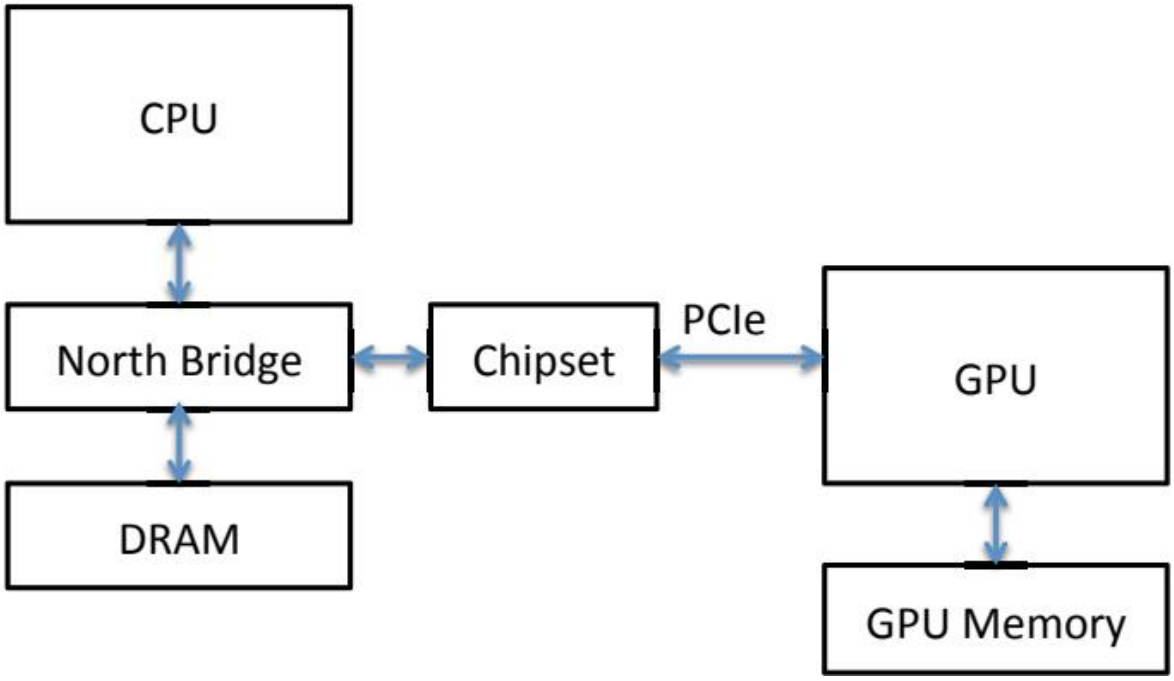
虽然这些早期研究努力相比着色器语言提高了 GPU 的通用可编程性，但 GPGPU 计算的转折点出现在 NVIDIA 推出名为 G80 系列的新型可编程统一图形和计算处理器时。该架构包括一种新的软硬件结合架构，称为统一计算设备架构（CUDA），它在使 GPGPU 计算流行方面发挥了重要作用。最近，更通用的术语异构计算已经涵盖了 GPGPU 编程，特别是随着更通用的编程模型（如 OpenCL、OpenACC 和 Microsoft C++ AMP）的引入。本书侧重于 GPGPU 编程，而不是更专门的图形编程概念或更通用的异构计算概念。此外，我们仅关注 GPU 架构的 GPGPU 方面，而不是图形应用特有的方面。

1.2　GPU 系统简要概述

在本节中，我们简要概述基于 NVIDIA G80/Fermi 架构的基线 GPU 架构，以及使用 CUDA 编程模型的 GPGPU 编程。

图 1.1 显示了 GPU 通常如何与现代处理器连接。GPU 是一个加速器（或协处理器），连接到主机处理器（通常是传统的通用 CPU 处理器）。主机处理器和 GPU 通过

PCI Express（PCIe）相互通信，提供每通道约 4 GT/s（第二代）或 8 GT/s（第三代）的传输速率。这种通信带宽往往成为最大的瓶颈之一；因此，只有当使用 GPU 的好处超过卸载成本时，将工作卸载到 GPU 才至关重要。预计未来随着系统内存的 CPU 总线带宽增加，通信带宽也将增长。



图中 North Bridge 表示北桥，Chipset 表示芯片组，GPU Memory 表示显存。

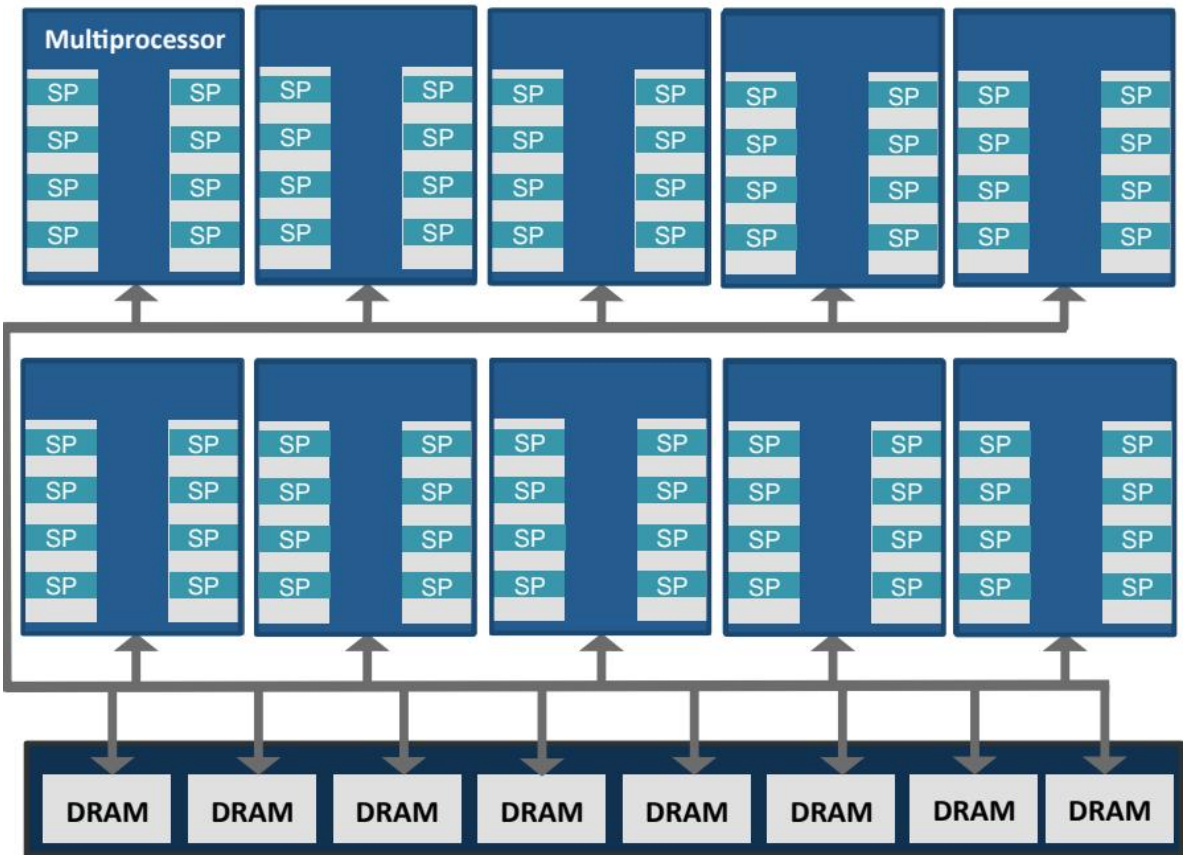
图 1.1　包含 CPU 和独立 GPU 的系统概览。

1.2.1　GPU 架构概述

图 1.2 基于 G80 的简化框图展示了通用图形处理器的主要组件。从高层来看，GPU 架构由多个流多处理器（SM）组成，这些 SM 与 GPU 的 DRAM 相连。（NVIDIA 称之为 SM，AMD 称之为计算单元（CU）。）每个 SM 拥有多个单指令多数据（SIMD）单元，也称为流处理器（SP），并支持多线程执行机制。GPU 架构采用两种重要的执行范式，我们将在下文进行解释。

SIMD/SIMT GPU 处理器提供高浮点（FP）执行带宽，这是设计图形应用的驱动力。为了高效利用大量 FP 单元，GPU 架构采用 SIMD 或 SIMT 执行范式（我们将在第 1.3.4 节解释 SIMT 并与 SIMD 进行对比）。在 SIMD 中，一条指令操作多个数据（即只取指、解码和调度一条指令，但作用于多个数据操作数）。根据字宽的不同，当前系统上单条指令可执行 32、64 或 128 次 FP 操作。这种技术显著提高了系统吞吐量，也提升了能效。这种 SIMD 风格的执行本质上与早期向量处理器使用的技术相同。为了支持 SIMD/向量操作，寄存器堆应提供高带宽的读写操作。

多线程 GPU 架构采用的另一种重要执行范式是硬件多线程。与更传统的高度多线程架构（如 HEP [86]、M-Machine [35]、Tera MTA [87]）类似，GPU 处理器使用基



图中 Multiprocessor 表示多处理器，SP 表示流处理器，DRAM 表示动态随机存储器。

图 1.2 NVIDIA G80 图形处理器的框图（未显示图形处理所需的组件）。

于硬件的快速上下文切换来容忍较长的内存和操作延迟。

多线程的有效性取决于应用程序能否提供大量并发线程。大多数图形应用具有这一特征，因为它们通常

需要同时处理许多对象（例如像素、顶点、多边形）。多线程是理解 GPGPU 应用性能行为的关键。在传统 CPU 系统中，线程上下文切换的成本相对高得多：所有程序状态（如 PC（程序计数器）、架构寄存器和栈信息）都需要由操作系统存储到内存中。然而，在 GPU 中，由于硬件原生支持这一过程，线程切换的成本要低得多。尽管现代 CPU 也实现了硬件多线程（例如 Intel 的超线程），但迄今为止，CPU 的多线程程度（同时硬件线程上下文的数量）远低于 GPU（例如两个 CPU 硬件线程与数百个 GPU 线程）。

为了在硬件中支持多线程，处理器必须维护大量寄存器、PC 寄存器和内存操作缓冲区。拥有大型寄存器堆尤为关键。例如，G80 架构具有 16 KB 的一级软件管理缓存（共享内存），同时拥有 32 KB 的寄存器堆。这种大型寄存器堆降低了线程间上下文切换的成本。这一事实是与传统 CPU 架构的关键区别。我们将在本章后面更详细地讨论这些及其他架构特性。

1.3　GPGPU 编程模型：CUDA

本节简要讨论 CUDA 编程模型。读者应参考最新的官方 CUDA 编程指南以获取更多细节 [73]。以下部分解释摘自 Hwu 和 Kirk 的文章 [31]。

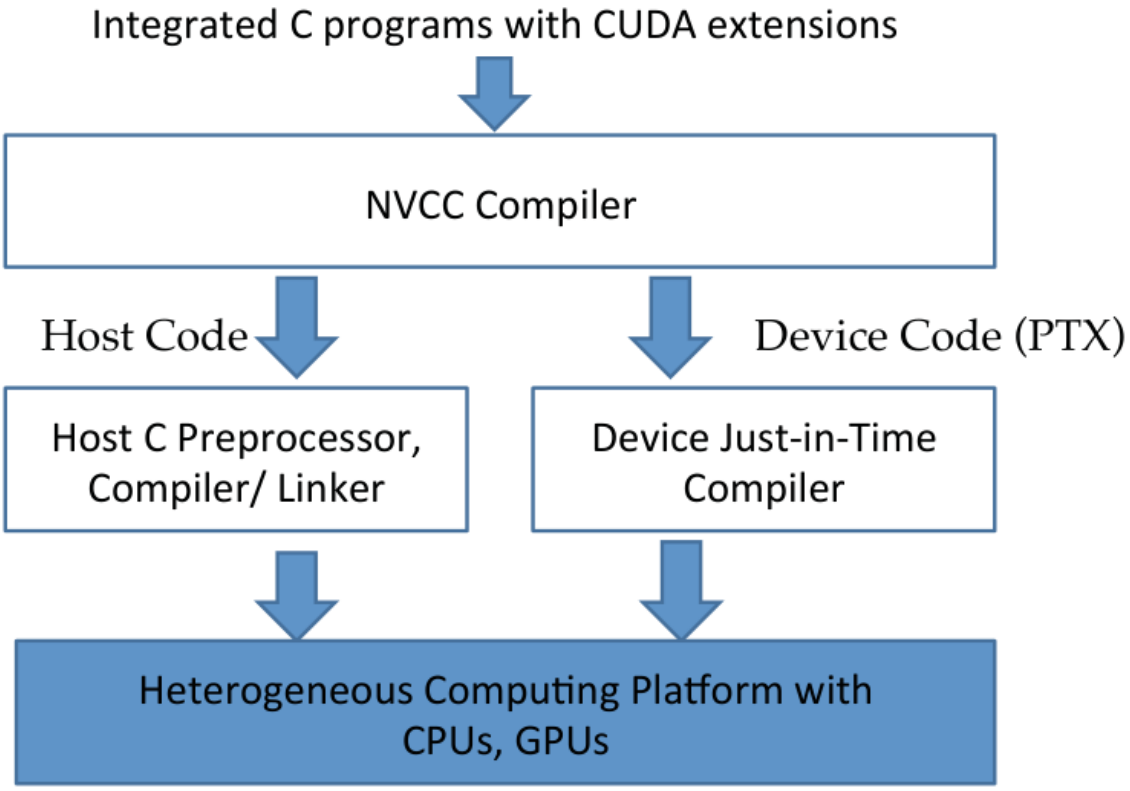
1.3.1　内核

NVIDIA 的 CUDA（统一计算设备架构）编程接口是用于 GPGPU 编程的 C/C++ API。CUDA 程序包含在主机（通常是主 CPU 处理器）上运行的代码和在设备（GPU 协处理器）上运行的内核。

CUDA 源代码混合了在 CPU 上运行的主机代码和在 GPU 上运行的设备代码。设备代码通常也称为内核函数。主机代码使用标准编译器编译；设备代码首先转换为称为 PTX 的中间设备语言，这实现了第一轮代码优化。随后，此 PTX 表示被转换为设备特定的二进制代码，其中封装了针对特定 GPU 的优化。图 1.3 展示了 CUDA 程序的编译过程概览。

图 1.4 展示了 CUDA 程序的执行过程，通常从主机（CPU）执行开始。当内核函数被调用或启动时，它由设备上的大量线程执行。内核函数在主机代码中使用扩展的函数调用语法进行调用，例如 `funcKernel<<<dimGrid, dimBlock>>>(args)`。`funcKernel()` 将在 GPU 上执行。内核启动生成的所有线程统称为一个网格。图 1.4 展示了两个线程网格的执行。内核中的所有线程完成后，相应的网格终止。主机恢复 CPU 代码。当主机代码遇到另一个内核时，它再次调用 GPU 计算。此示例展示了一个简化模型，其

中 CPU 执行和 GPU 执行不重叠。许多异构计算机应用，包括较新的 CUDA 编程模型，可以重叠 CPU 和 GPU 之间的计算，甚至可以重叠 GPU 内核执行。



图中 Host Code 为主机代码，Device Code 为设备代码，Compiler 为编译器。

图 1.3 CUDA 程序的编译过程概览（由 Kirk & Hwu 的书 [31] 提供）。

1.3.2 CUDA 中的线程层次结构

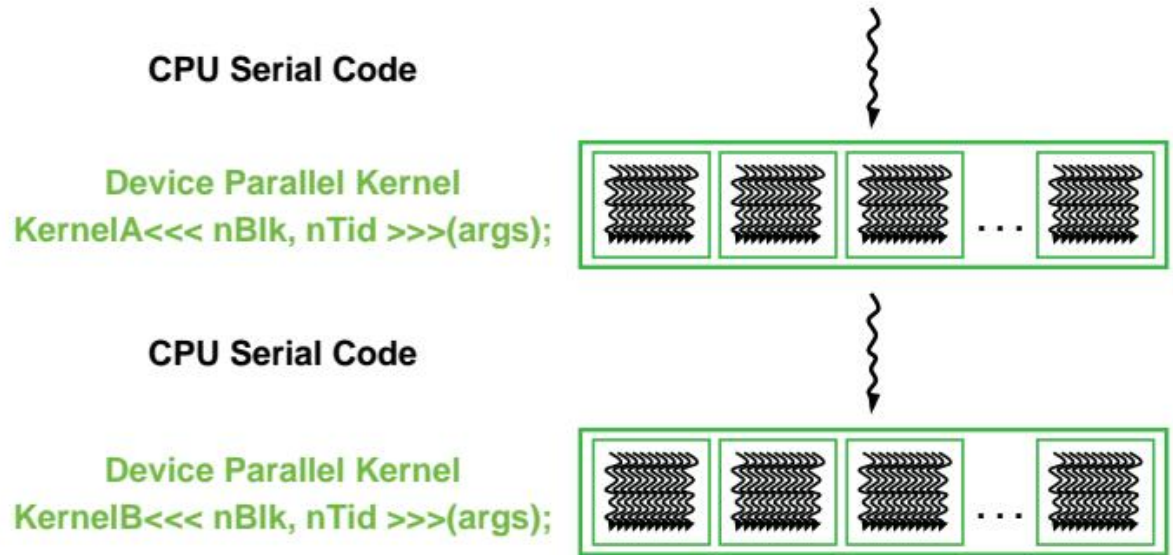
CUDA 编程模型使用由线程、块（或线程块）和网格组成的三级执行层次结构。最小的工作单元是线程。线程可以分组为块（线程块）。块也称为协作线程数组（CTA）。当内核执行时，

它在一个网格上执行，网格由一个或多个线程块组成。这种执行层次结构与（1）内存空间、（2）同步以及（3）调度和退役粒度紧密耦合。在实际硬件中，线程以锁步方式执行，形成另一个执行单元，称为 warp。（NVIDIA 术语中称为 warp，AMD 术语中称为 wavefront。）warp 的概念将在第 1.4.1 节讨论。

当内核 `funcKernel<<<dimGrid, dimBlock>>>(args)` 被调用时，总共有 dimGrid 个块，每个块有 dimBlock 个线程。因此，总共有 dimGrid × dimBlock 个线程执行同一个内核。

线程/块的执行。

一个块中的所有线程并发运行。这里的并发执行并不意味着所有线程每个周期都



图中 CPU Serial Code 为 CPU 串行代码，Device Parallel Kernel 为 GPU 并行内核。

图 1.4　CUDA 程序的执行（由 NVIDIA [72] 提供）。

执行。而是指一个块中的所有线程都处于活动状态[1]，并以时间多路复用的方式执行。

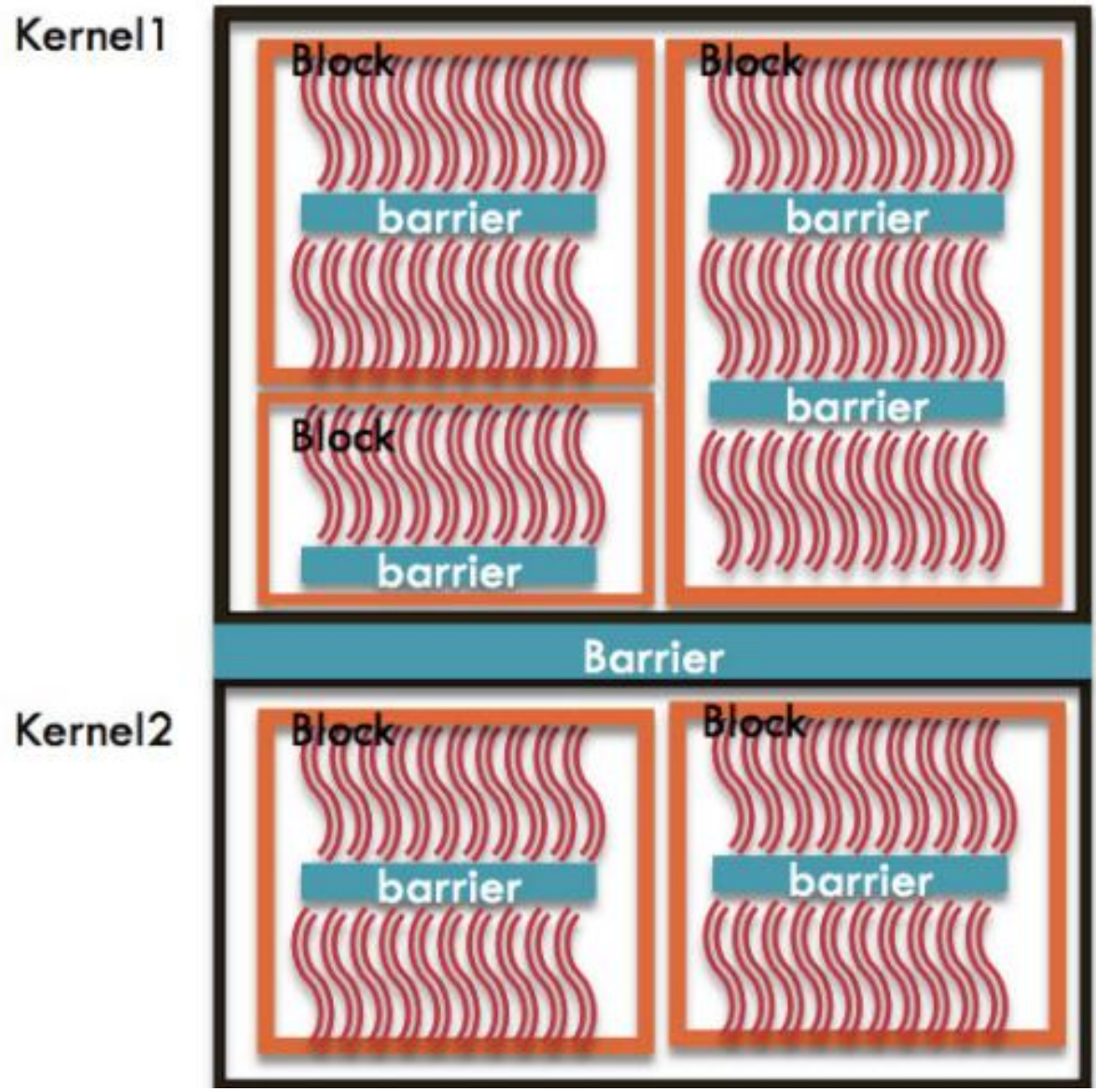
因此，线程块是最小的调度和退役单元。一个块中的所有线程一起调度，当块中的所有线程完成时，该块才能退役。目前，一旦一个块被分配给流多处理器（SM），它就不能迁移到另一个 SM。线程块中的所有线程异步执行，直到它们到达一个屏障。因此，CUDA 编程模型是批量同步并行编程模型的一个实例 [33，91]。在这个模型中，如图 1.5 所示，程序员使用屏障来同步所有线程。然而，没有机制可以在所有块之间进行同步，因为并非所有块都在硬件中同时执行。一个块只有在有足够资源（如寄存器）用于执行一个块时才能开始执行。在给定时间内，运行块的数量取决于程序的资源需求和可用的硬件资源。当所有块完成时，内核才能完成。因此，内核之间存在一个隐式屏障。

CUDA 编程模型没有指定块之间的执行顺序。在给定时间内，任意数量的块可能在硬件上执行。这种灵活性使得硬件能够实现可扩展性，如图 1.6 所示。根据机器的资源，不同数量的块同时执行。在图 1.6（左）所示的低成本系统中，只有两个块一起执行；在当前的高端系统中，最多可能有四个块在执行。目前，块由驱动程序调度。块与核心之间没有特定的映射关系。

1.3.3　内存层次结构

执行层次结构也与内存层次结构相关联。首先，每个线程有自己的私有寄存器。在内存方面，线程有自己的内存空间，称为本地内存。块也有自己的内存空间，称为共享内存。一个块中的所有线程可以访问相同的共享内存，而其他块中的线程则不能。

[1]有一些例外情况，其中一些线程可能比其他线程提前终止。但大多数情况下，在一个块的执行时间内，所有线程都是活跃的。



图中 Kernel 为内核，Block 为线程块，Barrier 为同步屏障。

图 1.5 CUDA：批量同步编程模型（由 [72] 提供）。

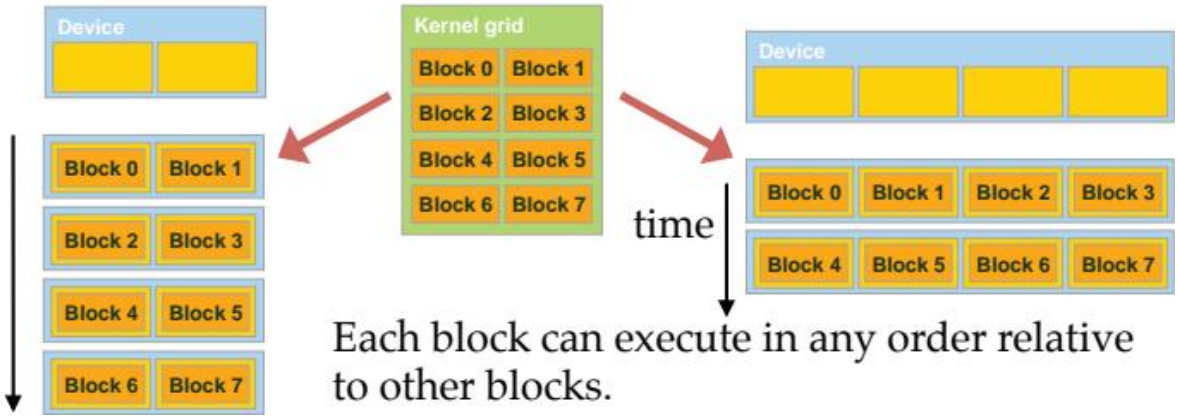


图 1.6 CUDA 块执行模式（由 [72] 提供）。

整个内核也有自己的内存空间，称为全局内存。之所以称为全局内存，是因为任何块中的任何线程都可以访问全局内存空间。表 1.1 总结了各种内存空间及其与执行层次结构的关系。

还有常量内存和纹理内存空间，它们与全局内存类似，所有线程和块都可以访问它们。然而，这些内存专门用于图形应用。纹理内存用于存储纹理数据（2D、3D 数据），常量内存用于存储非常少量的常量变量。纹理内存和常量内存从设备端都只能读取。只有主机处理器可以将数据写入这两个内存空间。

第 1.4 节中的表 1.4 显示了 GPU 架构中各种内存空间的典型访问时间。

表 1.1　内存空间比较

内存空间	**空间**	**≃ CPU**	**CPU 中的编程模型**
本地内存	线程内	栈	私有内存空间
共享内存	块内	分布式内存空间	分布式内存编程
全局内存	所有	集中式存储	OpenMP 编程
常量内存	所有	集中式只读存储（极小）	
纹理内存	所有	集中式只读存储（中等大小，2D 访问）	

表 1.1 还比较了传统的并行编程范式。访问共享内存空间与分布式内存（MPI）编程模型中的访问非常相似。在不同块拥有的共享内存区域之间，CUDA 程序通过全局内存空间显式复制数据。然而，在全局内存空间中，所有线程都可以访问该空间，因此 CUDA 程序在这方面类似于 OpenMP 编程模型。

1.3.4　SIMT 执行

在高层次上，GPU 编程模型基于单程序多数据（SPMD）模型；内核函数定义了由组成 GPU 应用的数千个细粒度线程中的每一个所执行的程序。图 1.11 显示所有线程执行相同的内核，但都访问不同的内存位置。

由于编程模型是 SPMD，大多数线程执行相同的工作。因此，一组线程以锁步方式执行，执行相同的指令（在不同的数据上）。这种线程的微架构分组可以影响控制流和内存访问效率，引入了 warp 的概念——一组以锁步方式一起执行的线程。

G80 的执行模型称为 SIMT（单指令多线程）。SIMT 与 SIMD 非常相似，但有细微差别。当程序员编写代码时，他们可以单独对待每个线程。编程模型允许每个单独的线程执行不同的工作。相比之下，在 SIMD 中，向量宽度由 ISA 级别决定，一条指令必须同时处理固定向量宽度的数据。在一个 warp 内，如果某些线程与其他线程走了不同的路径，这些分支称为发散分支。图 1.7 中给出了一个示例代码。

```
if (threadIdx.x>0) {
    // 执行工作
```

```
}
```

(a) if 语句

```
if (threadIdx.x%2) { // 发散分支
    // 执行工作 1
}
else {
    // 执行工作 2
}
```

(b) if-else 语句

图 1.7 发散分支示例：if 语句和 if-else 语句。

1.3.5 CUDA 语言扩展

CUDA 扩展了 C 函数声明语法。表 1.2 总结了这些关键字的使用。通过使用 `__global__`、`__device__` 或 `__host__` 中的一个，CUDA 程序员可以指示编译器生成内核函数、设备函数或主机函数。主机函数只是在主机上执行且只能从另一个主机函数调用的传统 C 函数。默认情况下，如果 CUDA 程序中的函数声明中没有任何 CUDA 关键字，则它们都是主机函数。

表 1.2 CUDA C 函数声明关键字

声明	执行位置	只能从以下位置调用
`__device__ float DeviceFunc()`	设备	设备
`__global__ void KernelFunc()`	设备	主机
`__host__ float HostFunc()`	主机	主机

C/C++ 的另一个重要扩展是使用特定的内置变量来标识线程：`threadIdx`、`blockIdx` 和 `blockDim`。这些变量与预定义的硬件寄存器相关联，提供标识线程的“坐标”。特别地，不同线程将看到这些变量的不同值；此外，这些变量的值可用于为给定线程构造全局唯一的整数线程 ID。例如，此功能允许程序员规定线程到数据的特定映射，如下所示。

1.3.6 向量加法示例

我们展示一个使用向量加法的内核调用示例。首先，我们在图 1.8 中展示一个传统的向量加法 C 程序（$\vec{C} = \vec{A} + \vec{B}$）。实际的向量加法操作由一个 for 循环执行。现在，将相同的代码更改为在设备（GPU）上执行向量加法。图 1.9 仅展示将启动 GPU 代码的

主机端代码。图 1.10 展示设备（GPU）端的代码。

```
// 计算向量和 C=A+B
void vecAdd(float *A, float *B, float *C, int n) {
    for (int ii = 0; ii < n; ii++) C[ii] = A[ii] + B[ii]; }

int main() {
    // 为 h_A、h_B 和 h_C 分配内存
    // I/O 以读取 h_A 和 h_B，每个 N 个元素 ...
    vecAdd(h_A, h_B, h_C, N); }
```

图 1.8　简化的传统 C 语言向量加法代码。

```
# include <cuda.h>
...
void vecAdd(float *A, float *B, float *C, int n) {
    // 为 h_A、h_B 和 h_C 分配内存
    // I/O 以读取 h_A 和 h_B，每个 N 个元素

    ...
    // 1. 为 A、B 和 C 分配设备内存
    // 将 A 和 B 复制到设备内存

    // 2. 内核启动代码
    // 设备执行实际的向量加法

    // 3. 从设备内存复制 C // 释放设备内存
}
```

图 1.9　CPU 端调用 GPU 内核的 vecAdd 函数。

```
// 每个线程执行一次成对加法
__global__ void vecAddKernel(float *A, float *B, float *C, int n) {
    int idx = threadIdx.x + blockDim.x * blockIdx.x;
    C[idx] = A[idx] + B[idx]; }
```

图 1.10　GPU 端的向量加法内核函数。

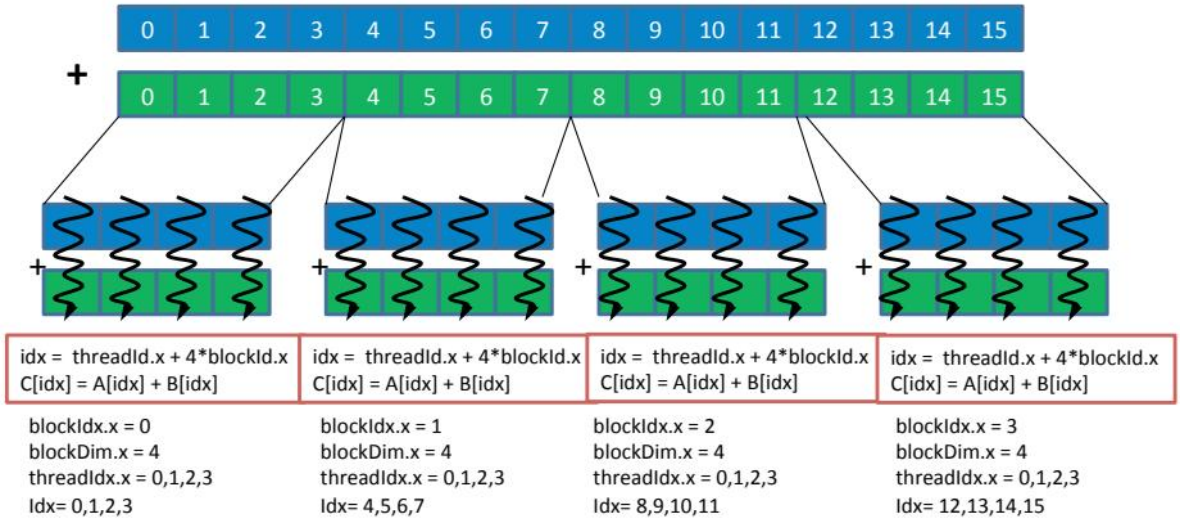


图 1.11 网格中的所有线程执行相同的内核代码，但访问不同的内存位置。（此处，blockDim.x 的值为 4。）

设备和全局内存 在 CUDA 中，主机和设备具有独立的存储空间。[2] 实际硬件也具有独立的物理内存，如图 1.1 所示：CPU 拥有自己的 DRAM，GPU 也拥有自己的 DRAM。GPU 的 DRAM 存储空间称为全局内存或设备内存。为了在设备上执行内核，程序员需要在设备上分配全局内存，并将相关数据从主机内存传输到分配的设备内存。这对应于图 1.9 中的第 1 部分。类似地，内核执行后，程序员需要将结果数据从设备内存传输回主机内存，并释放不再需要的设备内存（图 1.9 中的第 3 部分）。CUDA 运行时系统提供了用于管理设备内存中数据的 API 函数。例如，图 1.9 中 vecAdd（）函数的第 1 部分和第 3 部分需要使用这些 API 函数来分配设备内存、在设备/主机之间传输数据以及释放设备内存。

内存数据索引 在 SPMD 编程模型中，单个程序处理多个数据（SIMD 或 SIMT）。每个线程操作不同的数据。它要么访问自己的寄存器文件，要么访问不同的内存位置。在访问内存时，程序员或算法设计者通常期望线程到数据的特定分配。为了指定这种映射，CUDA 提供了用于标识线程的内置变量，如上所述。图 1.11 以向量加法为例说明了这一概念。这里，假设我们希望添加两个长度为 16 的向量，并且希望分配 1 个线程来执行每个加法。此外，假设我们创建一个包含 16 个线程的池，组织成 4 个线程块，每个线程块有 4 个线程。每个线程可以使用内置变量 blockIdx.x 发现其唯一的线程块 ID，以及使用变量 threadIdx.x 在其块内的唯一线程 ID，如图 1.11 所示。由此，我们可以轻松地将顺序代码中的循环迭代变量转换为线程代码中使用的索引。还要注意.x 字段表示法；碰巧还有.y 和.z 字段，当组织线程的最自然方式是二维或三维时（如图像和图形应用中常见的那样），可以使用这些字段。

[2]在 CUDA 4.0 中，提供了统一内存空间，但该功能的实现尚未完成。

1.3.7　PTX

PTX 是 NVIDIA GPU 架构使用的虚拟 ISA。编译器将 PTX 代码转换为给定 GPU 架构的原生 ISA。寄存器分配和基于特定架构的优化在从 PTX 生成原生二进制代码的过程中执行。

1.3.8　一致性模型和特殊内存操作

CUDA 编程模型未指定一致性模型。由于执行模型是批量同步的，跨线程的内存操作可以重新排序。人们可以将 CUDA 内存模型视为弱一致性模型。为了强制内存排序，最新版本的 CUDA 编程模型提供了用于栅栏的内置函数，例如 __threadfence（）和 mem_fence（）。这些函数保证在这些函数之前的所有内存请求对所有线程可见。换句话说，__threadfence（）用于暂停当前线程，直到对共享内存和全局内存的所有先前写入对其他线程可见。[3] CUDA 编程模型还提供了几个原子操作，这些操作对于实现锁操作很有用。原子函数的示例包括（atomic add/sub/exchange/inc/dec/min/max）。原子操作和 fence() 函数经常一起使用。例如：

1. 存储数据
2. __threadfence()
3. 原子地标记一个标志

这些步骤保证如果其他块看到该标志，它也将看到数据。[4]

1.3.9　IEEE 浮点支持

早期的 GPU 设计不遵循 IEEE 浮点标准，因为这些标准被认为对图形应用不是必需的。然而，随着 GPGPU 编程的兴起，浮点实现越来越符合 IEEE-754 标准，包括 G80 架构中的单精度支持以及 Fermi 架构中的双精度支持。

1.3.10　OpenCL 的执行模型

尽管到目前为止我们的讨论集中在 CUDA 上，但一个联盟已经组装了一个名为 OpenCL（开放计算语言）的替代开放标准。OpenCL 采用类似 CUDA 的构造，但承诺跨一系列平台的可移植性，包括 NVIDIA GPU、AMD GPU 和来自多个供应商的多核 CPU。用 OpenCL 编写的应用程序在运行时针对特定目标架构进行编译。OpenCL 联盟的目标是开发一个行业范围的标准化并行编程环境，并使得能够在不同类型的设备上运行单个应用程序代码，和/或在运行时从单个应用程序创建内核并将其分派到可用的 OpenCL 兼容计算设备。虽然 CUDA 主要基于细粒度 SPMD（单程序多数据）执行模型构建，线程间通信有限，但 OpenCL 还支持任务并行编程模型。OpenCL API 提

[3] http://stackoverflow.com/questions/11570789/cuda-threadfence

[4] http://stackoverflow.com/questions/5232689/cuda-threadfence/5233737#5233737

供数据结构和例程，以在不同设备上运行的内核之间同步执行和共享数据。
与 CUDA 类似，一个应用程序或程序部分地由若干函数或内核组成，这些函数或内核在 OpenCL 设备上执行。主机代码通过命令队列将内核分配给可用的计算设备。为此，程序员需要在主机端建立 OpenCL 上下文，以处理内存分配、内存对象之间的数据传输以及为设备创建命令队列。

表 1.3 CUDA 与 OpenCL 对比

	OpenCL	**CUDA**
执行模型	工作组/工作项	块/线程
内存模型	全局/常量/本地/私有	全局/常量/共享/本地 + 纹理
内存一致性	弱一致性	弱一致性
同步	使用工作组屏障（工作项之间）	在线程之间调用 __sync_threads()

1.4 GPU 架构

本节概述当前 GPU 架构的设计。大部分讨论涉及 NVIDIA 的 Tesla/Fermi 架构。然而，我们的描述并非旨在直接对应任何现有的工业产品。

1.4.1 GPU 流水线

图 1.12 展示了 GPU 架构流水线的概览。图中显示了一个流多处理器，它基于顺序调度器。与传统架构类似，它具有取指、译码、调度、寄存器读取、执行和写回阶段。

取指与译码阶段 前端与传统多线程架构非常相似。存在多个 PC 寄存器以支持多个 warp。调度器根据调度算法（如轮询或贪婪取指）选择一个 warp 进行取指。轮询策略从就绪 warp 列表中选择一个 warp，给予每个 warp 同等优先级。在贪婪取指策略中，流多处理器从一个 warp 取指，直到发生特定事件（如指令缓存缺失、取指到分支指令或指令缓冲区满）。由于前端有多个 warp 可供取指，当遇到此类事件时，它只需切换到另一个 warp 进行取指。出于同样的原因，分支预测器的作用减弱，通常不予实现。较新的 GPU 在一个周期内执行多个 warp，因此前端可以在同一周期内从不同 warp 取指，而不是一个周期只从一个 warp 取指。
指令取指后，在译码阶段进行译码。流多处理器可以为每个 warp 设置一个指令缓冲区，或为所有 warp 共享一个缓冲区。

调度器与记分板 GPU 处理器具有顺序调度器。在 G80 架构中，它一次只执行一个 warp；后续架构如 Fermi 可调度多个 warp。调度器使用记分板来寻找就绪的 warp。到目前为止，尚无 GPU 架构采用乱序调度器。然而，调度器可以选择任何就绪的 warp。因此，从程序员的角度看，程序可能看起来像乱序执行。例如，图 1.13 中的场景是可能的。

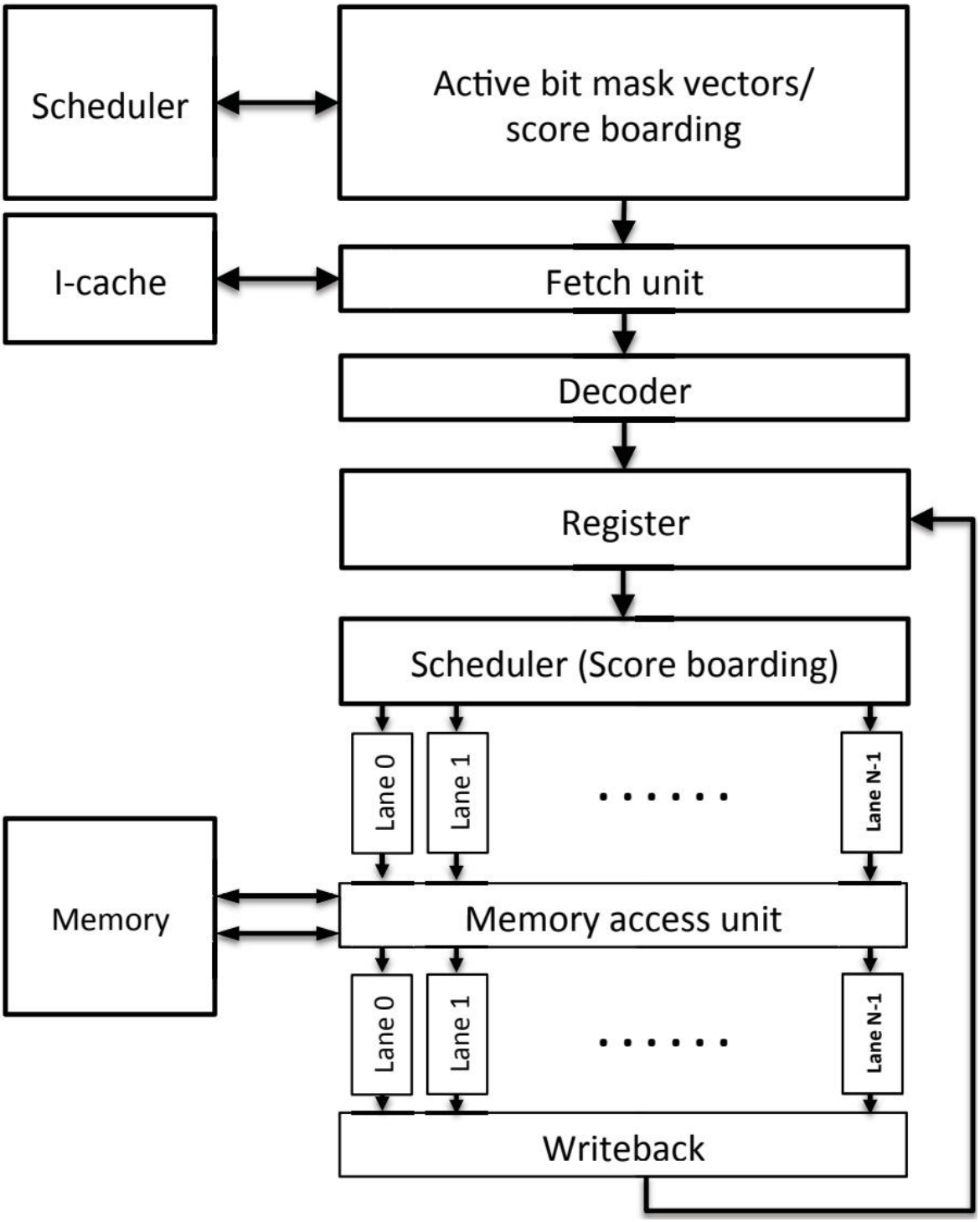


图 1.12　GPU 流多处理器流水线概览。

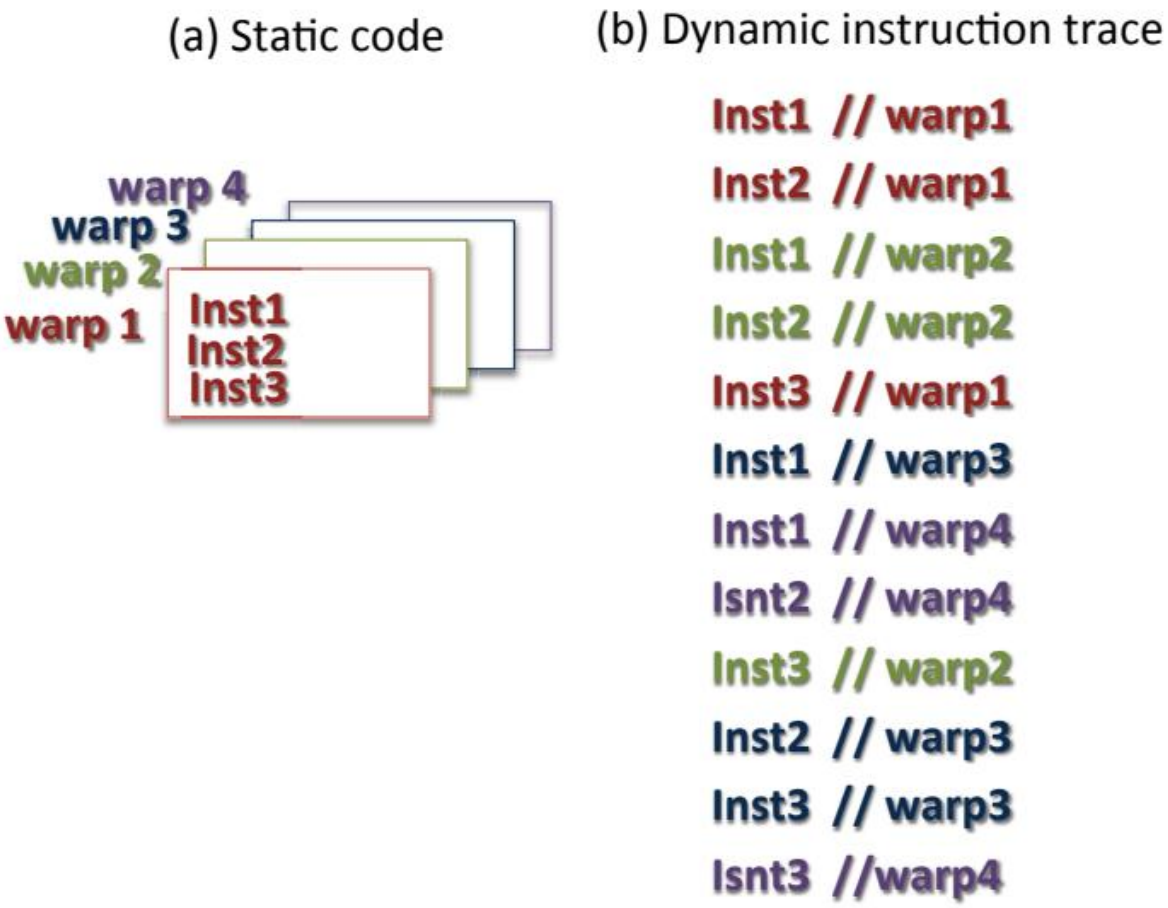


图 1.13 静态和动态指令轨迹示例。

记分板。记分板是实现动态调度的一种方法。它首次引入于 CDC6600 [89]。它检查写后读和写后写数据依赖。记分板不提供寄存器重命名机制，但当没有冲突且硬件可用时，指令可以乱序执行（即指令应按序调度，但可以乱序完成（完成功能单元/内存访问））。在 GPU 中，记分板用于检查任何 RAW 或 WAW 依赖，因此来自
同一 warp 的指令即使先前的指令尚未完成也可以执行。这种方法增加了指令/内存级并行性。

寄存器读/写　为了容纳相对大量的活跃线程，GPU 处理器维护大量的寄存器堆。例如，如果一个处理器支持 128 个线程，每个线程使用 64 个寄存器，总共需要 128×64 个寄存器。因此，G80 具有 64 KB 的寄存器堆，Fermi 每个流多处理器支持 128 KB 的寄存器堆存储（总共 2 MB 寄存器堆）。寄存器堆应具有高容量和高带宽。如果一个 GPU 具有 1 Tflop/s 的峰值性能，并且每个浮点操作至少需要两次寄存器读取和一次寄存器写入，则需要 $2 \times 10^{12} \times 32\ \mathrm{B/s} = 64\ \mathrm{TB/s}$ 的寄存器读取带宽。提供如此高的带宽尤其具有挑战性，因此使用了多种技术，包括多体结构和操作数缓冲/收集器。

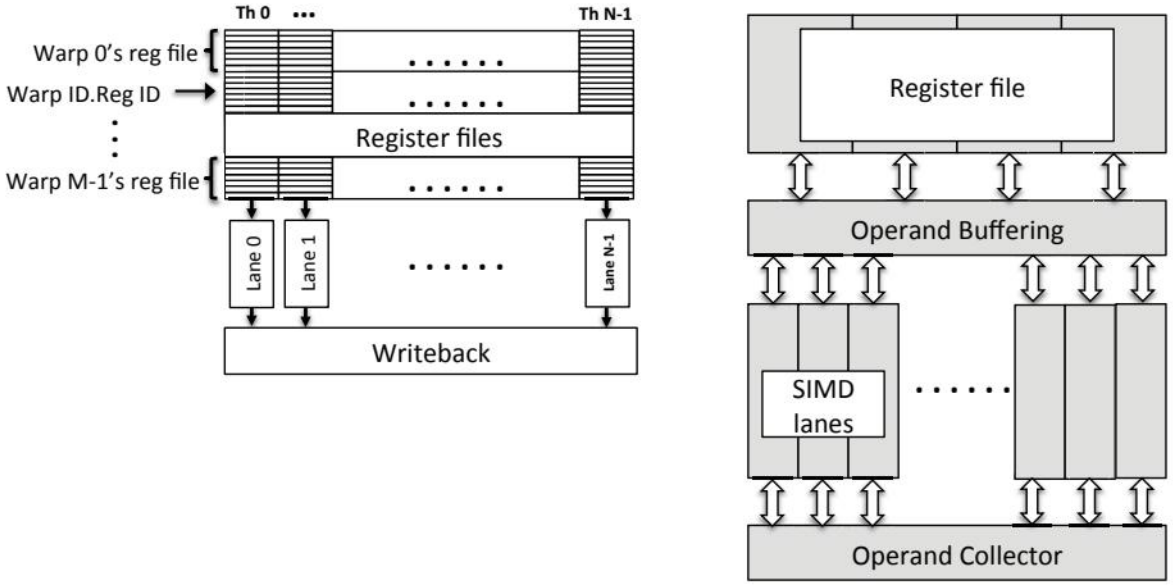


图 1.14　GPU 寄存器堆访问。左：使用多体结构（图由 [70] 提供）；右：使用操作数缓冲（图由 [41] 提供）。

多体结构。流多处理器通过将寄存器堆细分为多个体来提供高带宽。图 1.14 显示了寄存器堆结构。warp 中的所有线程通过由 warp ID 和寄存器 ID 索引的寄存器堆并行读取寄存器值 [70]。然后，这些寄存器值直接馈送到流水线的 SIMD 后端。（请记住，每个 SIMD 单元/通道仅由一个线程使用。）

操作数缓冲。Gebhart 等人 [41] 在图 1.14 中展示了另一种寄存器堆结构示例。在该设计中，寄存器堆和执行单元之间存在一个缓冲区。处理器不是恰好在需要值之前读取所有必要的寄存器值（这会给寄存器堆读/写端口带来非常高的压力），而是可以缓冲寄存器值。

缓冲区可以存储通过多个周期读取的寄存器值，从而降低寄存器读取带宽需求。在他们的设计中，四个 SIMT 通道形成一个簇。流多处理器主寄存器堆中的每个条目宽 128 位，其中 32 位分配给簇中四个 SIMT 通道中每个线程的同名寄存器。若干文献 [13, 57, 63] 表明他们采用了操作数收集器缓冲区。这些操作数收集器也用于聚合结果值。操作数收集器作为结果队列工作，在功能单元的输出写回寄存器堆之前对其进行缓冲。虽然结果队列的主要好处是提高有效写入吞吐量，但它也提供了进一步的优化机会。当指令的结果仅在动态调度上足够接近的指令中使用时，输出值可以转发到下一个操作的输入。这就像 CPU 的转发网络一样 [57]。

当一条指令需要对同一存储体进行多次访问时，寄存器值会在多个周期内被读取。由于寄存器地址是静态确定的，编译器原则上可以减少或消除存储体冲突。

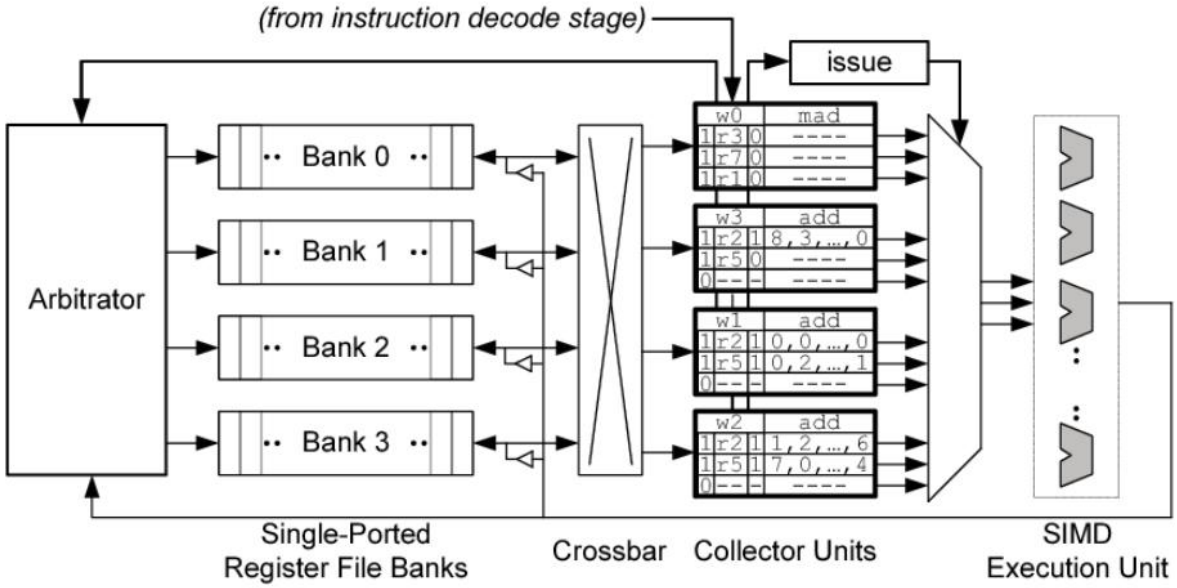


图 1.15 操作数收集器详细示意图 [13]。

寄存器堆缓存。为了减轻寄存器堆的压力，Gebhart 等人提出了一种寄存器堆缓存 [41]。虽然寄存器堆缓存最初是为了减少寄存器堆访问时间而提出的 [29]，但在 GPU 中，寄存器堆缓存被提出来是为了减少寄存器的读取和写入。当前的 GPU 架构不支持精确异常，因此寄存器堆不必维护架构状态。于是，许多写操作可以很容易地在寄存器堆缓存内部合并，这可能会显著减少寄存器写入 [70]。

执行阶段 在执行阶段，指令会访问内存单元或功能单元。内存系统将在 1.4.3 节讨论。

执行阶段由向量处理单元组成（AMD GPU 还有一个标量单元）。一个向量通道执行一个线程，在 NVIDIA 的术语中称为流处理器。最简单的设计是让通道数和 warp 大小相同。然而，这种方法也可能需要相对较大的静态功耗和较大的面积。相反，GPU 会在多个周期内执行同一条指令。例如，G80 只有 8 个向量通道，因此流多处理器需要四个周期来执行 32 个线程（即一个 warp）。计算结果被临时存储，然后一起写回寄

存器堆。

背靠背操作。GPU 的执行宽度很宽（32 个线程），这使得实现数据转发路径变得更加困难。例如，32×32 B（总共 1 KB）或 32×64 B（总共 2 KB）的宽度相当大。因此，结果要么直接写入寄存器堆，要么临时存储在一个小缓冲区中，然后在多个周期内写回寄存器堆。无论哪种情况，结果在执行后都不能立即供相关指令使用。这种情况与许多现代 CPU 广泛使用数据转发的情况大不相同。当存在转发路径时，寄存器读/写周期不在关键路径上，任何背靠背操作都会在执行延迟之后立即执行。然而，GPU 可以通过利用线程级并行（TLP）来避免这种惩罚。流水线只是调度来自其他 warp 的指令，从而隐藏延迟。当有足够的 TLP 时，执行和寄存器写入延迟可以被隐藏。

Volkov 在 2010 年 GTC 演讲中讨论了这个问题 [95]。当有四个线程时，流多处理器可以简单地切换到其他线程，从而隐藏执行延迟。然而，当只有一个线程时，这些背靠背操作需要更长的时间。这个性能问题也在第 4.1 节中讨论。

特殊功能单元。迄今为止，NVIDIA 架构采用了 SIMD 执行单元，AMD 架构使用了 VLIW 架构。此外，特殊功能单元在 NVIDIA GPU 架构中提供了类似 VLIW 的效果。图形应用程序通常需要超越函数来实现几何旋转和缩放等算法。通常可以使用精度不超高的相应超越函数实现。因此，许多 GPU 架构在基于硬件的特殊功能单元（SFU）中提供了快速、近似的实现。由于许多程序并不总是需要这些 SFU，处理器除了常规 FP 单元外只有两到四个 SFU。如果 SFU 指令和常规 FP 指令是独立的，例如当指令来自不同的 warp 时，调度器可以同时发射它们。在这种情况下，这些 SFU 单元提供了额外的执行带宽。第 4.1 节在实际计算中更详细地讨论了这一优势。

1.4.2　处理分支指令

分支在 GPU 中特别难以实现好。回想一下，当前的 GPU 硬件实现为每个 warp 只取一条指令。当这条指令是发散分支时，一个 warp 内的线程将需要执行不同的指令路径。这一挑战在 GPU 和早期的向量处理器中很常见。

向量处理器中的解决方案是使用向量通道掩码或谓词化 [17, 81]。然而，GPU 中的问题略有不同。不仅 SIMD 单元的一部分需要谓词化，而且处理器还需要从不同路径取指令。实际上，发散分支的存在是 SIMD 和 SIMT 执行之间的主要区别。Fung 等人首次在 GPGPU 中描述了这个问题 [38]。

图 1.16 说明了这个问题。假设 warp 大小为 4，2 个线程走 taken 路径，而其余 2 个线程走 fall-through 路径。流多处理器必须逐条取两条路径。活动掩码指示哪些线程参与每条路径。活动掩码位信息也用于任何寄存器读取或写入。

```
if (threadIdx.x < 2) {
    // do work 1
}
else {
    // do work 2
```

```
}
```

图 1.16 发散分支示例。

最朴素的解决方案是 SIMD 串行化。当在给定 PC 处发生发散时，处理器将 warp 内的线程串行化。这将导致性能下降。

提出了一种基于栈的重汇聚解决方案来处理发散分支 [38, 39, 62, 100]。在他们的方案中，发散线程以非锁步方式执行，直到程序结束；处理器检测线程可以再次以锁步方式重新汇合的点，即它们可以再次形成一个 warp 的点。控制流重汇聚点是所有线程合并的点。当程序发散时，处理器将重汇聚点插入栈中。当一条路径中的线程到达重汇聚点时，处理器从其他剩余路径取指令。一旦所有发散路径都被取完，换句话说，一旦所有线程到达它们的重汇聚点，发散线程就合并了。

在基于栈的重收敛系统中，流多处理器使用硬件栈结构来记录汇合位置和下一个取指地址。当流多处理器取到一条导致分歧的分支指令时，它首先将重收敛点（控制流汇合点）的 PC 地址压入栈中。然后，流多处理器先取一条路径。由于每个 warp 只有一个 PC，流多处理器必须将另一条路径的 PC 地址也存入栈中。编译器识别控制流汇合点并将该信息包含在内。

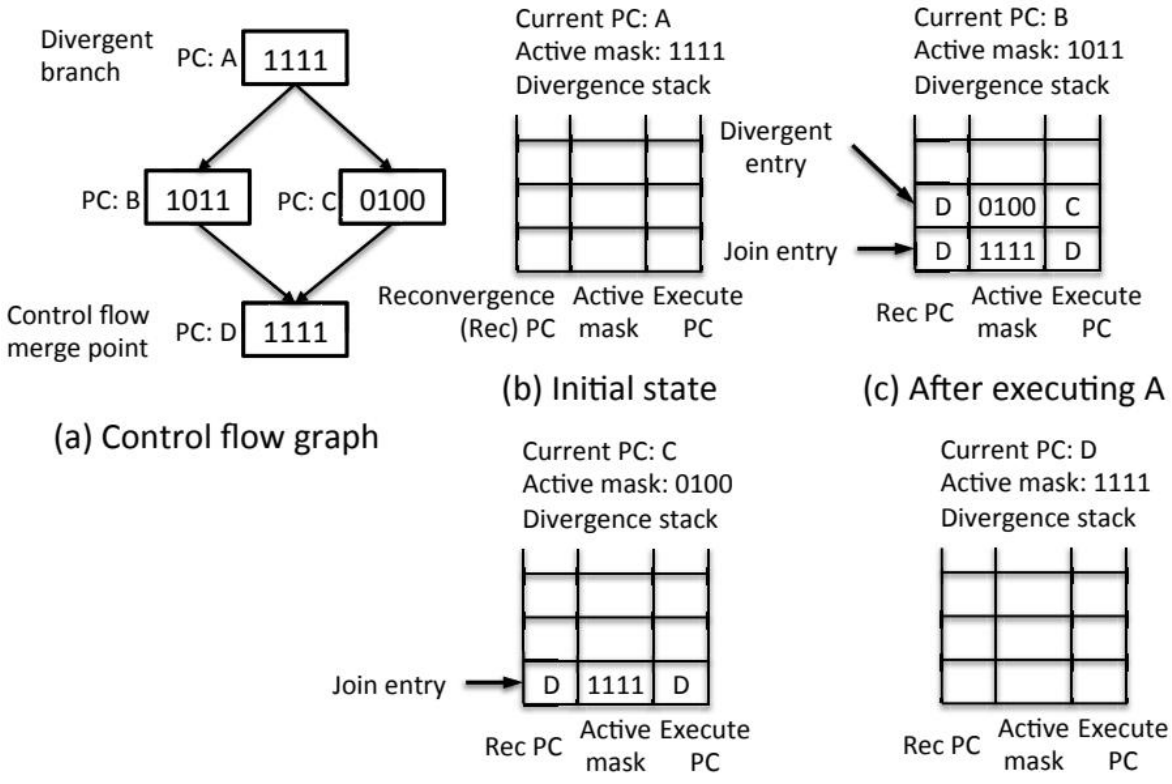


图 1.17 基于栈的发散分支处理机制（引自文献 [38,70]）

（d）执行 B 之后

每个栈条目有三个字段：重收敛点的 PC 地址、活动掩码位和下一个 PC 地址。图 1.17 展示了该过程 [38，70]。图 1.17（a）显示了一个控制流图。图 1.17（b）显示了栈的初

始状态。当处理器取到一条导致分歧的分支指令时，它会在栈中压入一个汇合条目。然后，处理器选择其中一条路径，并将另一条路径的信息压入栈中。活动掩码字段被设置为当前的活动掩码。（图 1.17（c）显示了执行分支 A 之后的栈。）栈顶显示了该路径的汇合条目和活动掩码。当处理器取指令时，它总是检查下一个 PC 地址是否与重收敛 PC 匹配。在取到 PC B 之后，下一个 PC 地址是 D，这与栈顶的值相同。因此，处理器从栈中弹出顶部条目，并开始从 PC C 取指，并使用该条目中的活动掩码。（图 1.17（d）显示了执行 PC B 之后的栈状态。）由于下一个 PC 地址再次是 D，栈顶条目被弹出。活动掩码全为 1，所以现在所有线程都参与该路径。（图 1.17（e）显示了执行 PC C 之后的状态）。

在硬件中减少分歧分支影响的解决方案。已经提出了几种解决方案来处理分歧分支问题。为了利用 warp 中的空闲线程，Fung 等人提出了一种动态 warp 形成方法 [38，39]。在运行时，硬件通过组合来自同一路径的线程来形成新的 warp。Meng 等人提出了动态 warp 细分 [69]。动态 warp 细分不仅处理分歧分支，还针对分歧内存操作。

在分歧内存操作中，一些线程命中缓存，而另一些则没有，因此线程在一个 warp 内产生不同的内存延迟。还提出了其他几种解决方案，例如动态改变执行单元，如线程压缩 [40]、大 warp 形成 [70]、基于收益预测的压缩 [79] 以及动态时交织线程 [20，32]。

在软件中减少分歧分支影响的解决方案。也存在一些基于编译器的解决方案。Zhang 等人通过重新映射数据位置来减少分歧分支 [105]。Han 和 Adelrahman 提出了编译器解决方案 [46]。Diamos 等人根据静态时的频率重新排列线程 [32]。他们的解决方案也可以减少分歧分支。

1.4.3　GPU 内存系统

GPU 内存系统具有多个层次的层级结构。由于可能有大量内存请求同时在途，系统需要几个大型缓冲区和/或队列。

流多处理器具有缓存以提供快速数据访问。在早期的 GPGPU 架构（如 G80）中，只有软件管理的一级缓存。较新的 GPGPU 架构采用了更积极的缓存层次结构。在 Fermi 中，一级缓存只能由软件（程序）控制，或以软硬件混合管理的方式控制。如果程序控制软件管理的缓存，指令会显式地将数据带入缓存并将其逐出。CUDA 中的共享内存空间由软件显式控制。由于内存地址在运行时确定，与寄存器文件不同，缓存中可能存在 bank 冲突。

表 1.4　内存空间与典型访问延迟

内存空间	Tesla	Fermi	访问延迟
本地内存	DRAM	DRAM 和硬件缓存	数百个周期
共享内存	软件缓存	软件缓存	4 – 32 个周期
全局内存	DRAM	DRAM 和硬件缓存	数百个周期
常量内存	DRAM 和常量缓存	DRAM 和常量缓存	数百个周期或 4 个周期（缓存命中）

纹理内存	DRAM 和纹理缓存	DRAM 和纹理缓存	数百个周期或 4 个周期（缓存命中）

表 1.4 显示了不同的内存空间和典型的内存延迟。到目前为止，计算机架构师提出了内存系统中的两个主要问题。一是管理大尺寸的缓冲区，二是处理长内存延迟。

DRAM 系统。 GPU 使用专为高带宽 DRAM 设计的 GDDR。它增加了内存系统的带宽，但 DRAM 延迟稍慢。GDDR 具有大量
DRAM bank 以进一步增加带宽。通常，GDDR 提供比 DDR 更高的 DRAM 带宽，因为其突发长度更宽，且 DRAM 直接连接到处理器。在 CPU 中，DRAM 通过 DIMM 连接，这降低了通信带宽。DRAM 调度算法经过调整以提供高带宽。FRFCFS 是提高吞吐量导向计算的最佳方式。由于 GPGPU 应用具有高空间局部性，FRFCFS 表现出比 FCFS 高得多的性能 [103]。

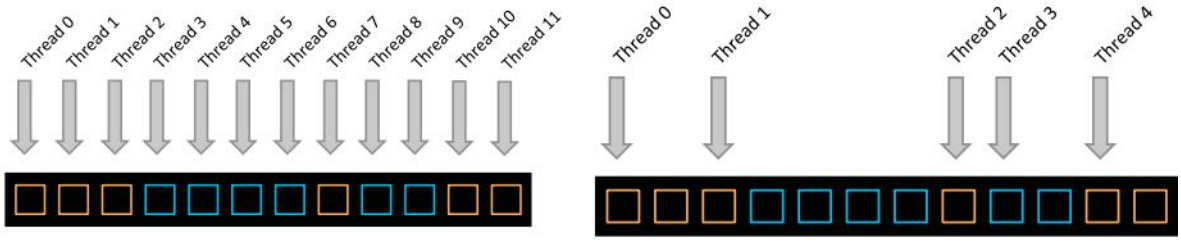


图 1.18 合并与非合并内存请求。左：所有线程访问连续的内存地址（合并）；右：线程访问非连续的内存地址（非合并）：由 [72] 提供。

多内存事务。 SIMD 执行单元的基础是一次执行多个数据。如果数据在寄存器中，所有寄存器文件可以一起访问。当 warp 执行内存操作时，挑战就出现了。内存地址可以在任何地方。在早期的 GPU 中，这是最重要的性能优化之一 [82，83，96]。当 warp 中的所有线程访问连续的内存地址时，这些内存地址可以很容易地用少量事务表示。然而，如果内存地址都是分散的，这些内存操作将生成多个事务。在早期的 GPU 中，特别是早于 CUDA 计算版本 1.2 的 GPU 中，这些被称为合并和非合并内存地址，如图 1.18 所示。合并内存地址根据事务大小生成 1 到 4 个内存事务，而非合并内存地址最多生成 32 个事务。经过一些硬件优化后，硬件尽可能多地组合内存地址并生成更少的事务。合并用于尽可能多地组合请求。减少内存事务的数量仍然是最基本的性能优化之一。硬件通常有一个特殊的硬件单元来处理这种分散/聚集操作。

多个在途内存请求。自然地，GPU 处理器可以具有较高的 MLP，这与 TLP 相同。为了进一步提高 MLP，GPU 处理器采用了在依赖指令处停顿的策略。尽管 GPU 采用顺序处理器，但缓存缺失并不会阻止同一线程中指令的执行。处理器仍然可以执行同一 warp 中的指令，直到依赖于缓存缺失的指令使该 warp 停顿。图 1.19 说明了这一行为。这提供了一个在 warp 内利用 MLP 的小窗口。通过这种方式，可以增加在途内存请求的数量。当程序具有高 TLP 时，这不会产生太大影响。然而，当线程较少时，利用 MLP 效应可以显著增加总 MLP。

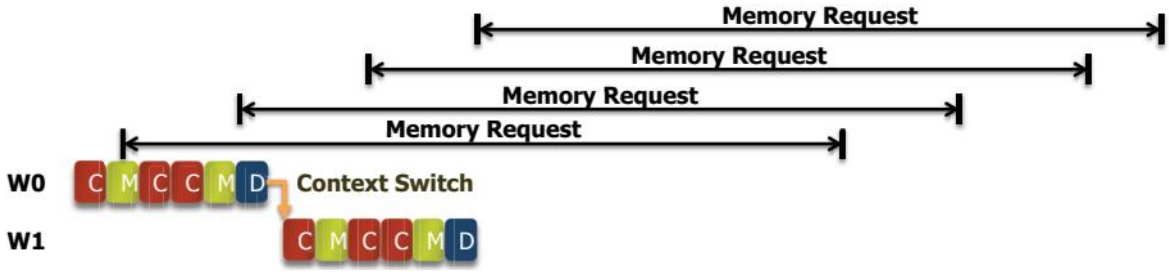


图 1.19　多个在途内存请求（即使只有 2 个 warp 在运行，也有 4 个内存请求被并发处理）。

1.5　其他 GPU 架构

1.5.1　Fermi 架构

G80 设计的主要后继产品基于 Fermi 架构，该架构于 2010 年首次发布。Fermi 架构在 GPU 架构设计上实现了重大飞跃，提供了两级缓存层次结构，以更好地支持那些无法高效使用 GPU 共享内存的应用程序。其他重大改进包括提升的双精度性能、更快的原子操作以降低线程块间通信成本，以及对纠错码（ECC）的支持。关于 Fermi 架构硬件和软件特性的更详细描述可在 [51] 中找到。

1.5.2　AMD 架构

ATI 流技术（由所有现代 AMD GPU 如 ATI Radeon 和 FireStream 支持）也提供了大规模并行计算。ATI 最初支持 Brook+ [3] 以提供 GPGPU 功能，但现在 ATI 转而支持 OpenCL。迄今为止，NVIDIA 的 GPU 架构与 AMD 的 GPU 架构之间的主要区别在于 AMD 采用 SIMD-VLIW 架构。四个 ALU 和一个分支单元或五个 ALU 和一个分支单元打包在一起，所有这些单元执行相同的指令。AMD 可以轻松支持标量指令作为 VLIW 单元之一，而 NVIDIA 迄今为止只有向量单元。

近来，AMD 架构的最大优势是融合 [6] 架构，它将 CPU 和 GPU 结合在同一芯片上。CPU 和 GPU 都被集成并共享相同的内存系统。AMD 的融合架构是第一个真正支持异构计算系统的 OpenCL 计算平台。共享相同的内存系统极大地降低了 CPU 和 GPU 之间的通信成本。然而，到目前为止，只有低端

端 GPU 被集成在融合架构中，因此 AMD 的融合架构尚未广泛用于 GPGPU 计算。尽管如此，我们预测 AMD 的架构很快将被广泛用作 GPGPU 计算平台。

1.5.3　集成众核架构（MIC）

英特尔的集成众核（MIC）架构也面向高吞吐量计算处理器 [26]。与 AMD 和 NVIDIA 不同，英特尔的 MIC 架构执行原生 x86 ISA，这是英特尔平台的最大优势。MIC 还具有宽 SIMD 单元（512 位宽）以实现高吞吐量计算，但它也具有传统 CPU 架构中的大多数特性，如虚拟内存、完全一致的缓存和分支预测器。然而，并发运行的线程数远小于 NVIDIA 或 AMD（4 个）。因此，MIC 不仅专门针对吞吐量导向的计算，还面向

延迟受限的应用。

1.5.4 在同一芯片上集成 CPU 与 GPU

英特尔的 Sandybridge 和 AMD 的 Llano 都将 CPU 和 GPU 放在同一芯片上。到目前为止，GPU 性能远低于独立 GPU。功耗和内存带宽可能是在同一芯片上结合强大 GPU 和 CPU 的主要原因。结合 CPU 和 GPU 引入了若干资源管理问题，例如关于缓存分区 [61] 和 DRAM 调度策略 [10，55] 的资源管理研究。

第二章　性能原则

为 GPGPU 开发算法，本质上就是应用与其他共享内存并行平台相关的、长期研究的并行化和 I/O 高效设计原则。本章回顾这些原则，重点关注并行算法在理论和实践方面的最新成果，并建议与 GPGPU 平台的联系。理想情况下，应用这些原则并配以正确的成本模型，不仅能得到可证明高效的算法，还能为架构师提供关于可能对给定计算性能影响最大的特性和配置的提示。因此，我们相信这一讨论将对并行计算各个方面的实践者有用，而不仅仅是那些特别关注 GPGPU 的人。

2.1　理论：算法设计模型概述

任何算法可能决定其性能的两个最重要特征是：（a）有多少可用并行性，以及（b）必须通过内存层次结构移动多少数据。因此，在设计算法时，我们希望有一个抽象机器模型，允许我们沿这些维度评估算法。在这样的模型中，我们可能希望进行与顺序情况下习惯的相同的“大 O”分析。这样做将使我们能够在进入较低层次的性能优化和调优之前，先正确把握高层算法设计。重要的是，抽象不应过于繁琐，以至于我们无法在合理的时间内设计和分析候选算法。

尽管算法设计模型的现状仍在变化，但我们有合理的选择。对于 GPGPU 和其他众核风格处理器，两个合适的模型是用于分析并行性的所谓工作-深度（或工作-跨度）模型 [15，28，54]，以及用于分析存在内存层次结构时的 I/O 行为的外部内存模型 [4，93，94]。这两个模型是独立发展的，但最近的工作显示了将它们联系起来的方法 [16，30]。本章的其余部分回顾了这项工作的一些方面。

2.2　刻画并行性：工作-深度模型

在工作-深度模型中，我们用一个有向无环图（DAG）来表示计算，其中边表示依赖关系，如图 2.1 所示。给定该 DAG，我们度量其工作（work）$W(n)$，即对于规模为 n 的输入的总单位代价操作数，以及其深度（depth）或跨度（span）$D(n)$，即同样以单位代价操作数度量的关键路径长度。

$D(n)$ 应当是的最小执行时间的下界，而比值 $W(n)/D(n)$ 有效地度量了每个关键路径节点执行时的平均可用并行量。事实上，比值 $D(n)/W(n)$ 类似于顺序比例的概念，正如人们在评估 Amdahl 定律时可能使用的那样 [7，47，99]。因此，我们的隐含目标是最大化 $W(n)/D(n)$，或者等价地，最小化 $D(n)/W(n)$。重要的是，该模型没有明确引用处理器数量。在这个意义上，我们可以将该模型视为与机器无关。然而，如果我们想知道为一个算法投入多少处理器，$W(n)/D(n)$ 是一个合适的指南。

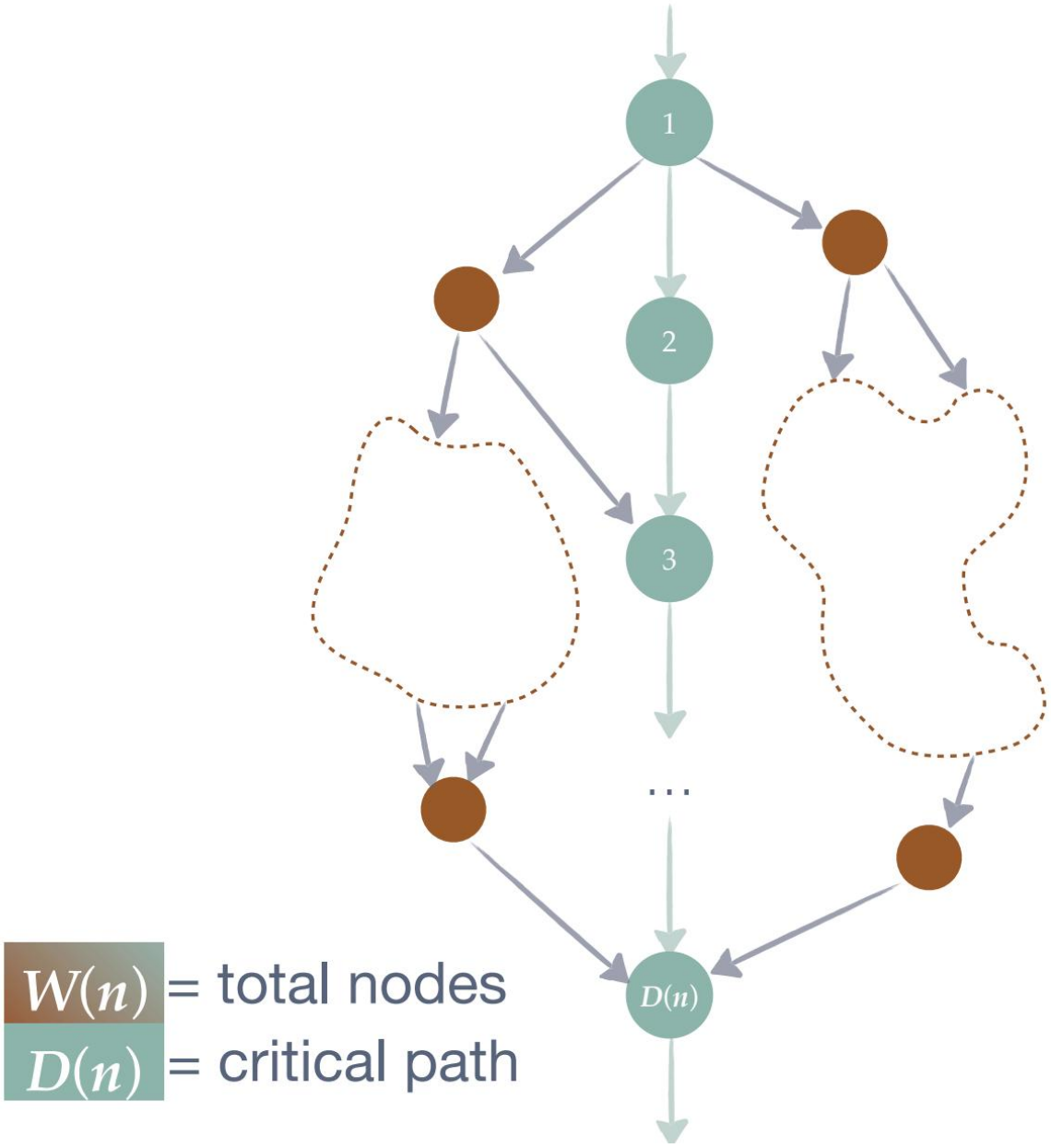


图 2.1 工作-深度（或工作-跨度）模型中的并行计算。对于规模为 n 的输入，计算是一个有向无环图，具有 $W(n)$ 个节点，每个节点代表一个单位代价操作；边表示这些操作之间的严格依赖关系；关键路径长度为 $D(n)$ 个节点。我们的目标是设计出既能达到工作最优性又能最大化平均可用并行度 $W(n)/D(n)$ 的算法。

然而，很容易最大化 $W(n)/D(n)$ 却仍然得到一个糟糕的算法。例如，我们可以人为地夸大总操作数 $W(n)$。因此，我们还应该努力确保我们的算法是工作最优的，这一性质表明 $W(n)$ 在渐近意义上不差于最佳顺序算法。确实，工作最优性是一个关键要求。

让我们看看为什么工作最优性很重要。首先，注意根据 Brent [19] 的一个定理，可以（粗略地）估计给定 p 个相同处理器时算法的运行时间。该定理表明，如果 DAG 的节点具有单位代价且机器有 p 个处理器，那么可以调度该 DAG，使得执行（计算）该 DAG 的时间 $T_{\text{comp}}(n;p)$ 为

$$T_{\text{comp}}(n;p) = D(n) + \frac{W(n) - D(n)}{p}. \tag{2.1}$$

假设我们设计了一个高度并行的算法，满足 $W(n) \gg p \cdot D(n)$，并且我们的算法仅比最佳顺序算法的工作量 $W_0(n)$ 多出一个因子 $\epsilon(n)$，即 $W(n) = W_0(n) \cdot \epsilon(n)$。那么，我们的算法在 p 个处理器上的加速比大致为 $W_0(n)/(W(n)/p) = p/\epsilon(n)$。也就是说，p 的最佳可能加速比将被 $\epsilon(n)$ 降低。考虑相对较小的因子 $\epsilon(n) = \log n$。即使对于 n = 1024，$\log n = 10$，意味着最佳可能加速比比我们期望的低一个数量级。换句话说，即使仅仅匹配顺序算法，我们也需要 $p > \epsilon(n)$。因此，如果我们在一个 $O(n)$ 顺序算法和一个 $O(n^2/p)$ 并行算法之间做决定，我们将需要 p = n 才能匹配顺序情况。这些例子强调了工作最优算法的重要性。

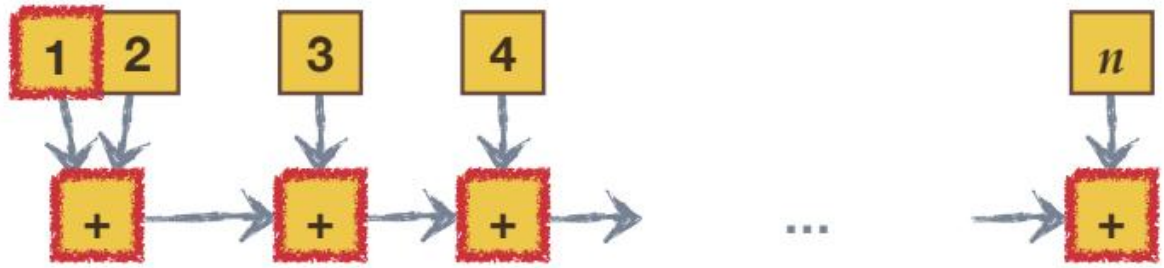
示例：归约。假设我们希望计算 n 个值的和。顺序算法将导致如图 2.2 所示的 DAG。在这种情况下，我们执行 $\Theta(n)$ 次操作，但关键路径长度也是 $\Theta(n)$。因此，可用并行度——即 $W(n)/D(n)$ 的比值——是一个常数：无论输入多大，并发量永远不会超过一个固定值。

具有更多并行性的算法可能具有如图 2.3 所示的 DAG。该算法将加法组织成一棵树，其中独立的子树可以并行执行。这棵树仍然执行 $W(n) = \Theta(n)$ 次总操作，因此是工作最优的。然而，这个 DAG 的深度仅为 $D(n) = \Theta(\log n)$，即树的高度。因此，随着 n 的增长，平均可用并行度也增长，其量为 $W(n)/D(n) = \Theta(n/\log n)$。按照这个度量，该算法比顺序算法更好，正如我们所期望的那样。

在图 2.3 的具体情况下,其中 n = 16,工作量为 $W(16) = 31$ 个节点,深度为 $D(16) = 5$。(想象图 2.3 顶部的“输入”节点代表加载操作以
获取这些值。）那么，$W(16)/D(16) = 31/5 = 6.2$。因此，在计算过程中平均有约 6 个节点可供并行执行。这意味着我们无法有效地使用超过大约六个“处理器”。[1]

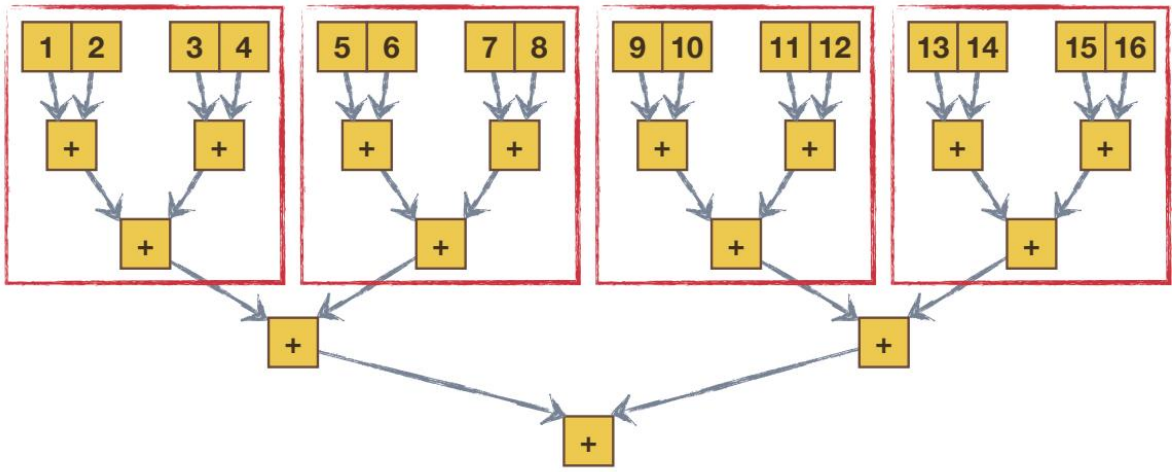
示例：矩阵乘法。考虑两个 $n \times n$ 矩阵的乘法，$C \leftarrow A \times B$，如图 2.4 所示。每个输出元素 c_{ij} 是 $A_{i,j}$（即 A 的第 i 行）与 $B_{:,j}$（即 B 的第 j 列）之间的点积。点积涉及向量的逐元素乘法，然后对这些结果进行和归约。这个点积需要 $\Theta(n)$ 次操作。由于 C 有 n^2 个元素，总工作量 $W(n) = \Theta(n^3)$。[2] 为了计算 $D(n)$，观察到 n^2 个输出元素之间没有依赖关系；唯一的依赖关系发生在计算每个输出元素的归约过程中。因此，

[1] 在这种情况下，我们实际上指的是六个加法器。



$$W(n) = \Theta(n)$$
$$D(n) = \Theta(n)$$

图 2.2 工作-深度示例：用顺序算法对 n 个元素的列表求和。工作量和深度均为 $\Theta(n)$，意味着可用并行度 $W(n)/D(n)$ 是一个常数。



$$W(n) = \mathcal{O}(n)$$
$$D(n) = \mathcal{O}(\log n)$$

图 2.3 工作-深度示例：用树形算法对 $n = 16$ 个元素的列表求和。

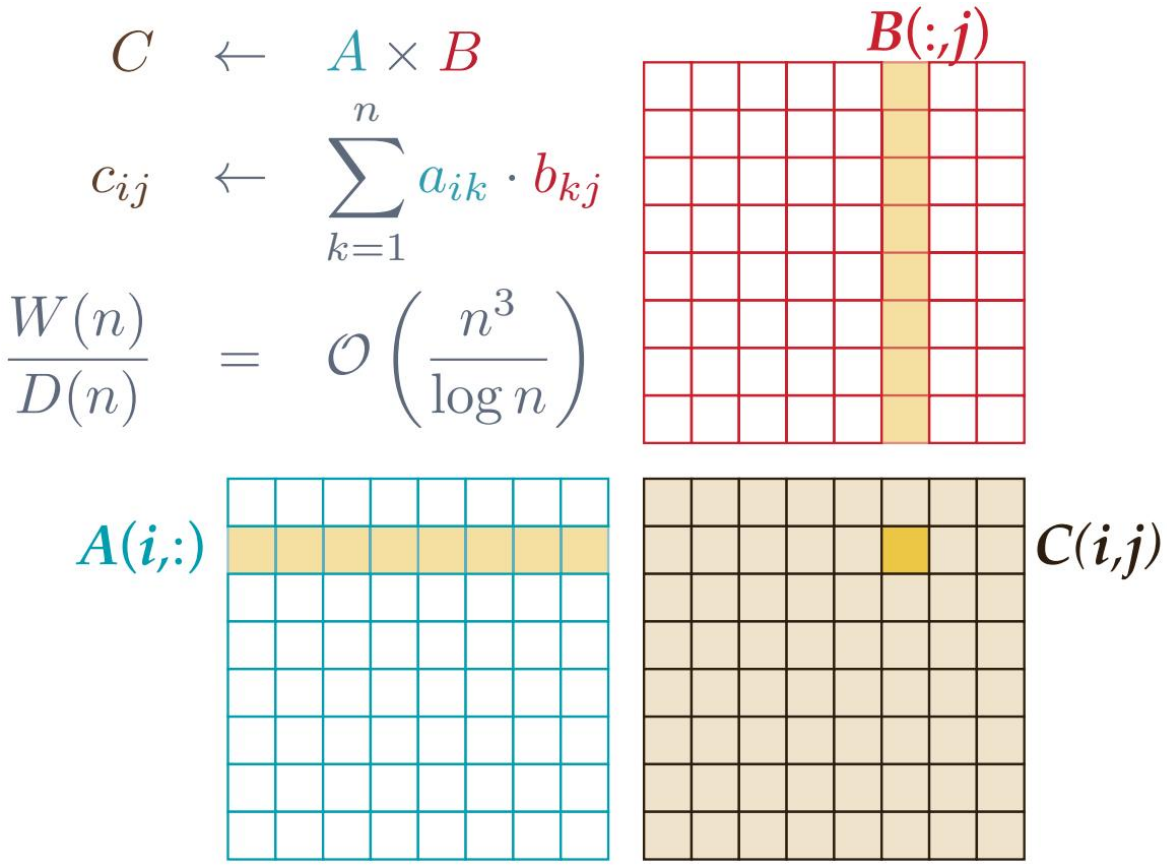


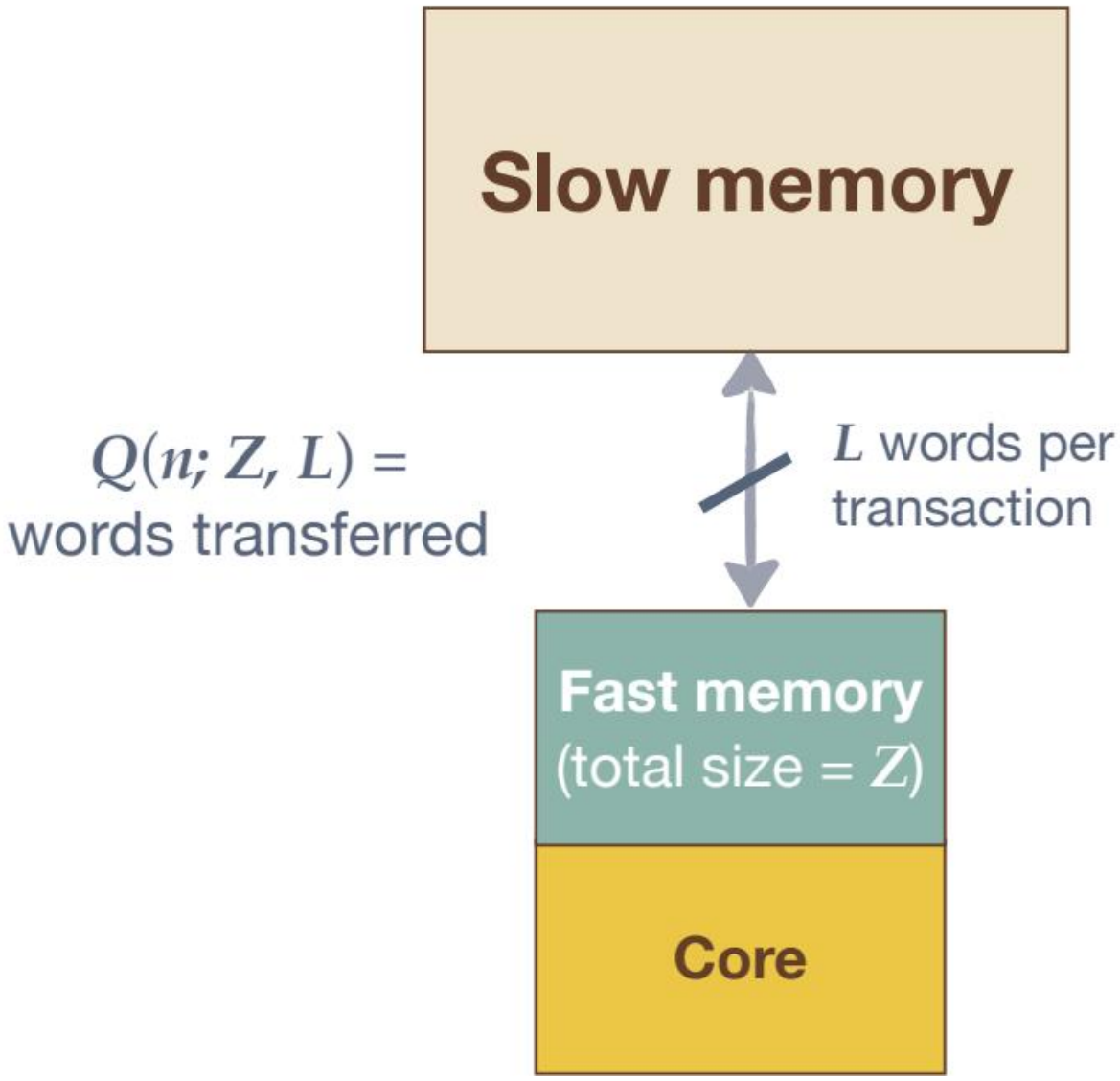
图 2.4　工作-深度示例：两个 $n \times n$ 矩阵相乘。

$D(n) = \Theta(\log n)$。比值 $W(n)/D(n) = \Theta(n^3/\log n)$ 在渐近意义上非常高，因此我们的分析证实了我们的预期，即即使对于中等大小的 n 值，矩阵乘法也具有充足的并行性。

2.3　刻画 I/O 行为：外部内存模型

除了并行性之外，数据移动是算法的另一个关键特性。为了分析数据移动，我们考虑经典的外部内存模型 [4]。

首先考虑一个具有两级内存层次的顺序处理器，该层次由一个大的但慢速的内存和一个小的快速内存组成，快速内存的大小为 Z 个字；工作操作只能对位于快速内存中的数据进行。这个快速内存可以是自动缓存或软件控制的暂存器；我们这里的分析对此选择不敏感。我们可以进一步考虑慢速内存和快速内存之间的传输必须以大小为 L 个字的离散事务（块）进行。当我们在该模型中设计算法时，我们再次测量工作量 $W(n)$；然而，我们也测量 $Q(n; Z, L)$，即对于大小为 n 的输入，慢速内存和快速内存之间 L 大小的传输次数。有几种方法可以设计缓存感知或缓存无关的算法，然后分析 $Q(n; Z, L)$[4，36，56，102]。无论哪种情况，当我们在该模型中设计算法时，我们再次追求工作最优性，同时尝试最大化算法的计算强度，即 $W(n)/(Q(n; Z, L) \cdot L)$。强度是执行的操作数与传输的字数之比，在实践中，通常被转换并引用为算法的“浮点操作与字节比”[97]。



图中 Slow memory 为慢速内存， Fast memory 为快速内存； latency 和 bandwidth 分别为延迟与带宽。

图 2.5 一个具有两级内存层次的抽象机器。快速内存（例如缓存或本地存储）可以容纳 Z 个字。我们的目标是设计算法，使得在给定大小为 n 的输入时，算法既是工作最优的，又能最小化慢速内存和快速内存之间的传输次数 $Q(n; Z)$（或者，算法是工作最优的并且最大化计算强度，$W(n)/Q(n; Z)$）。

示例：归约。让我们再次考虑求和归约问题（图 2.3），但这次分析其 I/O 行为。为简单起见，假设 n、Z 和 L 都是 2 的幂，且 n > Z > L。一个具有 Z 个输入节点的归约树，其中输入节点连续存储在慢速内存中，需要恰好 Z/L 次传输和 Z - 1 次操作（加法）来求值。这些只是加载输入节点的传输，之后求和元素不需要进一步的 I/O。任何 n 个元素的列表可以分解为 n/Z 个部分归约树，从而产生 $Q(n; Z, L) = n/L$ 次传输和 $W(n) = n - Z$ 次加法。也就是说，我们只需要执行强制加载，强度为 $1 - Z/n = \Theta(1)$。就架构而言，这种强度与 Z 和 L 无关。因此，只要架构能够提供至少所需的每字节浮点操作数，增加更多缓存或增加行大小不能从根本上提高性能或可扩展性。
示例：矩阵乘法。传统矩阵乘法的算法对 $\Theta(n^2)$ 个字的数据执行 $W(n) = \Theta(n^3)$ 次操作；因此，$Q(n; Z, L)$ 的朴素下界是 $\Omega(n^2/L)$ 次传输。因此强度可能高达 $O(n)$。
考虑两种候选的矩阵乘法算法。第一种是图 2.4 所示的算法，它基于执行 n^2 次长度为 n 的归约。假设前面的归约分析成立，该算法导致 $Q(n; Z, L) = O\left(n^3/L\right)$ 次传输。因此强度仅为 O（1）。与我们朴素的估计 O（n）相比，很可能存在更好的算法。
著名的替代算法出现在图 2.6 中。该算法一次更新 C 的块，而不是 C 的单个元素，并流式处理 A 和 B 的块，如图所示。也就是说，算法执行块更新而不是行/列点积。在这种情况下，如果
• 块大小为 $b \times b$；- 三个块可以同时放入快速内存，即 $3b^2 \leq Z$；并且- 块连续存储，因此加载一个块只需要 b^2/L 次传输，那么可以证明 $Q(n; Z, L) = \mathrm{O}\left(n^3/(L\sqrt{Z})\right)$。因此，强度为 $\mathrm{O}\left(\sqrt{Z}\right)$。如果 $Z = \Omega(L^2)$，该算法的强度远高于朴素算法。在实践中，本地存储或缓存大小 Z 通常远大于相应的最小传输或行大小 L，因此这个假设很可能成立。[2]人们可能会合理地问，鉴于我们朴素的 $O\left(n^2\right)$ 下界估计，这个 $Q(n; Z, L)$ 的值是否还能进一步改进。对于一般矩阵和传统矩阵乘法算法，答案是没有算法能比块算法移动更少的字
算法——分块算法是渐近最优的。关于这种情况的非常易读的分析，见 Irony 等人 [53]。所谓的缓存无关方法也仅能达到这个界限 [37]。
关于 I/O 的一些总结性观察。I/O 分析的情况显然比并行性分析更复杂，体现在以下方面。

- I/O 分析不独立于机器参数，而工作-深度分析独立于处理器数量 p。
- 两个 I/O 分析示例都包含了连续布局的假设。也就是说，参数 L 的引入迫使算法设计者考虑数据布局。

- 回顾一下，对于矩阵乘法，我们能够给出任何算法的 I/O 传输次数的下界。这是一个理论分析领域，当进一步细化以考虑额外的架构细节时，可以洞察从代码或基准测试中难以提取的性能信息。
此外，我们可能会问，增加硬件内存资源（例如 Z 和 L）相对于增加硬件并行性（例如 p）的相对渐近收益是什么。例如，对于计算密集型矩阵乘法，我们看到将 Z 加倍

[2]这个规则的一个例外是 TLB，对于典型配置，它就像一个具有非常大行且容量为少量行的缓存。

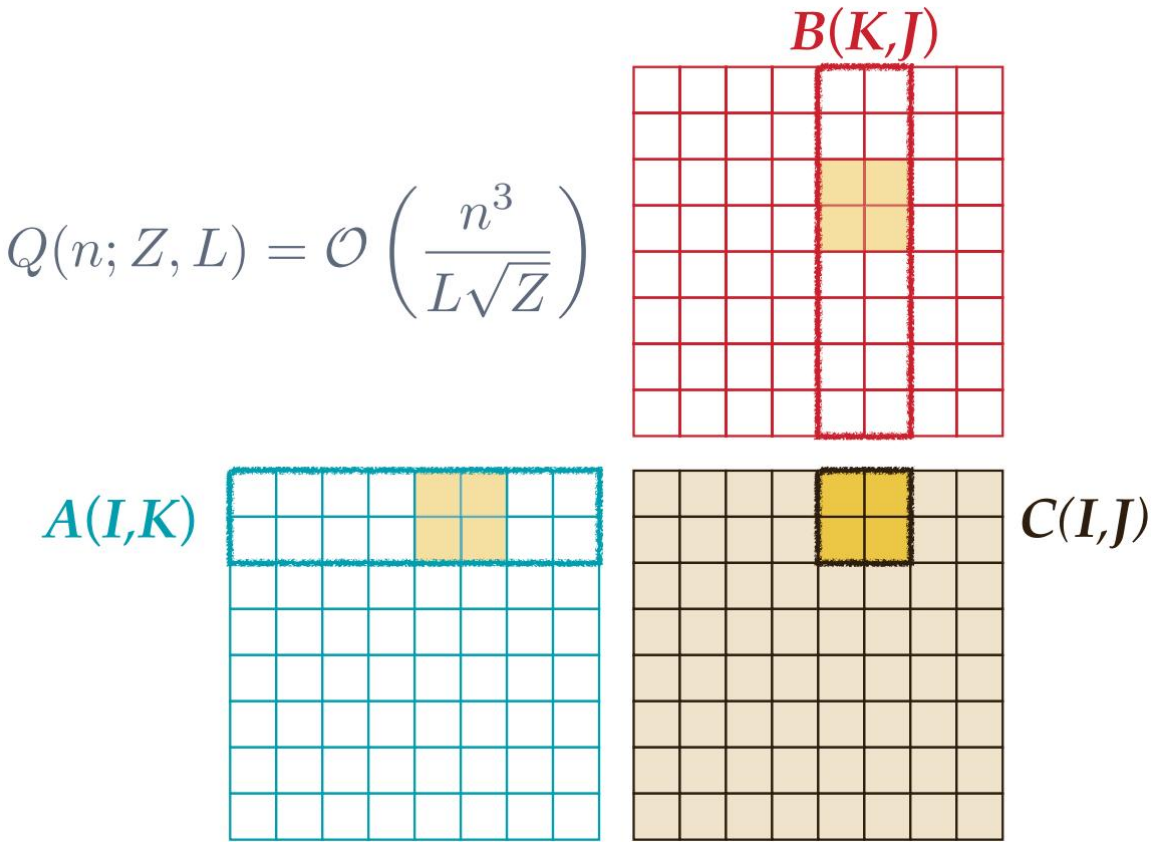


图 2.6 在外部内存模型中，分块矩阵乘法执行 $O(n^3/(L\sqrt{Z}))$ 次 I/O，假设：（i）计算 C 的输出块所需的 A、B 和 C 的三个块各自连续存储；（ii）这三个块恰好能放入快速内存。

会使 I/O 减少 $\sqrt{2}$ 倍，而将 L 加倍则使 I/O 减半。那么，如果加倍 Z 和加倍 L 的成本相等，加倍 L 将更具成本效益。这一分析证实了 GPU 设计背后的一种直觉：倾向于显著增加内存级并行性（更大的 L）而非更大的缓存（更大的 Z）。

2.4　并行性与 I/O 效率的综合分析

前面的讨论分别处理了并行性和局部性（I/O 效率）。然而，对于 GPGPU 平台，我们可能更倾向于同时考虑这两个算法属性的分析。
有几种模型在单一框架内结合了并行和 I/O 分析 [8, 14, 16, 25, 92]。它们的主要区别在于：

- 它们考虑私有缓存 [8] 或共享缓存，或两者兼有 [14，16，25]；
- 它们仅对 I/O 事务建模 [8，14，16，25] 还是包括其他成本，如延迟和带宽 [92]；
- 所得算法是资源感知的（例如缓存感知）[8，14，92] 还是资源无关的 [16，25]；以及- 是否将调度策略或提示作为执行模型的一部分 [8，14，16，25，92] 或否 [92]。

从这项工作中，正在涌现出重要的设计原则，这些原则应直接适用于当前和未来的 GPGPU 平台。这里有一个例子。Blelloch、Gibbons 和 Simhadri 提出，具有低深度和低顺序缓存复杂度[3]的嵌套并行算法很可能也具有良好的缓存高效并行行为 [16]。所谓顺序缓存复杂度，我们指的是嵌套并行执行的一种“自然”顺序排序，例如线程的深度优先执行。更准确地说，假设我们设计一个嵌套并行算法，其工作量为 $W(n)$，深度为 $D(n)$，在具有每个大小为 Z 的私有缓存和行大小 L 的一级内存层次结构上的顺序缓存缺失为 $Q_1(n; Z, L)$，并使用工作窃取调度。那么，Blelloch 等人证明了总并行执行中的缓存缺失数 $Q_p(n; Z, L)$ 的界限如下：

$$Q_p(n; Z, L) < Q_1(n; Z, L) + \mathrm{O}\left(\frac{p \cdot Z \cdot D(n)}{L}\right) \tag{2.2}$$

以高概率成立 [2]。人们可能会合理地猜测上界为 $p \cdot Q_1(n; Z, L)$，即算法在多线程情况下可能产生多达 p 倍的缺失，但公式（2.2）表明并非如此。另请注意，算法深度出现在界限中；因此，原则是低深度不仅有利于并行性，也有利于局部性。
尽管这类结果很有趣，但对于各种分析模型中哪个是“最佳”的，尚未达成明确共识，无论是在模型中设计算法的难易程度（与设计顺序 RAM 或并行 PRAM 算法相比），还是所得到的算法在真实机器上实现时的质量。这种共识的缺乏可能部分源于当前架构设计空间的不稳定性。然而，我们认为跟踪理论发展是有用且重要的，因为它们也可能为未来的硬件设计提供信息，正如下一节所暗示的那样。

[3]在缓存无关模型中 [36]。

2.5 抽象与具体度量

工作量、深度以及 I/O 或缓存缺失的数量都是抽象量，而最终目标——如果可能的话——是对执行时间和时间可扩展性做出更强的陈述。这种基于时间的分析至少需要一个架构成本模型。如果我们能够做到这一点，理论会告诉我们什么？

考虑图 2.7 所示的通用众核处理器系统示例。该系统有 p 个核心，每个核心每单位时间最多可执行 C_0 次操作；一个总大小为 Z 字的快速内存，分布在核心之间；以及一个内存系统，其传输 $m \cdot L$ 字的成本为 $\alpha + m \cdot L/\beta$，其中 α 是延迟，β 是以每单位时间字数为单位的带宽。[4]公式（2.1）为我们提供了一种估算仅完成该系统计算工作所需时间 $T_{\text{comp}}(n;p)$ 的方法。为了估算 I/O 时间 $T_{\text{mem}}(n)$，假设我们知道 $Q_p(n;Z,L)$，并对关键路径上的每个节点收取完整的延迟 α。进一步假设所有 $Q_p(n;Z,L)$ 次内存传输在最佳情况下将由内存系统聚合和流水线化，从而以峰值带宽 β 交付。那么，

$$T_{\text{mem}}(n;p,Z,L,\alpha,\beta) = \alpha \cdot D(n) + \frac{Q_p(n;Z,L)(n) \cdot L}{\beta}. \tag{2.3}$$

（这个成本模型并非定论，但足以说明几点。）

此计算以峰值速度运行（即计算受限）的一个必要条件是计算时间超过内存时间，即 $T_{comp} \geq T_{mem}$。如果这个条件成立，那么原则上我们可以隐藏数据移动的成本。我们将此约束称为平衡约束，它类似于 Kung 等人提出的系统平衡的经典定义，以及排队论中使用的定义 [21, 48, 59, 65]。在施加此平衡约束并应用一个

稍作代数运算，我们得到方程（2.4）：

$$\underbrace{\frac{p \cdot C_0}{\beta}}_{\text{balance}} \left(1 + \underbrace{\frac{\alpha \cdot \beta/L}{Q_p(n;Z,L)/D(n)}}_{\text{Little's Law}} \right) \leq \underbrace{\frac{W(n)}{Q_p(n;Z,L) \cdot L}}_{\text{intensity}} \left(1 + \underbrace{\frac{p}{W(n)/D(n)}}_{\text{Amdahl's Law}} \right). \tag{2.4}$$

将方程（2.4）与 Kung 的经典平衡原理 [59] 进行比较，用我们的符号表示应为：

$$\frac{p \cdot C_0}{\beta} \leq \frac{W(n)}{Q_p(n;Z,L) \cdot L}. \tag{2.5}$$

方程（2.5）表明，机器的固有平衡点（左侧）不应大于算法的固有计算强度（右侧）。事实上，方程（2.5）也出现在方程（2.4）中，但带有两个额外的修正项，而这两个修正项恰好体现了熟悉且直观的性能工程原理。

例如，方程（2.4）中的 $\alpha \cdot \beta/L$ 因子是延迟乘以带宽；如果该因子很大，算法设计者可以通过寻找具有高 $Q_p(n;Z,L)/D(n)$ 值的算法来补偿，我们可以将其解释为可用内存级并行度的平均值。实际上，这一项只是利特尔法则的重新表述。类似地，如前所述，我们可以将 $W(n)/D(n)$ 因子解释为 Amdahl 分数的倒数，从而使其所在的项融入了 Amdahl 定律。有趣的是，方程（2.4）统一并关联了这些经典原理 [30]。当然，方

[4]这个抽象机器旨在非常粗略地近似 NVIDIA Fermi 级架构，其中 Z 可能代表聚合寄存器和多处理器私有的第一级本地存储和缓存容量。

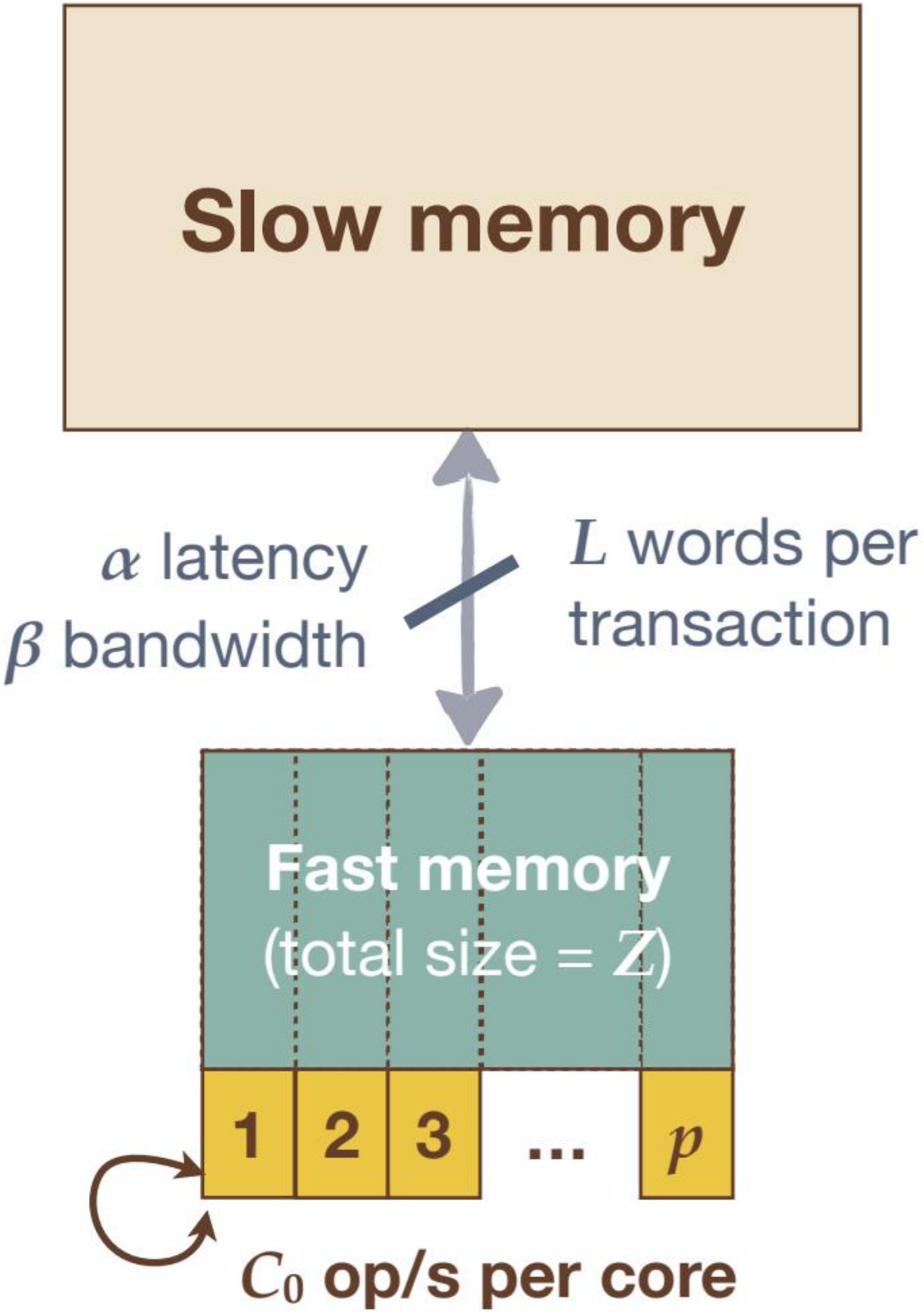


图 2.7　一个抽象众核处理器，具有 p 个核心，每个核心的峰值执行速率为每单位时间 C_0 次操作；总快速内存容量为 Z 字，分布在核心之间；以及慢速和快速内存之间的通道，具有延迟 α 时间单位，带宽 β 字每单位时间，最小事务大小 L 字。

程（2.4）在某种程度上仍是概念性的，因为算法特征——$W(n)$、$D(n)$ 和 $Q_p(n;Z,L)$——通常是在大 O 意义下计算的，且常数未定。然而，可以想象通过经验来估计这些特征。无论如何，令人欣慰的是，当将这些理论模型应用于类 GPGPU 系统时，会得出熟悉的经典原理。

2.6 小结

总之，缓存高效并行算法设计理论虽然仍在发展中，但提出了以下最佳实践，所有这些都与 GPGPU 平台相关。

- 在并行性方面，设计工作最优且深度低的算法，即具有渐近匹配最佳串行算法工作量的 $W(n)$，并且 $W(n) \gg D(n)$。
- 在内存局部性方面，设计高强度的算法，即具有 $W(n) \gg Q_p(n;Z,L)$，同时注意有限的内存容量（Z）和块传输（L）。
- 寻求具有丰富内存级并行性的算法，即 $Q_p(n;Z,L) \gg D(n)$。

我们有意省略了如何对 GPGPU 协处理器的有限带宽 I/O 通道（目前由 PCI Express（PCIe）实现）的影响进行建模的讨论。从某种意义上说，如果假设计算方案是将所有输入传输到 GPGPU，运行 GPGPU 代码，然后将结果复制回来，那么这种成本很容易建模。然而，我们完全预计，从长远来看，随着 GPGPU 单元与现在的主机处理器一起移到芯片上，GPGPU 协处理器的 PCIe 接口将会消失。

注释

- 2 我们假设采用传统的矩阵乘法，而非渐近更快的算法，如 Strassen 算法 [28]。

第三章　从原理到实践：分析与调优

根据第 2 章的原理设计了高层算法后，下一步是实现和性能工程以实现可扩展性。虽然没有硬性规则，但有许多广泛适用的启发式方法。在本节中，我们使用上一节的理论，探讨它如何指导和启发在 GPGPU 平台上的实际性能工程。为了具体起见，我们将使用一个基于 n 体粒子模拟中使用的快速多极子方法（FMM）[44] 的案例研究。[1] 我们的具体示例描述了作为血流模拟项目 [22，23，60，78] 一部分开发的 GPGPU 实现，尽管 n 体问题通常出现在图形学、计算物理、游戏和统计机器学习等众多应用中 [42]。

3.1　计算问题：粒子相互作用

在 n 体问题中，我们希望计算一组源粒子对一组目标粒子的影响。例如，粒子可能是恒星，我们希望在给定每颗恒星都受到所有其他恒星的引力的情况下预测它们的轨迹。更形式化地说，假设给定一个由 n 个源组成的系统，其位置由 $\{s_1, \ldots, s_n\}$ 给出，以及 n 个目标，其位置为 $\{t_1, \ldots, t_n\}$。那么，我们求解 n 体问题的目标是计算 n 个和，

$$\phi(t_i) = \sum_{j=1}^{n} \mathcal{K}(s_j, t_i) \cdot \delta(s_j), \quad i = 1, \ldots, n, \tag{3.1}$$

其中 $\phi(t)$ 称为目标点 t 处的势；$\delta(s)$ 是源点 s 处的密度；$\mathcal{K}(s,t)$ 是指定我们建模效果的相互作用核。例如，模拟由静电或引力引起的粒子相互作用的所谓单层拉普拉斯核具有 $\mathcal{K}(x,y) \propto \frac{1}{||x-y||}$，这表示相互作用强度与 x 和 y 之间的距离成反比。由于每个和需要 O（n）次操作，并且有 n 个这样的和，以直接方式计算这些和需要 O（n^2）次操作。

3.2　最优近似：快速多极子方法

快速多极子方法（FMM）近似计算方程（3.1）的和，将总操作数从 $O\left(n^2\right)$ 减少到 $O(n)$。该算法还接受期望的精度水平作为输入，这会影响复杂度常数 [43]。这种显著的渐近改进来自两个关键思想：（i）使用树在空间上组织点；（ii）使用快速近似评估，其中我们在树的每个节点上执行“汇总计算”，使用常数次整体树遍历，每个树节点的工作量恒定。本节提供了理解 GPU 调优案例研究所需的 FMM 背景知识。

构建树的方法通常如下，如图 3.1 所示为一个二维示例。首先创建一个对应于整个空

[1] 我们考虑 FMM 的一个特定变体，称为“核无关”方法。该算法复杂，其细节超出了本综合讲座的范围；我们改为请感兴趣的读者参阅其他资料 [43, 101]。

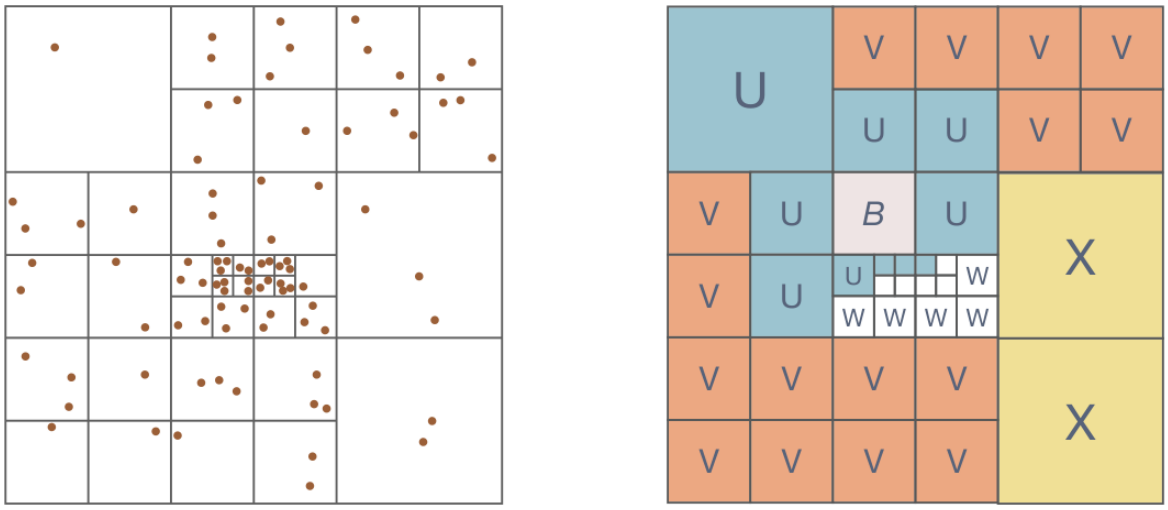


图 3.1 自适应四叉树及相互作用列表。左：空间中粒子的四叉树叶节点，每个叶节点最多包含 $q = 3$ 个粒子；右：相对于节点 B 的 U、V、W 和 X 列表。

间区域的根节点。接下来，将此根节点细分为固定数量的子节点，这些子节点对应于大小相等且不相交的空间子区域。最后，递归地重复此细分过程，直到所有叶子节点最多包含 q 个点，其中 q 是一个可调参数，用于控制算法的总体成本。[2]给定树后，快速近似评估包括几个阶段。每个阶段遍历树节点，并执行涉及该树节点和其他树节点子集的计算。这些阶段在计算强度以及并行类型和程度上有所不同。为了具体起见，我们接下来考虑其中两个最昂贵的阶段。

在 FMM 的所谓直接评估或 U 列表阶段中，每个叶子节点 B 都有一个关联的 U 列表，记为 $\mathcal{U}(B)$，其中包含 B 本身以及与 B 相邻的所有叶子节点（见
图 3.1）。U 列表计算遍历所有 $B' \in \mathcal{U}(B)$，执行方程（3.1）中的精确求和，其中 B 中的目标点作为目标，而 B' 中的源点作为源。图 3.2 展示了目标节点和源节点的假设集合，并说明了目标节点 B 与其 $\mathcal{U}(B)$ 之间的稀疏依赖关系。U 列表阶段的顺序伪代码见算法 1。

算法 1：FMM 的 U 列表直接求值

1. 对每个目标叶节点 $B \in \mathcal{T}$ 执行：
   (a) 对每个目标粒子 $t \in B$：
   (b) 遍历每个相邻源节点 $B' \in \mathcal{U}(B)$：
   (c) 对每个源粒子 $s \in B'$ 累加
   $\Phi(t) \leftarrow \Phi(t) + \mathcal{K}(s,t)\,\delta(s)$。

由于我们构建树时每个节点最多包含 q 个点，因此对每个 (B, B') 对评估方程（3.1）的成本为 $\mathrm{O}(q^2)$。当点在 d 维空间中均匀分布时，$|U(B)| = 3^d$，如果 d 是常数，我们

[2]注意，精度由独立于 q 的单独参数控制 [43，101]。然而，最小化执行时间的 q 设置确实取决于所需精度等因素。

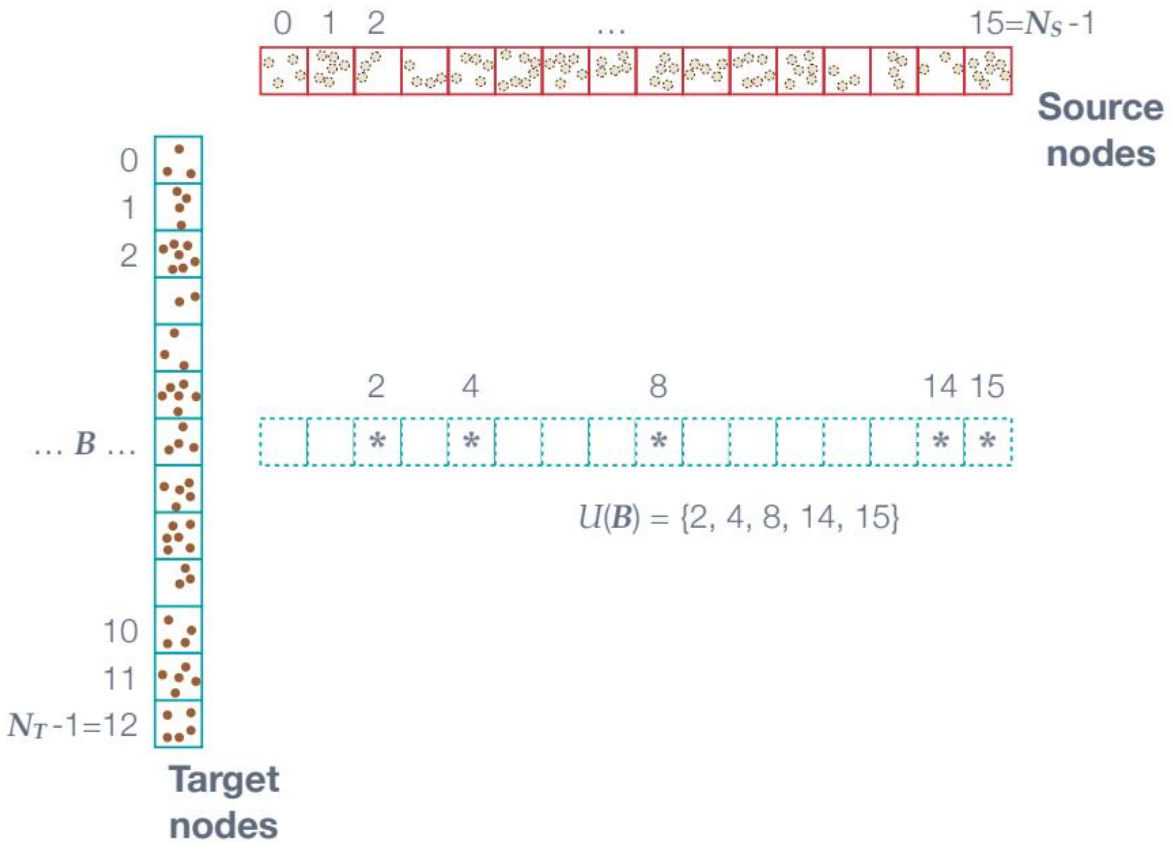


图 3.2　每个目标节点 B 将与由 $\mathcal{U}(B)$ 给出的源节点子集进行交互。

可以将其视为常数。

此外，叶子节点的数量将为 O（n/q），导致 U 列表阶段的总体复杂度为 O（nq）。

另一个重要阶段是所谓的 V 列表阶段。同样，对于每个叶节点 B，我们关联一个其他叶子的有界列表 $V(B)$[4]（见图 3.1）。V 列表阶段的总体成本恰好是 O（n/q）。与 U 列表的成本相比，我们看到 U 列表和 V 列表的成本之间存在权衡，由算法调优参数 q 控制。图 3.3 展示了在实际实验中测量的这种权衡 [23]。

3.3　设计并行且 I/O 高效的算法

U 列表阶段（算法 1）的并行性显而易见。首先，所有目标点都是独立的，因此两个最外层循环（算法 1，第 1-2 行）的迭代可以并行执行。其次，给定目标点 t，最内层循环构成一个归约。此循环嵌套不执行额外工作，因此总体并行工作 $W(n) = \mathrm{O}(nq)$ 与顺序算法相同。关于深度，最外层循环具有单位深度，而最内层循环由于归约而具有 $\log n$ 深度。因此，总体深度为 $D(n) = \mathrm{O}(\log n)$。可用的并行性为 $O(nq/\log n)$，随着 n 的增加而增长。因此，对于足够大的 n，应该有充足的并行性。

I/O 复杂度如何？我们从顺序情况开始，并进一步假设一个具有均匀点分布的 3-D 问题。在这种情况下，每个点是 4 个字——3 个坐标加上目标点的势或源点的密度——并且每个目标节点 B 将具有 $|\mathcal{U}(B)| = 27$，因为将有一个位于相同位置的源节点和 26 个

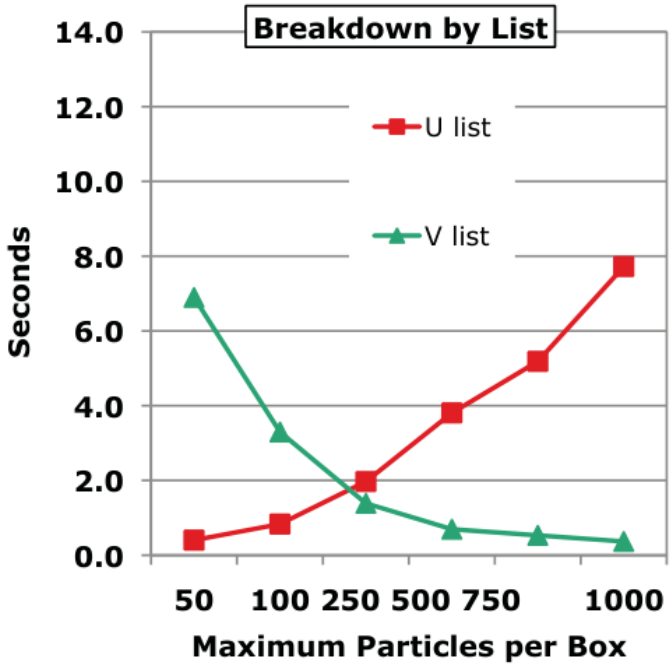


图 3.3 FMM 中 U 列表与 V 列表阶段随叶节点最大粒子数 q 变化的实测开销权衡。

距离 1 个节点的邻居。

假设有足够的缓存或本地存储 Z 来存储一个目标节点和一个源节点的所有点。即 $Z = q \times (4\text{ 字/点}) \times 2 = 8q$。在实践中，缓存或本地存储通常至少这么大，或者可以选择 q 使得该等式成立。然后，对于每个目标节点 B，我们将加载 q 个目标点、27q 个源点，执行 $27q^2$ 次交互，最后存储 q 个目标密度。因此，整个 U 列表计算，在所有 n qmax 个目标节点上，将传输 $Q(n; Z = 8q) = 116n$ 个字的数据。如果每个节点到节点的交互需要 κq^2 次浮点运算，则总体强度为每字 $\kappa q/116$ 次浮点运算。

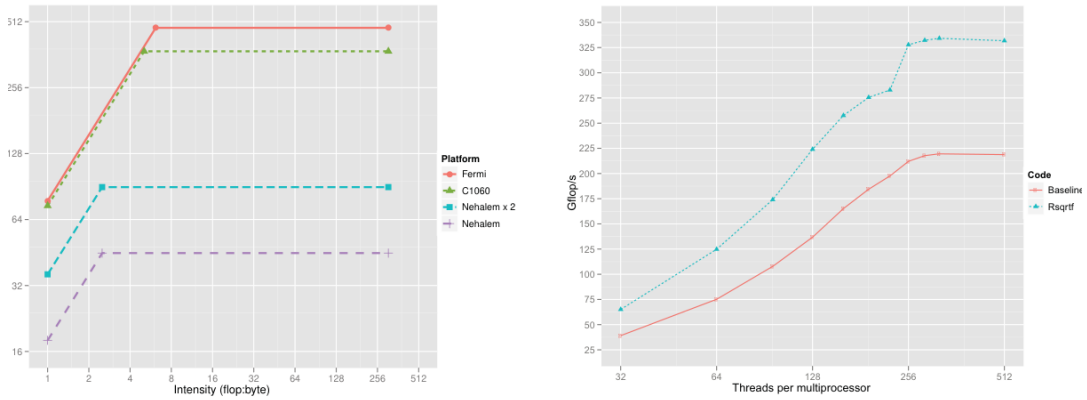
给定上述算法草图，我们现在可以将算法 1 转换为具体的基线代码实现，例如清单 3.1 的 CUDA 实现。这是每个 GPU 线程执行的单程序多数据（SPMD）风格代码。

代码清单 3.1：算法 1 的初始 CUDA 内核实现

```
int tid_min = Tptr[tnodeid]; // 此盒子中的第一个点
int tid_max = Tptr[tnodeid+1]; // 下一个盒子中的第一个点
for (int tid = tid_min + di; tid < tid_max; tid += q) {
    float4 tgt = T[tid]; // 加载点 (x, y, z, potential)
    for (int snode = snode_min; snode < snode_max; ++snode) {
    int snodeid = Ulist[snode]; int sid_min = Sptr[snodeid];
    int sid_max = Sptr[snodeid+1];
    for (int sid = sid_min; sid < sid_max; ++sid) {
    float4 src = S[sid]; // 加载源 (x, y, z, density)
    float3 dr = make_float3(
```

```
        tgt.x - src.x, tgt.y - src.y, tgt.z - src.z);
    float rsq = (dr.x*dr.x) + (dr.y*dr.y) + (dr.z*dr.z);
    float r = sqrtf(rsq);
    tgt.w += src.w / r;
    } // sid
    } // snode
    T[tid].w = tgt.w; // 存储势
} // tid
```

3.4　基线实现



(a) Roofline analysis helps to bound performance and distinguish between memory bandwidth vs. compute-boundedness.

(b) An empirical test on an NVIDIA C1060 to estimate compute-bound performance for the specific instruction mix of interest.

图 3.4　设定优化目标：算法调优参数与内存/计算限制之间的关系。

3.5　设定优化目标

我们的首要任务是利用并发性和 I/O 分析来设定优化目标。

例如，考虑当今典型的 GPU 能够达到约 1 Tflop/s，带宽约为 100 GB/s，或者“理想”的浮点运算与字节平衡比约为每字节 10 次浮点运算。对于重力或静电势之类的问题，如清单 3.1 所示，$\kappa \approx 11$ 次浮点运算。因此，如果数据是单精度（每字 4 字节），那么为了匹配每字节 10 次浮点运算，我们需要 $q \geq (10\text{ 次浮点运算/字节}) \times (464\text{ 字节})/(11\text{ 次浮点运算}) \approx 422$ 个点，或约 1.6 KB，以便受计算限制而非带宽限制。此容量完全在 GPU 多处理器上典型的本地存储大小范围内，因此我们应该预期可能实现计算限制，尽管我们需要通过经验调整 q 以确保如此大的值不会使我们过于远离真正的总体时间最小化器（回顾图 3.3）。

我们首先考虑一个纯 CPU 的基线实现，它运行在基于 Nehalem 处理器的双路 Intel x86 平台上，使用 OpenMP 和显式 SIMD（SSE）向量化进行并行化。对于这段代码，我们

观察到单精度下的性能在每秒 600 至 900 亿次浮点运算（Gflop/s）之间。正如图 3.4（a）所证实的，这恰好非常接近我们平台的单精度峰值。

对于我们的 GPU 实现（之前提到它应该是计算受限的），我们可以构建一个仅包含代码清单 3.1 第 12–15 行计算操作的微基准测试。也就是说，该微基准测试省略了所有内存引用，以估算一个真正计算受限的内核可能达到的最大吞吐量。这样的实验结果出现在图 3.4（b）中，针对两种情况。在第一种情况下，我们测试如代码清单 3.1 所示的指令组合。在第二种情况下，我们将单独的 sqrtf 和除法操作替换为对组合倒数平方根函数 rsqrtf 的单次调用。这一替换将性能额外提升了近 50%，我们的绝对性能目标是 340 Gflop/s。

相比之下，代码清单 3.1 的基线 GPU 实现在 NVIDIA C1060 上达到约 70 Gflop/s，与上述调优后的 CPU 基线相当。最内层循环每执行 11 次浮点运算就加载 16 字节数据（第 11 行），给出的浮点运算与字节比为 $11/16 \approx 0.7$ flops 每字节。与图 3.4（a）的屋顶线对比，我们看到该值确实大致符合我们的预期。

3.5.1 识别候选优化

鉴于我们预期性能是计算受限的，我们最初关注内存层次优化。众所周知的技术包括局部存储分块（tiling）、预取以及展开并合并（unroll-and-jam）技术，所有这些技术都使数据更接近处理器以提高重用率 [5]。此外，每种技术还通过调优参数进行参数化，例如块或瓦片大小、预取距离和展开深度。另外，对于 GPU，有一些民间证据表明，人们可能希望考虑使用结构体数组（AOS）还是数组结构体（SOA）布局。（代码清单 3.1 的参考代码将坐标打包成 float4 数据的数组，因此是 AOS 布局的一个例子。）一旦我们应用这些内存层次优化进入计算受限状态，我们就可以考虑更低层次的技术，例如使用组合倒数平方根操作。接下来，我们简要总结如何在 U-list 内核的上下文中应用这些优化；FMM 的其他内核可以类似地应用类似的变换。

共享内存的分块（tiling）。如图 3.2 所示，在目标节点到源节点交互的层面上存在自然的分块边界。因此，一个自然的想法是将尽可能多的源点和目标点加载到 GPU 多处理器本地存储（在 NVIDIA CUDA 术语中称为“共享内存”）中。对于代码清单 3.1，自然的是让线程块内的每个线程将一个或多个源点加载到共享内存中，并使用适当的同步调用（`__syncthreads()`）确保数据可用。这种变换与直接 n 体问题所执行的变换非常相似，后者是 GPU 文献中的一个著名例子 [9，75]。[3]

预取。作为分块（一种预取形式）的替代方案，人们也可以考虑以下更简单的预取方式。特别地，想象我们简单地修改代码清单 3.1，将下一个源点预取到本地寄存器变量中。如果最内层循环的延迟足够大以隐藏源点加载，这种方法可能会很好地工作。

展开并合并（Unroll-and-jam）。基线代码使用所有者计算策略，每个线程只分配一个输出（目标）点，并依赖大量线程来隐藏延迟。我们可以改为每个线程分配多个输出，以增加指令级并行性（ILP）和寄存器级局部性。这样做需要展开代码清单 3.1 第 4 行

[3]参见 https://developer.nvidia.com/gpugems3/gpugems3_ch31.html。

的目标点循环，然后融合最内层循环。代价是指令数增加和寄存器压力增大。然而，通过增加 ILP，我们还可以减少所需的线程数，从而实际上增加每个线程可用的寄存器数量。

AOS 与 SOA。考虑 SOA 布局而非 AOS 布局的主要潜在好处是增加内存事务的聚合程度。回顾我们在第 2.3 节中对内存层次流量的理论讨论，我们指出，增加内存事务宽度比增加本地存储容量可能更具成本效益；因此，选择 SOA 而非 AOS 可能是可衡量的。

3.5.2　探索优化空间

第 3.5.1 节中的优化定义了一个候选优化空间，但并未告诉我们哪些优化会提高性能以及提高多少。在本节中，我们考虑一种探索该空间的特别（启发式）方法。这种特定方法恰好在特定平台上找到了最优实现，但绝不保证这一点。关于算法、代码和架构之间相互作用的更精细模型是一个活跃的研究领域；我们在第 4.2 节中回顾了一些适用于 GPGPU 平台的工作。

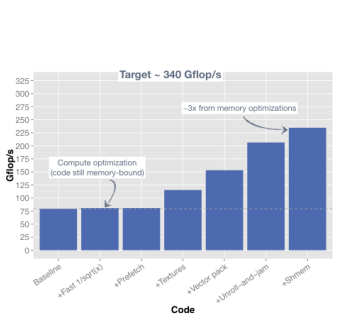
该特别方法很简单，大致基于统计学中的实验设计技术 [18]。令 C_0 表示基线代码，令 $S = \{s_1, s_2, \ldots, s_l\}$ 为我们考虑的所有优化的集合。为简单起见，假设这些优化可以按所有 $l!$ 种组合进行组合，假设顺序不重要。为了选择这些优化的一个子集，我们首先分别尝试 l 个优化中的每一个。假设 s_a 是最好的一个。然后，我们创建一个包含优化 s_a 的新代码 C_1。使用 C_1，我们重复相同的实验，仅使用从 S 中移除 s_a 后剩余的优化，即从 $S - \{s_a\}$ 中。如果其中最好的是 s_b，我们则从 C_1 生成新代码 C_2，使其包含优化 $s_a \in S$ 和 $s_b \in S - \{s_a\}$。我们重复此过程，直到没有优化可考虑为止。

将这一方法应用于清单 3.1，并使用第 3.5.1 节的优化，得到图 3.5 所示的实验序列。每个图代表一个阶段，其中我们有一组候选优化，分别应用于初始代码；初始代码是每个子图中最左侧的条形。观察第一阶段，图 3.5（a），基于本地存储的优化收益最大，使代码实现最接近计算受限；相比之下，倒数平方根没有收益，但主要是因为代码尚未处于计算受限状态，而不是因为它没有帮助。最终实现通过共享内存分块、倒数平方根和使用 SOA（图中标记为“vecpack”）的组合，达到了约 300 Gflop/s。有趣的是，SOA 在与其他两个优化结合之前从未是最优的。总体而言，我们达到的性能水平似乎相当接近我们基于微基准测试估计的 340 Gflop/s 上限。

3.5.3　总结

本节中的案例研究概述了如何从算法分析开始，并以此驱动后续的代码实现和调优过程。关键要素是：

- 从并发性和局部性的原则性分析开始；
- 使用微基准测试帮助建立性能边界；
- 枚举候选优化的“空间”；最后，



(a) Just one optimization applied.



(b) Two optimizations applied: the best from Figure 3.5(a) and paired with all remaining optimizations.



(c) Three optimizations applied: the best pair from Figure 3.5(b) combined with each of the remaining.

图 3.5 在基线性能为 75 Gflop/s 时，启发式优化搜索方法的应用结果。

- 系统地探索该空间。尽管如此，潜在优化的空间通常可能非常大。悬而未决的问题是，我们可以在多大程度上使用建模或其他定量分析来帮助定义和探索这个空间，我们将在下一节讨论。

原著注释

1. 为简单起见，假设源点和目标点的数量相同；实际可能不同，两组点也可能相同，此时求和中排除自身相互作用。

2. 精度由另一个独立于 q 的参数控制；不过，使执行时间最短的 q 值会受到目标精度等因素影响 [43, 101]。

3. 对于每个叶节点 B，V 列表 $V(B)$ 是 B 的父节点的各邻居的子节点构成的集合与 $U(B)$ 的差集。

第四章　用详细的性能分析指导优化

在前面的章节中，我们解释了如何在高层算法和性能分析中识别性能优化候选。工作深度模型主要考虑实际的计算工作和内存操作，这些在高层表示。在本章中，我们关注利用低级信息（如指令和内存事务）的性能行为，而不是或补充高层信息，然后讨论这些信息如何指导优化。

4.1　指令级分析与调优

在本节中，我们描述了一种指令级分析，也包括处理器微架构方面的细节。针对 CPU 和 GPU 已经进行了大量的性能建模。其他工作的总结在第 4.2 节中讨论。这里，我们解释一个基于 Sim 等人 [85] 方法的模型。首先，我们描述如何对 GPGPU 的性能行为进行建模，然后展示如何识别可以优化的性能瓶颈。性能模型基于 NVIDIA 的 Fermi 架构。

4.1.1　执行时间建模

首先，我们定义 T_{exec} 为总体执行时间，它是 T_{comp}、T_{mem} 和 $T_{overlap}$ 的函数，如公式（4.1）所示：

$$T_{\text{exec}} = T_{\text{comp}} + T_{\text{mem}} - T_{\text{overlap}}. \tag{4.1}$$

执行时间通过将计算和内存成本相加，同时减去由于 GPGPU 中的多线程特性而产生的重叠成本来计算。图 4.1 说明了当四个 warp 运行时的公式。公式（4.1）的三个输入如下所述。

计算计算成本 T_{comp}

T_{comp} 是执行计算指令的时间量（不包括内存操作等待时间，但包括执行内存指令的成本），并使用公式（4.2）–（4.10）进行评估。

我们将计算成本视为两个组成部分：可并行化的基本执行时间，加上由于串行化效应产生的开销成本：

$$T_{\text{comp}} = \underbrace{W_{\text{parallel}}}_{\text{Base}} + \underbrace{W_{\text{serial}}}_{\text{Overhead}}. \tag{4.2}$$

基本时间 W_{parallel} 考虑了操作数量和并行度，并根据基本指令与硬件参数计算，如公式（4.3）所示：

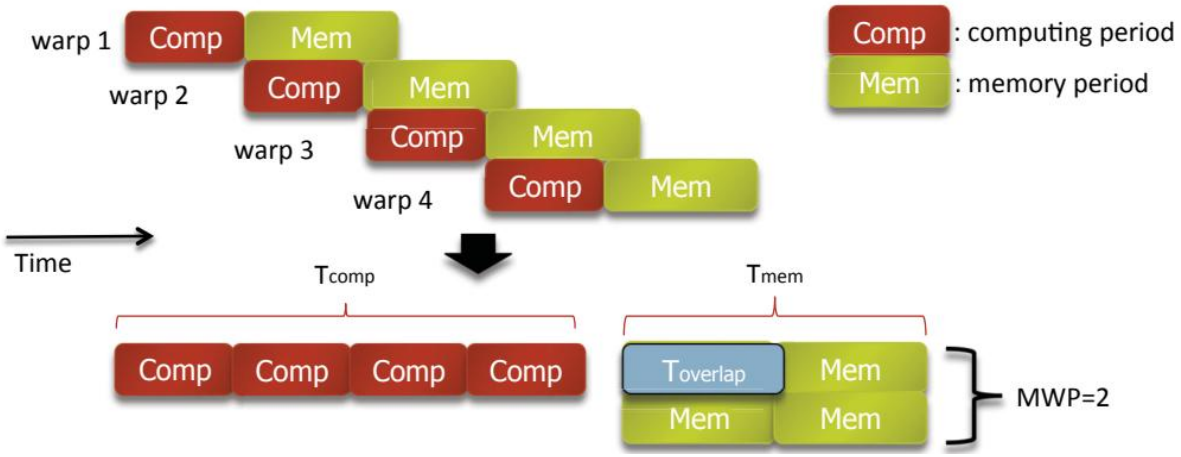


图 4.1 执行时间的可视化。（有四个内存请求，但由于内存级 warp 并行性，只有两个内存请求对总执行时间有贡献。）

$$W_{\text{parallel}} = \underbrace{\frac{\#\,\text{insts} \times \#\,\text{total_warps}}{\#\,\text{active_SMs}}}_{\text{每个 SM 的总指令数}} \times \underbrace{\frac{\text{avg_inst_lat}}{\text{ITILP}}}_{\text{有效吞吐率}}. \tag{4.3}$$

公式（4.3）中的第一个因子是每个 SM 的动态指令总数，第二个因子表示有效指令吞吐率。关于后者，平均指令延迟 `avg_inst_lat` 可以用 GPGPU 中 FP 操作的延迟来近似。必要时，也可以通过考虑指令组合以及底层硬件上各类指令的延迟来精确计算。值 ITILP 用于刻画 GPGPU 中线程间指令级并行（inter-thread ILP）的可能性。ITILP 的概念由 Baghsorkhi 等人 [11] 引入。特别地，指令可能从 GPGPU 上的多个 warp 发出；因此，我们考虑全局 ILP（即 warp 间的 ILP），而不是 warp 局部 ILP（即单个 warp 的 ILP）。也就是说，ITILP 表示所有执行线程中有多少 ILP 可用于隐藏流水线延迟。ITILP 可由以下公式获得：

$$\text{ITILP} = \min(\text{ILP} \times N,\ \text{ITILP}_{\text{max}})\,. \tag{4.4}$$

$$\text{ITILP}_{\text{max}} = \frac{\text{avg_inst_lat}}{\text{warp_size}/\text{SIMD_width}}. \tag{4.5}$$

其中，N 是一个 SM 上的活动 warp 数，`SIMD_width` 和 `warp_size` 分别表示向量单元宽度和每个 warp 的线程数。在 Fermi 架构上，`SIMD_width` = `warp_size` = 32。ITILP 不能大于 $\text{ITILP}_{\text{max}}$，即完全隐藏流水线延迟所需的 ITILP。

我们将公式（4.2）中的串行化开销 W_{serial} 建模为

$$W_{\text{serial}} = O_{\text{sync}} + O_{\text{SFU}} + O_{\text{CFdiv}} + O_{\text{bank}}. \tag{4.6}$$

其中四项分别表示同步、SFU 资源争用、控制流发散和共享内存 bank 冲突带来的开销。下面依次说明这些开销项。

同步开销 O_{sync}：当存在同步点时，同步点之后的指令在所有线程都到达该点之前不能执行。如果所有线程以相同速率推进，则等待时间开销很小。不幸的是，每个线程（或 warp）会根据源操作数的可用性按照各自的速度前进，因此会出现一个进度范围，而且这个范围有时会很大。造成这种范围的原因主要是 DRAM 访问延迟的变化（如队列延迟、DRAM 行缓冲命中/未命中等）以及控制流发散。结果，当存在较多内存指令和同步指令时，开销会增加，如公式（4.7）和（4.8）所示：

$$O_{\text{sync}} = \frac{\#\,\text{sync_insts} \times \#\,\text{total_warps}}{\#\,\text{active_SMs}} \times F_{\text{sync}}. \tag{4.7}$$

$$F_{\text{sync}} = \Gamma \times \text{avg_DRAM_lat} \times \underbrace{\frac{\#\,\text{mem_insts}}{\#\,\text{insts}}}_{\text{内存指令比例}}. \tag{4.8}$$

其中 Γ 是与机器相关的参数。对于所建模的架构，假定该值为 64。

SFU 资源争用开销 O_{SFU}：此开销主要由特殊功能单元（SFU）的特性引起，并由下面的公式（4.9）和（4.10）计算。

对于大多数 GPGPU，超越函数和平方根等昂贵的数学运算可以由专用执行单元——特殊功能单元（SFU）——处理。由于 SFU 指令的执行可以与其他浮点（FP）指令重叠，因此在 SFU 指令与 FP 指令的比例较为合适时，SFU 指令的开销几乎可以被隐藏。否则，SFU 争用会降低性能。因此，SFU 指令的可见执行开销取决于 SFU 指令与其他指令的比例，以及各类指令对应执行单元的数量。在公式（4.9）中，可见性由 F_{SFU} 建模，其取值范围为 $[0, 1]$。当 $F_{\text{SFU}} = 0$ 时，表示 SFU 执行开销不会增加到总执行时间中。当 SFU 指令比例小于特殊功能单元与 SIMD 单元数量之比时，就会出现这种情况，如公式（4.10）所示：

$$O_{\text{SFU}} = \frac{\#\,\text{SFU_insts} \times \#\,\text{total_warps}}{\#\,\text{active_SMs}} \times \frac{\text{warp_size}}{\text{SFU_width}} \times F_{\text{SFU}}. \tag{4.9}$$

$$F_{\text{SFU}} = \min\left\{\max\left[\frac{\#\,\text{SFU_insts}}{\#\,\text{insts}} - \frac{\text{SFU_width}}{\text{SIMD_width}},\ 0\right],\ 1\right\}. \tag{4.10}$$

控制流发散与存储体冲突开销 O_{CFdiv} 和 O_{bank}：控制流发散的开销（O_{CFdiv}）是指由于发散分支等原因 [50] 而额外执行指令的成本。该成本通过统计两条路径中的所有指令来建模。共享内存 bank 冲突的开销（O_{bank}）则可通过测量共享内存 bank 冲突的数量来计算。O_{CFdiv} 和 O_{bank} 均可以使用硬件计数器进行测量。

计算内存访问成本 T_{mem}

T_{mem} 表示内存请求与数据传输所花费的时间。该成本是内存请求数量、每次请求的内存延迟以及内存级并行度的函数。我们使用公式（4.11）对 T_{mem} 建模：

$$T_{\text{mem}} = \underbrace{\frac{\#\,\text{mem_insts} \times \#\,\text{total_warps}}{\#\,\text{active_SMs} \times \text{ITMLP}}}_{\text{每个 SM 的有效内存请求数}} \times \text{AMAT}. \tag{4.11}$$

其中，AMAT（平均内存访问时间）用于建模平均内存访问时间，并考虑了缓存效应。我们使用公式（4.12）和（4.13）计算 AMAT：

$$\text{AMAT} = \text{avg_DRAM_lat} \cdot \text{miss_ratio} + \text{hit_lat}. \tag{4.12}$$

$$\text{avg_DRAM_lat} = \text{DRAM_lat} + (\text{avg_trans_warp} - 1)\,\Delta. \tag{4.13}$$

`avg_DRAM_lat` 表示平均 DRAM 访问延迟，它是基线 DRAM 访问延迟 `DRAM_lat` 与事务发出间隔 Δ 的函数。在 GPGPU 中，一个内存请求可以被拆分为多个事务。在所述模型中，`avg_trans_warp` 表示一个 warp 中每次内存请求的平均事务数。注意，公式（4.12）可以扩展到多级缓存情形，为简洁起见这里省略。

我们用线程间 MLP（记为 ITMLP）来建模内存级并行度，并将其定义为一个 SM 上可并发处理的内存请求数量。与 ITILP 类似，不同 warp 发出的内存请求可以彼此重叠。由于 MLP 只是 warp 内内存级并行度的指标，我们仍然需要考虑多个 warp 的重叠因子。ITMLP 可以使用公式（4.14）和（4.15）计算：

$$\text{ITMLP} = \min(\text{MLP} \times \text{MWP}_{\text{cp}},\ \text{MWP}_{\text{peak_bw}})\,. \tag{4.14}$$

$$\text{MWP}_{\text{cp}} = \min(\max(1,\ \text{CWP} - 1),\ \text{MWP})\,. \tag{4.15}$$

在公式（4.14）中，MWP_{cp} 表示在一个计算阶段中，其内存请求能够彼此重叠的 warp 数量。MWP 表示可同时访问内存的最大 warp 数；但是，受 CWP 限制，可并发发出内存请求的 warp 数量是有限的。

$\text{MWP}_{\text{peak_bw}}$ 表示在峰值内存带宽下，每个 SM 所能支持的 memory warp 数。由于该值等同于每个 SM 能够达到的最大内存请求数，ITMLP 不能大于 $\text{MWP}_{\text{peak_bw}}$。

计算重叠成本 T_{overlap}

T_{overlap} 表示多线程能够隐藏多少内存访问成本。在 GPGPU 执行过程中，当一个 warp 发出内存请求并等待所请求的数据时，处理器会切换到另一个 warp。因此，T_{comp} 与 T_{mem} 在一定程度上可以重叠。例如，如果多线程完全隐藏了所有内存访问成本，那么 T_{overlap} 将等于 T_{mem}。也就是说，在这种情况下，总执行时间 T_{exec} 仅由计算成本 T_{comp} 决定。相反，如果在最坏情况下完全无法隐藏内存访问，那么 T_{overlap} 将为 0。

我们使用公式（4.16）和（4.17）来计算 T_{overlap}。在这些公式中，F_{overlap} 近似表示 T_{comp} 与 T_{mem} 的重叠程度，而 N 表示每个 SM 的活动 warp 数，如公式（4.4）所定义。

注意，F_{overlap} 会同时随 MWP 与 CWP 的变化而变化。当 CWP 大于 MWP 时（例如受内存操作限制的应用），F_{overlap} 会变为 1，这意味着全部 T_{comp} 都可以与 T_{mem} 重叠。另一方面，当 MWP 大于 CWP 时（例如受计算限制的应用），只有部分计算成本可以被重叠隐藏：

$$T_{\text{overlap}} = \min(T_{\text{comp}} \times F_{\text{overlap}},\ T_{\text{mem}})\,. \tag{4.16}$$

$$F_{\text{overlap}} = \frac{N-\zeta}{N}, \qquad \zeta = \begin{cases} 1, & \text{CWP} \le \text{MWP}, \\ 0, & \text{CWP} > \text{MWP}. \end{cases} \tag{4.17}$$

计算 CWP 和 MWP　`mem_cycles`：每个 warp 的内存等待周期数；`comp_cycles`：每个 warp 的计算周期数。

#insts：每个 warp 的指令数（不包括 SFU 指令）

#mem_insts：每个 warp 的内存指令数

`mem_peak_bandwidth`：DRAM 与 GPU 核心之间的带宽（例如，Tesla C2050 中为 144.0 GB/s）

$$\text{CWP} = \min(\text{CWP}_{\text{full}},\ N)\,. \tag{4.18}$$

$$\text{CWP}_{\text{full}} = \frac{\text{mem_cycles} + \text{comp_cycles}}{\text{comp_cycles}}. \tag{4.19}$$

$$\text{comp_cycles} = \frac{\#\,\text{insts} \times \text{avg_inst_lat}}{\text{ITILP}}. \tag{4.20}$$

$$\text{mem_cycles} = \frac{\#\,\text{mem_insts} \times \text{AMAT}}{\text{MLP}}. \tag{4.21}$$

$$\text{MWP} = \min\left(\frac{\text{avg_DRAM_lat}}{\Delta},\ \text{MWP}_{\text{peak_bw}},\ N\right). \tag{4.22}$$

$$\text{MWP}_{\text{peak_bw}} = \frac{\text{mem_peak_bandwidth}}{\text{BW}_{\text{per_warp}} \times \#\,\text{active_SMs}}. \tag{4.23}$$

$$\text{BW}_{\text{per_warp}} = \frac{\text{freq} \times \text{transaction_size}}{\text{avg_DRAM_lat}}. \tag{4.24}$$

`freq`：SM 处理器的时钟频率（例如，Tesla C2050 中为 1.15 GHz）

`transaction_size`：DRAM 请求的事务大小（例如，Tesla C2050 中为 128B）

`BW_per_warp`：每个 warp 的带宽需求

潜在收益预测　潜在收益指标表示当我们有可能消除理想性能与当前性能之间的差值时，所能获得的性能改进。公式（4.25）和（4.26）用于估计理想的计算性能与内存性能（时间）；当然，算法开发者也可以直接提供这些估计值：

$$T_{\text{fp}} = \frac{\#\,\text{FP_insts} \times \#\,\text{total_warps} \times \text{FP_lat}}{\#\,\text{active_SMs} \times \text{ITILP}}. \tag{4.25}$$

$$T_{\text{mem_min}} = \frac{\text{size_of_data} \times \text{avg_DRAM_lat}}{\text{MWP}_{\text{peak_bw}}}. \tag{4.26}$$

然后，我们使用公式（4.27）–（4.30）得到收益指标，其中 $\text{ITILP}_{\text{max}}$ 在公式（4.5）中定义：

$$B_{\text{itilp}} = W_{\text{parallel}} - \frac{\#\,\text{insts} \times \#\,\text{total_warps} \times \text{avg_inst_lat}}{\#\,\text{active_SMs} \times \text{ITILP}_{\max}}. \tag{4.27}$$

$$B_{\text{serial}} = W_{\text{serial}}. \tag{4.28}$$

$$B_{\text{fp}} = T_{\text{comp}} - T_{\text{fp}} - B_{\text{itilp}} - B_{\text{serial}}. \tag{4.29}$$

$$B_{\text{memlp}} = \max(T'_{\text{mem}} - T_{\text{mem_min}},\ 0)\,. \tag{4.30}$$

表 4.1 性能模型的输入参数及来源

模型参数	定义	来源
#insts	每个 warp 的指令总数（不含 SFU 指令）	硬件性能计数器
#mem_insts	每个 warp 的内存指令数	硬件性能计数器
#sync_insts	每个 warp 的同步指令数	指令分析器
#SFU_insts	每个 warp 的 SFU 指令数	指令分析器
#FP_insts	每个 warp 的浮点指令数	指令分析器
#total_warps	一个内核中的 warp 总数	程序输入
#active_SMs	活跃 SM 数量	硬件规格
N	一个 SM 上同时运行的 warp 数	硬件性能计数器
AMAT	平均内存访问延迟	硬件性能计数器
avg_trans_warp	每次内存请求的平均事务数	硬件性能计数器
avg_inst_lat	平均指令延迟	硬件规格
miss_ratio	缓存未命中率	硬件性能计数器
size_of_data	输入数据大小	源代码
ILP	单个 warp 内的指令级并行度	指令分析器
MLP	单个 warp 内的内存级并行度	指令分析器
MWP（每 SM）	可并发访问内存的最大 warp 数	模型方程
CWP（每 SM）	一个内存等待周期内可执行的 warp 数加一	模型方程
$\text{MWP}_{\text{peak_bw}}$（每 SM）	峰值内存带宽下的 MWP	模型方程
warp_size	每个 warp 中的线程数	32
Γ	同步开销的机器参数	64
Δ	内存事务发出间隔	测得的机器参数
DRAM_lat	基准 DRAM 访问延迟	测得的机器参数
FP_lat	浮点指令延迟	硬件规格
hit_lat	缓存命中延迟	硬件规格
SIMD_width	每个 SM 的标量处理器（SP）数量	硬件规格
SFU_width	每个 SM 的特殊函数单元（SFU）数量	硬件规格

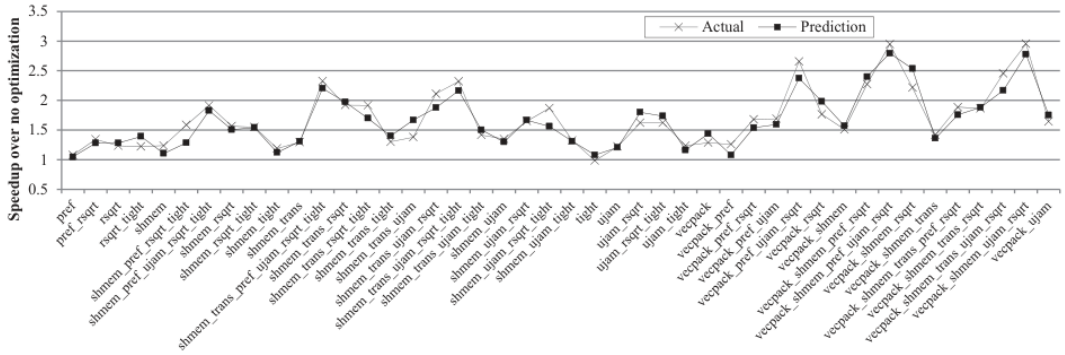


图 4.2　44 种优化配置下，实测加速比与模型预测值的比较（图例 Actual 表示实测值，Prediction 表示模型预测值）。

4.1.2　将模型应用于 FMM

我们将此模型应用于快速多极子方法（FMM）的实现，该方法已在第 3 章中描述。我们采用与第 3.5.1 节所述相同的优化集。图 4.2 展示了相对于基线内核的实际执行加速比以及使用所提出模型的预测值。横轴表示代码优化空间，其中展示了 44 种优化组合。结果表明，总体而言，模型紧密估计了不同优化的加速比。它还表明，分析模型成功识别了性能最佳的优化组合。

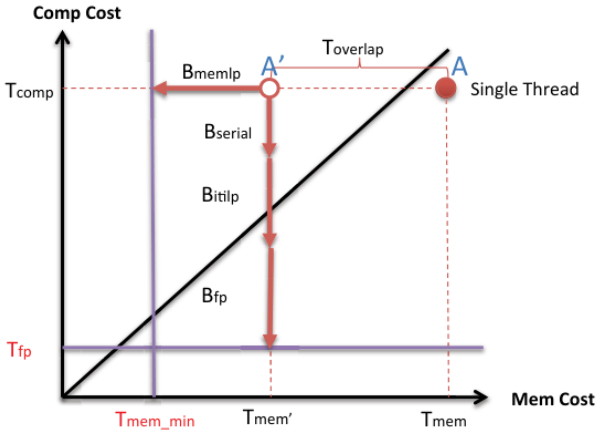


图 4.3　性能潜在收益示意：纵轴 Comp Cost 为计算开销，横轴 Mem Cost 为内存开销；图中各箭头表示潜在可节省的时间。

4.1.3　性能优化指南

基于所提出的性能模型，我们可以识别性能瓶颈信息并估计通过减少或消除这些瓶颈可能获得的潜在收益。它通过四个潜在收益指标来实现这一点，其影响可以使用如图 4.3 所示的图表进行可视化。横轴表示内存操作的成本，纵轴表示计算成本。应用程序代码是该图上的一个点（此处为点 A）。横轴和纵轴值之和是执行成本，但由于计算和内存成本可以重叠，最终执行成本 T_{exec}（例如，挂钟时间）是相对于 A 偏移的另一个点 A'。偏移量表示为 $T_{overlap}$。通过 y = x 的对角线将图表分为计算受限区和内存

受限区，分别指示应用程序是受计算限制还是受内存操作限制。从点 A’ 出发，收益图显示了四个不同的潜在收益指标如何在该空间中移动应用程序执行时间。

给定算法还可以通过两个附加值进一步表征。第一个是理想计算成本，通常执行所有基本计算工作（例如，浮点运算）的最短时间，在图 4.3 中记为 T_{fp}。第二个是将所有数据从 DRAM 移动到核心的最短时间，记为 T_{mem_min}。当内存请求被预取或所有内存服务被其他计算隐藏时，我们可能希望隐藏或或许消除所有内存操作成本。理想情况下，算法设计者或程序员可以提供 T_{fp} 和 T_{mem_min} 的估计或界限。然而，当信息不可用时，我们可以尝试从内核中执行的 FP 指令数等来估计 T_{fp}。[1]假设我们在点 A’ 处有一个具有不同种类低效的内核。我们计算的收益因子旨在量化通过消除这些低效可能实现的改进程度。我们使用四个潜在收益指标，总结如下。

- B_{itilp} 表示通过增加线程间指令级并行性带来的潜在收益。
- B_{memlp} 表示通过增加内存级并行性带来的潜在收益。
- B_{fp} 表示当我们理想地移除低效计算成本时的潜在收益。与其他收益不同，我们无法实现 100% 的 B_{fp}，因为内核必须有一些操作，如数据移动。
- B_{serial} 显示了当我们消除由于串行化效应（如同步和资源争用）引起的开销时的节省量。

B_{itilp}、B_{fp} 和 B_{serial} 与计算成本相关，而 B_{memlp} 与内存成本相关。这些指标总结在表 4.2 中。

表 4.2 性能优化指导指标

指标	含义	类型
T_{exec}	最终预测的执行时间	时间
T_{comp}	计算时间	时间
T_{mem}	内存访问时间	时间
T_{overlap}	计算和内存访问重叠执行的时间	时间
T'_{mem}	$T_{\text{mem}} - T_{\text{overlap}}$	时间
T_{fp}	理想情况下完成必要浮点计算所需的最短时间	理想时间
$T_{\text{mem_min}}$	理想情况下的数据传输时间	理想时间
B_{serial}	消除串行化效应的潜在收益	收益
B_{itilp}	增加线程间指令级并行度的潜在收益	收益
B_{memlp}	增加内存级并行度的潜在收益	收益
B_{fp}	提高计算效率的潜在收益	收益

[1]原著评估还利用浮点操作数量估计 T_{fp}。

潜在收益预测示例　为了理解这些性能优化指导指标，我们首先计算基线实现的潜在收益指标：$B_{\text{serial}} = 0$，$B_{\text{memlp}} = 0$，$B_{\text{itilp}} = 6068$，$B_{\text{fp}} = 9691$。即使在基线版本中，内核也已经呈现计算受限特征。因此，降低计算成本的技术至关重要。

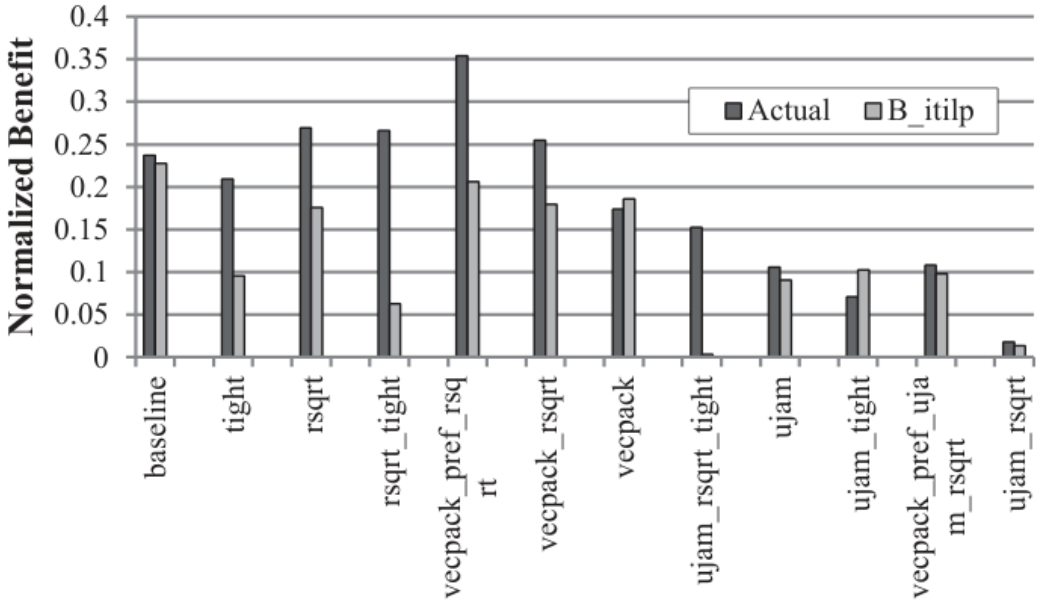


图 4.4　在不同优化组合上应用共享内存优化时，实测收益与 B_{itilp} 的比较。

图 4.4 显示了在不同优化组合之上应用共享内存优化（shmem）时的实际性能收益和 B_{itilp}。例如，ujam 表示 ujam 和 ujam + shmem 优化之间的性能差异。在图中，Actual 和 B_{itilp} 都归一化为应用 shmem 之前的执行时间。使用共享内存改善了 MLP 和 ILP。它增加了源点的重用，这也增加了 ILP。因此，

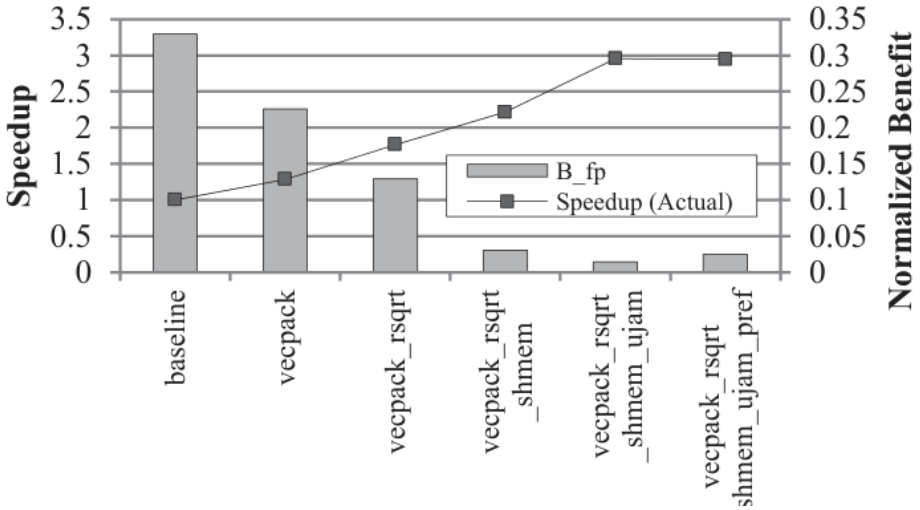


图 4.5　逐步应用优化时，相对于基线的实际加速比与潜在收益。

4.2 其他性能建模技术和工具

使用共享内存的收益可以使用 B_{itilp} 来预测，因为 $B_{\text{memlp}} = 0$。如图所示，B_{itilp} 紧密预测了大多数优化组合的实际性能收益，除了紧密优化。shmem 和紧密优化之间的交互还需要进一步分析。

图 4.5 表示当逐个应用优化时，相对于基线执行时间归一化的性能加速比和潜在收益。对于基线，较高的 B_{fp} 值表明内核仍然具有进一步改进的潜力。因此，尝试优化基线内核是合理的。当我们应用适当的优化技术改进内核时，潜在收益减少。当应用 vecpack、rsqrt、shmem 和 ujam 后，潜在收益指标表明内核可能只有非常少量的低效计算，通过进一步优化可能获得的潜在性能提升可能有限。如最后一个条形图所示，pref 并未导致性能改进。相反，由于预取代码中的低效计算，B_{fp} 略有增加。

除了上一节的模型之外，关于 GPGPU 性能建模的研究层出不穷。本节综述了当前可用于理解 GPGPU 性能的指标和工具的全貌。

4.2.1 有限的性能可见性

高效地编写 GPU 程序在智力上仍然具有挑战性，并且需要付出巨大的努力。GPU 编程的主要挑战通常是如何及时地向高带宽计算处理单元提供所需数据，从而使其饱和。为了实现这一

目标，GPGPU 应用程序必须有效利用 GPU 内存层次结构。因此，关于应用程序与内存系统交互的信息对于优化内存访问及其性能是必要的。

开发人员需要手动检查 GPGPU 内核源代码，并识别性能下降的来源，例如紧密循环关键路径上的长延迟加载。这种分析通常通过猜测和试错实验来完成。

此外，多级并发和大量并发事件（例如在途内存操作）可能使开发人员甚至优化编译器都难以对性能进行推理。高度多线程的 GPU 就像一个具有紧密耦合动态特性的复杂非线性系统；微架构事件（例如内存加载）应在有意义的上下文中与其他共存线程的事件一起评估。

因此，即使对于经验丰富的应用程序开发人员来说，性能评估过程也非常耗时，而对于知识较少的程序员来说则更加困难和繁琐。目前有数百万个单元在使用中，GPU 已成为部署最广泛的并行系统。随着越来越多的用户尝试使用 GPU，并且它们在所有类型的计算系统中的使用不断增长，其成本将继续下降。这一趋势反过来可能会扩大 GPGPU 计算社区，并需要开发在高度多线程执行环境的上下文中系统分析源代码的技术，以取代手动调查代码。

为了提供一个研究这个问题的正式框架，Baghsorkhi 等人引入了平衡 GPGPU 计算的概念 [11]。该模型使用平均 warp 所承载的计算来表示 GPGPU 计算。在 GPU 中执行计算时，如果满足以下条件，则称该计算是平衡的：

1. 计算核心流水线不会因数据冒险（寄存器依赖或长延迟全局内存加载）、结构冒险（共享内存体冲突）或控制冒险（SIMD 分支发散）而停顿；并且

2. 计算时间等于内存时间。前者是在假设理想内存系统的情况下完成计算所需的时

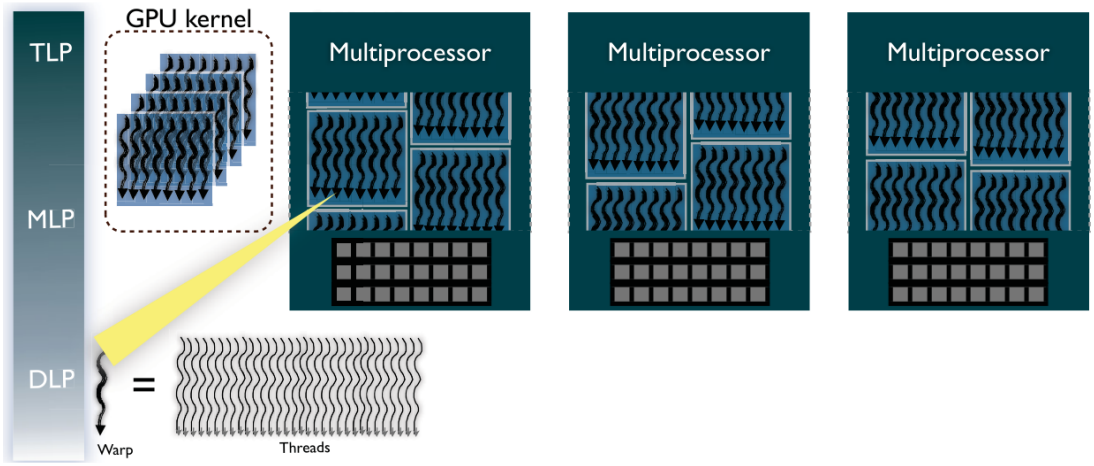


图 4.6　GPGPU 内核中的多级并行性：线程级、内存级、数据级与指令级并行性。

间，而后者是计算核心所需的所有数据流入和流出内存所需的时间。可以将此框架视为将第 2 章的平衡原则框架专门化用于 GPGPU，并采用了细化的处理器模型。

4.2.2　工作流图

为了识别导致计算不平衡的瓶颈，Baghsorkhi 等人结合了内核中可用的并发量（TLP、DLP 和 ILP）——考虑了共存线程的影响——以及 SIMD 流水线和内存系统的延迟，从而保留了不同性能因素的交互效应 [11]。为此，他们引入了工作流图（WFG）。WFG 是 GPGPU 内核控制流图的扩展。WFG 中的节点是长延迟（全局）内存操作、低延迟（暂存器）内存操作、屏障同步、n 条连续计算指令的块，或合成的入口和出口节点。除了对应于控制流图的边（转移弧）之外，WFG 还包含连接全局内存加载与其对应使用的数据依赖弧。

初始 WFG 抽象了计算内核，与底层硬件无关。最初没有为转移弧分配权重。转移弧仅指示目标节点可以在源节点之后立即执行。WFG 表示 GPGPU 内核中的平均 warp。在分支发散的情况下，不同的线程可能遵循不同的控制流路径。因此，应为发散的控制流路径分配适当的权重，以指示执行每条路径的 warp 比例。类似地，如果这些发散路径上存在长延迟或低延迟内存访问，则应根据发出内存指令的 warp 比例和线程模式来调整内存事务数或体冲突串行化延迟。

为了估计特定 GPU 上的性能，Baghsorkhi 等人提出基于硬件参数（如内存带宽、读取延迟、内存合并规则、内存体配置、SIMD 工作粒度、SIMD 引擎宽度和流水线延迟等因素），通过符号评估和运行时检查 GPGPU 内核收集的信息来专门化 WFG 中的弧。得到的 WFG 将 GPGPU 内核代码剖面

绑定到 GPU 硬件配置。在针对特定硬件专门化 WFG 之后，分配给每个 WFG 弧的权重表示以 warp 为粒度平均执行源节点处的指令所需的周期数。

4.2.3 随机内存层次模型

图形处理器针对吞吐量导向的工作负载进行了优化，这使得早期的 GPU 可以省略传统的数据缓存。较新的 GPU 增加了小型每核 L1 缓存以捕获线程间重用，以及更大的统一 L2 缓存以利用核间共享。这是向前迈出的重要一步，可以更好地支持更多样化的工作负载，并减少以前存在的一些性能差异。然而，大多数 GPGPU 应用的性能强烈依赖于内存子系统的有效使用。当前一代 GPU 提供了一组性能计数器，用于收集 L1 和 L2 缓存的总体未命中数等统计数据，但计数器值无法在程序执行期间进行监控或采样。此外，可以在提供的插桩接口之上构建基于时间的采样，以识别程序中的热点。虽然高采样率可能会扭曲剖析数据的准确性，但在存在数千个并发运行线程的情况下，低采样率下也难以获得高分辨率的性能信息。为了克服这个问题，Baghsorkhi 等人提出了一种新的基于软件的方法，用于监控高度多线程的通用图形处理器中的内存层次性能 [12]。

所提出的分析基于为应用程序执行快照收集的内存跟踪。内存跟踪通过在源代码级别对 GPU 内核进行插桩来收集，以记录内核执行期间访问的内存地址。插桩框架设计为源到源转换模块。如图 4.1 所示，在 GPU 内核源代码中插入静态探针，以记录 GPU 内核执行时活动线程对一部分线程块读取和写入的内存地址。

为了理解 GPU 内核相对于内存层次的性能，有必要收集足够的跟踪以捕获内存操作的线程内和线程间交互。为了捕获线程间干扰，跟踪收集的执行快照在水平方向上扩展。他们收集可能在不同核心（流多处理器）上紧密调度的线程块的跟踪。为了考虑线程内交互，执行快照被扩展，以便在单个线程内收集足够大的跟踪。如果需要，执行快照会跨越在单个核心上背靠背调度的多个线程块的边界进行扩展。通过这种方法，收集了内存跟踪的一个子集，但该子集足够详细，可以反映并发运行线程组内的局部性和交互。

收集的跟踪展示了内存引用的精确线程束内（线程内）顺序。但跟踪不保留来自不同线程束或线程块的内存引用的相对顺序的任何信息，因为他们的方法不依赖于内存的执行顺序

代码清单 4.1：插桩的 SpMV —设备代码

```
__global__ void SpMV(_sampling_device_buffer *
    _samples, float *x, const float *val, ...)
{
    _sampling_status _this_thread_status;
    _init_sampling_status(samples, &this_thread_status);
    tid = threadIdx.y;
    bid = blockIdx.y;
    t=0;
    myi = bid * BLOCKSIZE + tid;

    if ( myi < (numRows )){
```

```
_sample_mem_index(samples, &this_thread_status, 0,
&(rowInd[myi]));
lb = rowInd[myi];
_sample_mem_index(samples, &this_thread_status, 2,
&(rowInd[myi + 1]));
ub = rowInd[myi+1];

for (j=lb; j<ub; j++) { ...
```

在收集跟踪时加载和存储。不依赖于插桩 GPU 内核执行期间顺序的理由是，向内核源代码添加静态探针：

1. 改变内核资源使用（寄存器数量），这可能因此改变每个流多处理器上并发活动线程块的数量；
2. 增加在途内存操作的数量，这将扭曲缓存的状态和内存层次中的拥塞程度；以及
3. 改变 GPU 内核的指令混合，并引入虚假的同步或停顿点。因此，对内核进行插桩将扭曲内存操作的执行顺序。

为了解决这个问题，通过蒙特卡洛方法重建到达内存层次各级别的内存请求的顺序，这是一种对于个体行为高度耦合的系统的高效采样方法。在每次运行中，根据随机采样的顺序，将跟踪驱动到缓存模拟器中内存层次的每个级别，具体步骤如下。

1. 给定在内存层次每个级别等待服务的内存请求池：
   (a) 从可用内存请求池中生成有效的随机顺序。
   (b) 按照步骤（a）中得出的顺序将跟踪驱动到当前内存层次模块；以及
   (c) 获得当前级别的性能估计，并准备由下一个内存层次级别服务的内存请求池。
2. 重复步骤 1 足够大的次数。
3. 使用直方图确定结果的概率分布，并总结预测的置信度。

上述模型的输出是内存系统的概率性能行为，例如第一级和第二级缓存的命中率。图 4.7 显示了通过上述方法计算的 SpMV 内核的 L2 缓存命中率分布。如果某些性能行为最频繁地被观察到——即使对线程间内存请求的确切相对顺序了解有限——它们也是系统性能的统计上可靠的表示。如果不同的随机顺序导致明显不同的性能统计数据（直方图分布广泛），那么预测就不可靠。否则，尽管 Baghsorkhi 等人 [12] 在将跟踪驱动到模拟器时没有遵循实际的线程间顺序，但他们建立了一个与实际情况相似且足够接近的执行上下文。

Bahsorkhi 等人 [12] 还传播了内存加载和存储的源代码位置。因此，他们为 GPU 内核源代码中的单个内存操作生成了精确且高分辨率的剖析统计数据。

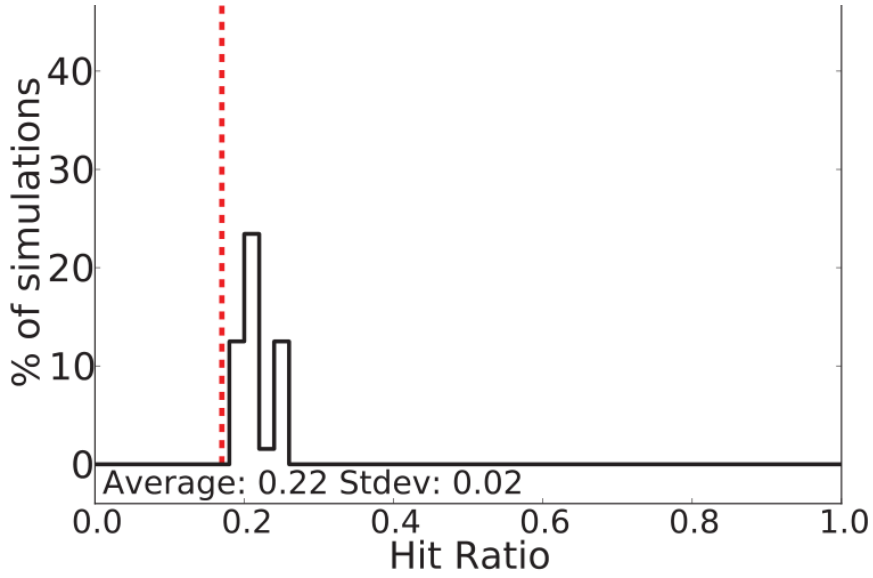


图 4.7 稀疏矩阵—向量乘法内核的 L2 读命中率概率分布（横轴表示命中率，纵轴表示该结果出现的频率）。

4.2.4 屋顶线模型

Williams 等人提出了屋顶线模型，该模型有助于可视化计算受限或内存受限的多核架构 [97]。[2] 屋顶线根据内核的操作强度设定了内核性能的上限。当操作强度作为列达到屋顶时，要么触及屋顶的平坦部分，意味着性能受计算限制，要么性能最终受内存限制。图 4.8 展示了不同架构的屋顶线示例。公式（4.31）表示该模型：

$$\text{可达到的 Gflop / s} = \min(\text{峰值 Gflop / s}, \text{流 BW} \times \text{实际 flop} : \text{字节比率}). \quad (4.31)$$

屋顶线不仅限于表征峰值性能。人们可以将特定的架构特征映射到峰值屋顶线以下的其他屋顶和脊线上，从而直观地理解改变操作强度以及采用特定架构特征对性能的影响。

4.2.5 使用 Ocelot 对 CUDA 工作负载进行性能分析与剖析

——由 Andrew Kerr 撰写——GPU

Ocelot 是一个开源的动态 JIT 编译框架，面向 GPU 计算应用，支持多种 GPU 和非 GPU 执行目标。Ocelot 通过自定义实现的 CUDA 运行时 API 支持未经修改的 CUDA 应用程序。在内部，Ocelot 解析结构化为 PTX 模块的 CUDA 内核。PTX 是 NVIDIA 的并行线程执行（PTX）虚拟指令集，用于提供设备无关的程序表示，捕获 CUDA 的单指令多线程（SIMT）执行模型中显式并行的语义。Ocelot 支持多个后端执行目标——PTX 模拟器、NVIDIA GPU、AMD GPU，以及一个到 LLVM 的转换器，用于在多核 CPU 上高效执行 GPU 内核。设备可移植性如图 4.9 所示。通过 GPU Ocelot 执行 CUDA 应用

[2] 在性能分析背景下绘制的第一个原始”屋顶线图”出现在 Hockney 和 Curington [49] 的图 2 中。

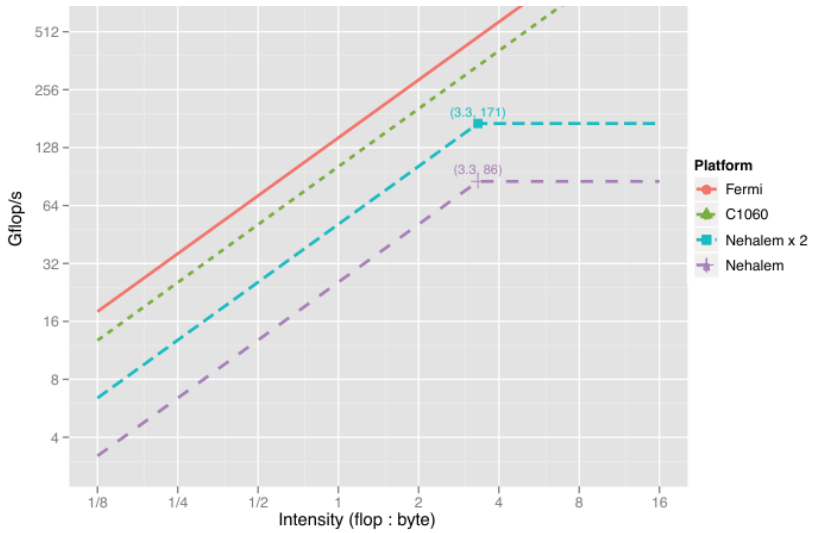


图 4.8　Roofline（屋顶线）性能模型示例。

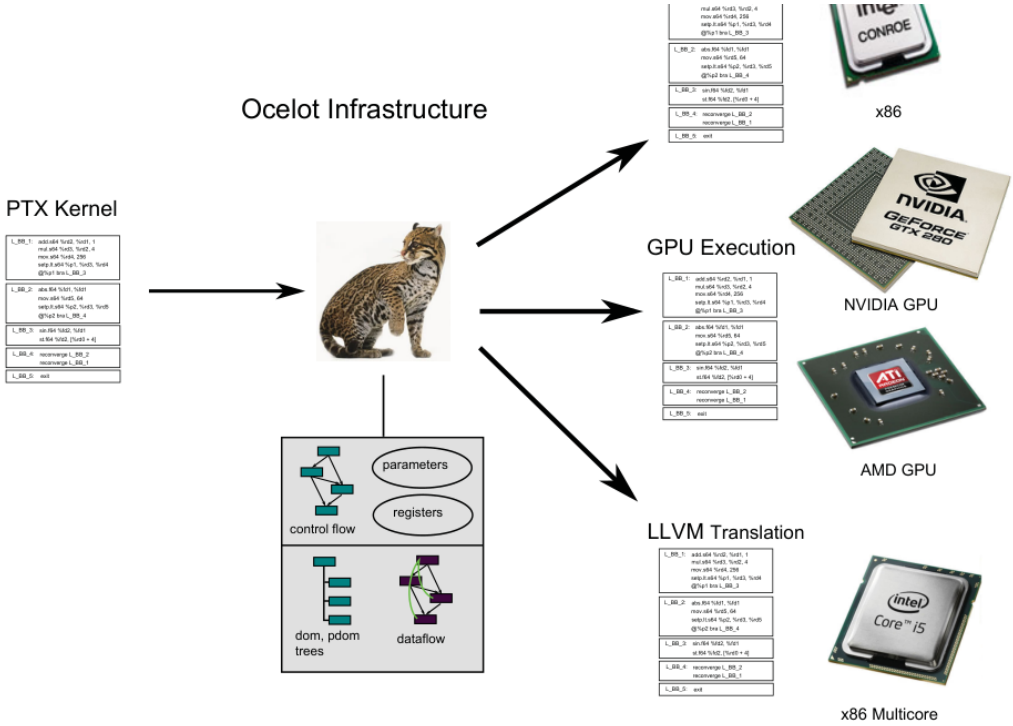


图 4.9　GPU Ocelot 的动态编译与执行框架。

程序，可以利用这些设备后端来增强对 CUDA 工作负载的分析、剖析和模拟，以理解性能特征和瓶颈，并应用优化以提升原生 GPU 执行目标上的性能。PTX 模拟与跟踪分析 GPU Ocelot 的 PTX 模拟器在 PTX 级别执行 CUDA 内核，并为每条动态执行的指令提供 GPU 的完整架构状态。事件跟踪分析器被编写来处理执行事件流，以创建用户定义的跟踪生成器，这些生成器在程序执行时对动态指令跟踪做出反应，从而实现实时工作负载表征和正确性检查。Ocelot 目前拥有超过十几个事件跟踪分析器，提供对内存访问检查、竞争检测、交互式调试器以及用于执行调优的反馈的支持。

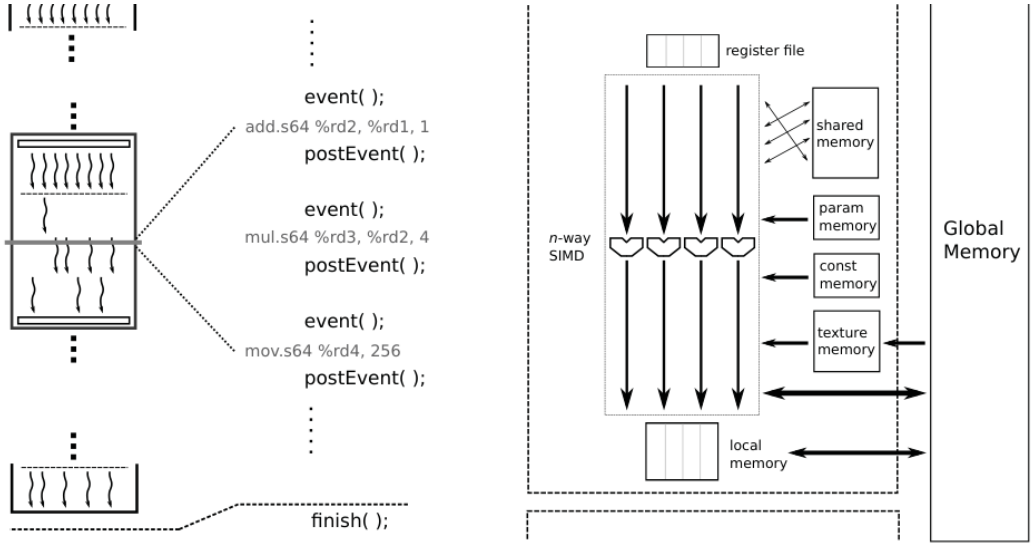


图 4.10 带抽象机器模型的 PTX 仿真器追踪生成框架。

图 4.10 展示了针对给定动态指令流调用跟踪生成器事件处理器的顺序。当内核启动时，所有已注册的跟踪生成器都会被调用，使它们能够检查内存分配表、分析内核的内部表示并观察参数值。每条指令执行时，跟踪生成器在模拟器更新其架构状态之前和之后被调用。当内核退出时，再次调用跟踪生成器以执行最终分析，并可能将结果写入外部数据存储。GPU Ocelot 支持创建和应用多种分析工具。它们剖析内核执行并以热图形式提供可视化反馈，说明程序的哪一部分主导执行时间。这些区域内的代码相关统计数据向程序员指示潜在的瓶颈。它们还可以评估计算与内存传输的比率，并确定内核是确定的内存受限还是确定的计算受限。

工作负载表征的一个常见目的是识别内核中的计算密集型区域，并理解低效执行的潜在原因。几个感兴趣的指标包括 SIMD 利用率、内存效率、存储体冲突和低计算指令密度。测量应用程序行为为程序员提供执行优化的反馈。例如，他们可以重构内核的控制行为以提高 SIMD 利用率，通过倾斜静态内存分配来减少共享内存的存储体冲突数量，或修改线程访问模式。计算强度可以通过其他优化（如循环展开、指针运算和屏障消除）来提高。这些优化对动态指令数的影响可以被精确测量和可视化，如图 4.11 所示。插桩 GPU Ocelot 还定义了一个接口，用于实现 PTX 转换遍，能够在 JIT 编译内核以在 GPU 或其他 Ocelot 设备后端执行之前，将插桩代码插入内核中。此外，可以向 Ocelot 注册回调，以构建、初始化并分析与转换遍相关的数据结构。这使得

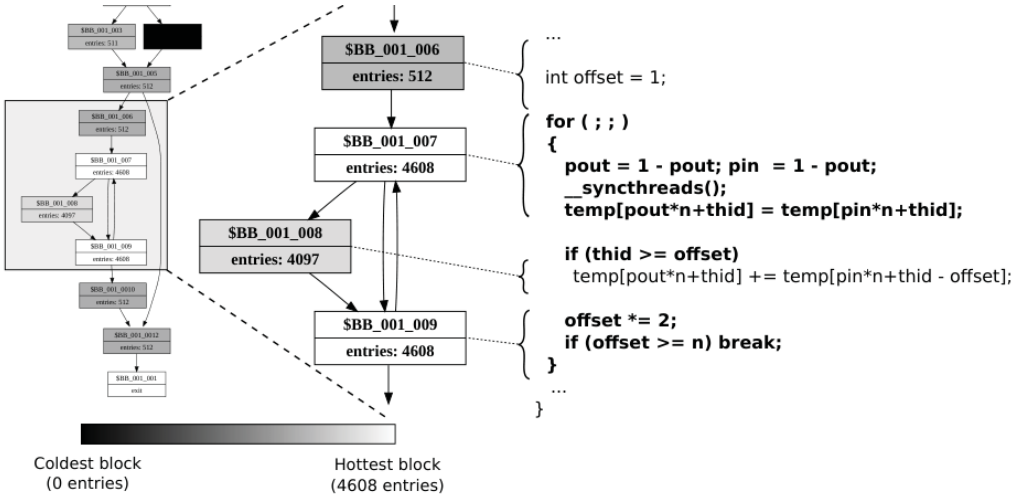


图 4.11　CUDA SDK scan 应用的热点区域分析图（原图中的 Hot Region 表示热点区域）。

GPU 内核的在线动态插桩成为可能，在评估相同分析指标时，相比 Ocelot 的模拟器可提供显著的加速。使用 Ocelot 的 PTX IR 构建转换可能很繁琐，因此开发了一个自定义的 C 到 PTX 编译工具来简化过程。这依赖于 C 代码中的枚举类型和标签来告知编译器在内核中的何处放置插桩探针。例如，可以将插桩代码块插入原始内核的每个基本块中，或仅插入循环头部。内置函数支持跨共享内存缓冲区的无分歧归约，因此可以观察和测量与基于 warp 的 SIMT 执行模型相关的行为。

备注　GPU Ocelot 以新 BSD 许可证开源；原著给出的项目地址为 `https://code.google.com/p/gpuocelot/`。其 PTX 仿真器设备后端提供 CUDA 工作负载的非常详细的指令和内存跟踪，可用于直接剖析应用程序或驱动其他分析工具，如跟踪驱动的时序模拟器。Ocelot 中集成了几个正确性验证工具示例，默认启用，可检测内核中的同步和越界错误，其他工具则提供详细的剖析结果。PTX 插桩可在原生 GPU 设备上执行时实现类似功能，而无需修改原始应用程序。Ocelot 对 CUDA 运行时 API 的完整实现、丰富的 PTX 分析通道集以及内核转换通道管理器，为开发针对 GPU 计算工作负载的额外剖析和分析工具提供了强大的平台。

4.2.6　其他 GPGPU 性能建模技术与工具

Zhang 和 Owens 提出了一个性能模型，他们测量了指令流水线、共享内存和全局内存上花费的执行时间，以找出瓶颈 [106]。他们也针对瓶颈识别，但其方法并未直接估计性能收益。在 GPGPU 应用程序的优化和调优方面已有大量工作 [24, 34, 68, 76, 84]。Ryoo 等人 [84] 引入了两个度量标准，通过计算 GPGPU 应用程序的利用率和效率来修剪优化空间。Choi 等人提出了一个稀疏矩阵向量乘法（SpMV）内核的性能模型，用于自动调优 [24]。

CUDA 性能分析工具 有许多工具可用于分析 CUDA 应用程序的性能。目前, GPU 提供的硬件性能计数器越来越多。Kim 和 Shrivastava [58] 提出了一个工具，可用于分析 CUDA 程序的内存访问模式。他们对内存合并和存储体冲突等主要内存效应进行了建模。这是编译时分析，缺乏动态信息。Meng 等人 [67] 提出了一个 GPGPU 性能预测框架。给定 CPU 代码骨架，该框架预测 GPGPU 加速的成本和收益。还有可用于性能分析的 GPU 模拟器。Bakhoda 等人 [13] 实现了一个 GPU 模拟器（GPGPU-Sim），并使用模拟输出分析了 CUDA 应用程序的性能。Collange 等人 [27] 开发的一个名为 Barra 的 G80 功能模拟器可以在收集统计数据的同时执行 NVIDIA CUBIN 文件。GPGPU-Sim [13] 已被广泛用于研究 CUDA 工作负载特性。最近，出现了许多 CPU+GPU 模拟器: Multi2sim [90]、FusionSim [104]、MV5 [45] 和 MacSim [1]。这些都是周期级模拟器, 可以提供性能甚至功耗行为的详细信息。在 OpenCL 中，ATI Stream Profiler [77]、NVIDIA 的 Parallel Nsight [71] 和 Visual Profiler 等几个工具提供 OpenCL 程序的统计数据。最近发布的 Intel OpenCL SDK [52] 还包括用于 OpenCL 内核的分析工具，如 Graphics Performance Analyzer 和 VTune Amplifier XE 2011。程序员可以使用这些工具将内核的执行数据存储到跟踪文件中进行离线分析，并分析汇编内核代码。

参考文献

注：文献的正式英文论文题名、期刊名及会议名保留原文，便于学术检索。

[1] MacSim. https://code.google.com/p/macsim/. 引用页码 70

[2] Umut A. Acar, Guy E. Blelloch, 和 Robert D. Blumofe. 工作窃取的数据局部性. 第十二届 ACM 并行算法与架构年度研讨会论文集 - SPAA '00, 第 1–12 页, 美国纽约州纽约市, 2000 年 7 月. ACM Press. DOI: 10.1145/341800.341801 引用页码 34

[3] Advanced Micro Devices, Inc. AMD Brook+. https://ati.amd.com/technology/streamcomputing/AMD-Brookplus.pdf. 引用页码 1, 23

[4] Alok Aggarwal 和 S. Vitter, Jeffrey. 排序及相关问题的输入/输出复杂性. ACM 通讯, 31（9）: 1116–1127, 1988 年 8 月. DOI: 10.1145/48529.48535 引用页码 25, 30

[5] Randy Allen 和 Ken Kennedy. 现代架构的优化编译器. Morgan-Kaufmann 出版社, 美国加利福尼亚州旧金山, 2002. 引用页码 45

[6] AMD. Fusion. https://sites.amd.com/us/fusion/apu/Pages/fusion.aspx. DOI: 10.2307/3974600 引用页码 23

[7] Gene M. Amdahl. 单处理器方法实现大规模计算能力的有效性. AFIPS 联合计算机会议论文集, 第 30 卷, 第 483–485 页, 美国新泽西州大西洋城, 1967 年 4 月. DOI: 10.1145/1465482.1465560 引用页码 27

[8] Lars Arge, Michael T. Goodrich, Michael Nelson, 和 Nodari Sitchinava. 私有缓存芯片多处理器的基本并行算法. 第二十届并行算法与架构年度研讨会论文集 - SPAA '08, 第 197 页, 美国纽约州纽约市, 2008. ACM Press. DOI: 10.1145/1378533.1378573 引用页码 33

[9] Nitin Arora, Ryan P. Russell, 和 Richard W. Vuduc. 数值优化的快速灵敏度计算. AAS/ALAA 航天动力学专家会议论文集, AAS 09-435, 美国宾夕法尼亚州匹兹堡, 2009 年 8 月. 引用页码 45

[10] Rachata Ausavarungnirun, Kevin Kai-Wei Chang, Lavanya Subramanian, Gabriel H. Loh, 和 Onur Mutlu. 分阶段内存调度：在异构系统中实现高性能和可扩展性. 第 39 届国际计算机架构研讨会论文集, ISCA '12, 第 416–427 页, 美国新泽西州皮斯卡塔韦, 2012. IEEE Press. DOI: 10.1109/ISCA.2012.6237036 引用页码 24

[11] Sara S. Baghsorkhi, Matthieu Delahaye, Sanjay J. Patel, William D. Gropp, and Wen-mei W. Hwu. An adaptive performance modeling tool for gpu architectures. In PPoPP, 2010. DOI: 10.1145/1693453.1693470

[12] Sara S. Baghsorkhi, Isaac Gelado, Matthieu Delahaye, and Wen-mei W. Hwu. Efficient performance evaluation of memory hierarchy for highly multithreaded graphics processors. In Proceedings of the 17th ACM SIGPLAN symposium on Principles and Practice of Parallel Programming, PPoPP '12, pages 23–34, New York, NY, USA, 2012. ACM. DOI: 10.1145/2145816.2145820

[13] A. Bakhoda, G.L. Yuan, W.W.L. Fung, H. Wong, and T.M. Aamodt. Analyzing cuda workloads using a detailed gpu simulator. In Performance Analysis of Systems and Software, 2009. ISPASS 2009. IEEE International Symposium on, pages 163–174, April 2009. DOI: 10.1109/ISPASS.2009.4919648

[14] G.E. Blelloch, R.A. Chowdhury, P.B. Gibbons, V Ramachandran, S Chen, and M Kozuch. Provably good multicore cache performance for divide-and-conquer algorithms. In Proceedings of the nineteenth annual ACM-SIAM symposium on Discrete algorithms, volume pp, pages 501–510. Society for Industrial and Applied Mathematics, 2008. DOI: 10.1145/1347082.1347137

[15] Guy E. Blelloch. Programming parallel algorithms. Communications of the ACM, 39(3):85–97, March 1996. DOI: 10.1145/227234.227246

[16] Guy E. Blelloch, Phillip B. Gibbons, and Harsha Vardhan Simhadri. Low depth cache-oblivious algorithms. In Proceedings of the 22nd ACM symposium on Parallelism in algorithms and architectures - SPAA '10, page 189, New York, New York, USA, June 2010. ACM Press. DOI: 10.1145/1810479.1810519

[17] W.J. Bouknight, S.A. Denenberg, D.E. McIntyre, J.M. Randall, A.H. Sameh, and D.L. Slotnick. The illiac iv system. Proceedings of the IEEE, 60(4):369–388, April 1972. DOI: 10.1109/PROC.1972.8647

[18] George E.P. Box, J. Stuart Hunter, and William G. Hunter. Statistics for Experimenters: {Design}, Innovation, and Discovery. Wiley-Interscience, 2nd edition, 2005.

[19] Richard P. Brent. The Parallel Evaluation of General Arithmetic Expressions. Journal of the ACM, 21(2):201–206, April 1974. DOI: 10.1145/321812.321815

[20] N. Brunie, S. Collange, and G. Diamos. Simultaneous branch and warp interweaving for sustained gpu performance. In Computer Architecture (ISCA), 2012 39th Annual International Symposium on, pages 49–60, June 2012.

[21] David Callahan, John Cocke, and Ken Kennedy. Estimating interlock and improving balance for pipelined architectures. Journal of Parallel and Distributed Computing, 5(4):334–358, 1988. DOI: 10.1016/0743-7315(88)90002-0.

[22] Aparna Chandramowlishwaran, Kamesh Madduri, and Richard Vuduc. Diagnosis, tuning, and redesign for multicore performance: A case study of the fast multipole

method. In Proc. ACM/IEEE Conf. Supercomputing (SC), New Orleans, LA, USA, November 2010. DOI: 10.1109/SC.2010.19

[23] Aparna Chandramowlishwaran, Samuel Williams, Leonid Oliker, Ilya Lashuk, George Biros, and Richard Vuduc. Optimizing and tuning the fast multipole method for state-of-the-art multicore architectures. In Proc. IEEE Int'l. Parallel and Distributed Processing Symp. (IPDPS), Atlanta, GA, USA, April 2010. DOI: 10.1109/IPDPS.2010.5470415

[24] Jee W. Choi, Amik Singh, and Richard W. Vuduc. Model-driven autotuning of sparse matrix-vector multiply on gpus. In PPoPP, 2010. DOI: 10.1145/1837853.1693471

[25] Rezaul Alam Chowdhury, Francesco Silvestri, Brandon Blakeley, and Vijaya Ramachandran. Oblivious algorithms for multicores and network of processors. In 2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS), pages 1–12. IEEE, 2010. DOI: 10.1109/IPDPS.2010.5470354

[26] George Chrysos. Intel (R) xeon phi TM coprocessor (codename knights corner). http://www.slideshare.net/IntelXeon/under-the-armor-of-knights-corner-intel-mic-architecture-at-hotchips-2012, 2012.

[27] Sylvain Collange, Marc Daumas, David Defour, and David Parello. Barra: A parallel functional simulator for gpgpu. Modeling, Analysis, and Simulation of Computer Systems, International Symposium on, 0:351–360, 2010. DOI: 10.1109/MASCOTS.2010.43

[28] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to algorithms. MIT Press, 3rd edition, 2009.

[29] J.-L. Cruz, A. Gonzalez, M. Valero, and N.P. Topham. Multiple-banked register file architectures. In Computer Architecture, 2000. Proceedings of the 27th International Symposium on, pages 316–325, June 2000. DOI: 10.1145/342001.339708

[30] Kenneth Czechowski, Casey Battaglino, Chris McClanahan, Aparna Chandramowlishwaran, and Richard Vuduc. Balance principles for algorithm-architecture co-design. In Proc. USENIX Wkshp. Hot Topics in Parallelism (HotPar), Berkeley, CA, USA, May 2011.

[31] Wen-mei Hwu David Kirk. Programming Massively Parallel Processors: A Hands-on Approach, Second Edition. Morgan Kaufmann, 2012.

[32] Gregory Diamos, Benjamin Ashbaugh, Subramaniam Maiyuran, Andrew Kerr, Haicheng Wu, and Sudhakar Yalamanchili. Simd re-convergence at thread frontiers. In Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-44 '11, pages 477–488, New York, NY, USA, 2011. ACM. DOI:

10.1145/2155620.2155676

[33] Gregory Diamos, Andrew Kerr, Sudhakar Yalamanchili, and Nathan Clark. Ocelot: A dynamic compiler for bulk-synchronous applications in heterogeneous systems. In PACT-19, 2010. DOI: 10.1145/1854273.1854318x, 6, 66

[34] Yuri Dotsenko, Sara S. Baghsorkhi, Brandon Lloyd, and Naga K. Govindaraju. Auto-tuning of fast fourier transform on graphics processors. In Proceedings of the 16th ACM symposium on Principles and practice of parallel programming, PPOPP '11, pages 257–266, New York, NY, USA, 2011. ACM. DOI: 10.1145/1941553.1941589

[35] Marco Fillo, Stephen W. Keckler, William J. Dally, Nicholas P. Carter, Andrew Chang, Yevgeny Gurevich, and Whay S. Lee. The M-Machine multicomputer. In Proceedings of the 28th annual international symposium on Microarchitecture, pages 146–156, 1995. DOI: 10.1109/MICRO.1995.476822

[36] Matteo Frigo, C.E. Leiserson, Harald Prokop, and Sridhar Ramachandran. Cache-oblivious algorithms. In 40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039), pages 285–297, New York, NY, USA, October 1999. IEEE Comput. Soc. DOI: 10.1109/SFFCS.1999.814600

[37] Matteo Frigo and Volker Strumpen. Cache oblivious stencil computations. Proceedings of the 19th annual international conference on Supercomputing ICS 05, 1(212):361, 2005. DOI: 10.1145/1088149.1088197

[38] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. Dynamic warp formation and scheduling for efficient gpu control flow. In Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 40, pages 407–420, Washington, DC, USA, 2007. IEEE Computer Society. DOI: 10.1109/MICRO.2007.12

[39] Wilson W. L. Fung, Ivan Sham, George Yuan, and Tor M. Aamodt. Dynamic warp formation: Efficient mimd control flow on simd graphics hardware. ACM Trans. Archit. Code Optim., 6(2):7:1–7:37, July 2009.

[40] W. W. L. Fung and T. M. Aamodt. Thread block compaction for efficient SIMT control flow. In IEEE 17th International Symposium on High Performance Computer Architecture, pp. 25–36, 2011. DOI: 10.1109/HPCA.2011.5749714.

[41] Mark Gebhart, Daniel R. Johnson, David Tarjan, Stephen W. Keckler, William J. Dally, Erik Lindholm, and Kevin Skadron. Energy-efficient mechanisms for managing thread context in throughput processors. In Proceedings of the 38th annual international symposium on Computer architecture, ISCA '11, pages 235–246, New York, NY, USA, 2011. ACM. DOI: 10.1145/2000064.2000093

[42] Alexander G. Gray and Andrew W. Moore. “N-Body” problems in statistical learning. In Proc. Advances in Neural Information Processing Systems (NIPS), Vancouver, British Columbia, Canada, December 2000.

[43] L Greengard and V Rokhlin. A fast algorithm for particle simulations. Journal of Computational Physics, 73(2):325–348, December 1987. DOI: 10.1016/0021-9991(87)90140-9

[44] Leslie Greengard and Vladimir Rokhlin. A new version of the Fast Multipole Method for the Laplace equation in three dimensions. Acta Numerica, 6:229, November 2008. DOI: 10.1017/S0962492900002725

[45] LAVA group. The MV5 simulator. https://sites.google.com/site/mv5sim/benchmarks.

[46] T.D. Han and T.S. Abdelrahman.

[47] Mark D. Hill and Michael R. Marty. Amdahl’s Law in the Multicore Era. Computer, 41(7):33–38, July 2008. DOI: 10.1109/MC.2008.209

[48] W. Daniel Hillis. Balancing a Design. IEEE Spectrum, 1987.

[49] Roger W. Hockney and Ian J. Curington. f1/2: A parameter to characterize memory and communication bottlenecks. Parallel Computing, 10(3):277–286, May 1989. DOI: 10.1016/0167-8191(89)90100-2

[50] Sunpyo Hong and Hyesoon Kim. An analytical model for a gpu architecture with memory-level and thread-level parallelism awareness. In ISCA, 2009. DOI: 10.1145/1555754.1555775

[51] Wenmei W. Hwu and John Stone. A programmerâ€™s view of the new GPU computing capabilities in the Fermi architecture and CUDA 3.0. White paper, University of Illinois, 2009.

[52] Intel Corporation. Intel OpenCL SDK. http://software.intel.com/en-us/articles/intel-opencl-sdk/.

[53] Dror Irony, Sivan Toledo, and Alexander Tiskin. Communication lower bounds for distributed-memory matrix multiplication. J. Parallel Distrib. Comput., 64:1017–1026, 2004. DOI: 10.1016/j.jpdc.2004.03.021

[54] Joseph JàJà. Introduction to parallel algorithms. Addison-Wesley, 1992.

[55] Min Kyu Jeong, Chander Sudanthi, Nigel Paver, and Mattan Erez. A qos-aware memory controller for dynamically balancing gpu and cpu bandwidth use in an mpsoc. In the Proceedings of the 2012 Design Automation Conference (DAC-12), June 2012. DOI: 10.1145/2228360.2228513

[56] Hong Jia-Wei and H. T. Kung. I/O complexity: The red-blue pebble game. In Proceedings of the thirteenth annual ACM symposium on Theory of computing - STOC '81, pages 326–333, New York, New York, USA, May 1981. ACM Press. DOI: 10.1145/800076.802486

[57] David Kanter. Inside fermi: Nvidia TM s hpc push. http://www.realworldtech.com/fermi/6/.

[58] Yooseong Kim and Aviral Shrivastava. Cumpaz: A tool to analyze memory access patterns in cuda. In DAC '11: Proc. of the 48th conference on Design automation, June 2011. DOI: 10.1145/2024724.2024754

[59] H. T. Kung. Memory requirements for balanced computer architectures. In Proceedings of the ACM Int'l. Symp. Computer Architecture (ISCA), Tokyo, Japan, 1986.

[60] Ilya Lashuk, Aparna Chandramowlishwaran, Harper Langston, Tuan-Anh Nguyen, Rahul Sampath, Aashay Shringarpure, Richard Vuduc, Lexing Ying, Denis Zorin, and George Biros. A massively parallel adaptive fast multipole method on heterogeneous architectures. In Proc. ACM/IEEE Conf. Supercomputing (SC), Portland, OR, USA, November 2009. DOI: 10.1145/1654059.1654118

[61] Jaekyu Lee and Hyesoon Kim. TLP aware cache management schemes in a cpu-gpu heterogeneous architecture. In HPCA-18, 2012.

[62] Adam Levinthal and Thomas Porter. Chap - a simd graphics processor. In Proceedings of the 11th annual conference on Computer graphics and interactive techniques, SIGGRAPH '84, pages 77–82, New York, NY, USA, 1984. ACM. DOI: 10.1145/964965.808581

[63] Samuel Liu, John Erik Lindholm, Ming Y. Siu, Brett W. Coon, and Stuart F. Oberman. Operand collector architecture. U.S. Patent Number 7,834,881, 2010.

[64] William R. Mark, R. Steven Glanville, Kurt Akeley, and Mark J. Kilgard. Cg: A system for programming graphics hardware in a C-like language, 2003. DOI: 10.1145/1201775.882362.

[65] John McCalpin. Memory Bandwidth and Machine Balance in High Performance Computers. IEEE Technical Committee on Computer Architecture (TCCA) Newsletter, December 1995.

[66] Michael D. McCool, Zheng Qin, and Tiberiu S. Popa. Shader metaprogramming. In Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware, HWWS '02, pages 57–68, Aire-la-Ville, Switzerland, Switzerland, 2002. Eurographics Association.

[67] Jiayuan Meng, Vitali Morozov, Kalyan Kumaran, Venkatram Vishwanath, and

Thomas Uram. Grophecy: GPU performance projection from cpu code skeletons. In SC'11, February 2011. DOI: 10.1145/2063384.2063402

[68] Jiayuan Meng and Kevin Skadron. Performance modeling and automatic ghost zone optimization for iterative stencil loops on gpus. In ICS, 2009. DOI: 10.1145/1542275.1542313

[69] Jiayuan Meng, David Tarjan, and Kevin Skadron. Dynamic warp subdivision for integrated branch and memory divergence tolerance. In ISCA-32, 2010. DOI: 10.1145/1816038.1815992

[70] Veynu Narasiman, Michael Shebanow, Chang Joo Lee, Rustam Miftakhutdinov, Onur Mutlu, and Yale N. Patt. Improving gpu performance via large warps and two-level warp scheduling. In Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-44 '11, pages 308–317, New York, NY, USA, 2011. ACM. DOI: 10.1145/2155620.2155656

[71] NVIDIA. NVIDIA Parallel Nsight. http://developer.nvidia.com.

[72] NVIDIA Corporation. CUDA Programming Guide, V4.0.

[73] NVIDIA Corporation. CUDA Toolkit, 2012. Version 4.2 as of Sep. 2012, http://developer.nvidia.com/cuda/cuda-downloads.

[74] CORPORATE OpenGL Architecture ReviewBoard. OpenGL reference manual: the official reference document for OpenGL, release 1. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1992.

[75] Matt Pharr and Randima Fernando. GPU Gems 2. Addison-Wesley Professional, 2005.

[76] Louis-Noël Pouchet, Uday Bondhugula, Cédric Bastoul, Albert Cohen, J. Ramanujam, and P. Sadayappan. Combined iterative and model-driven optimization in an automatic parallelization framework. In SC '10, pages 1–11, Washington, DC, USA, 2010. IEEE Computer Society. DOI: 10.1109/SC.2010.14

[77] Budirijanto Purnomo, Norman Rubin, and Michael Houston. ATI Stream profiler: a tool to optimize an opencl kernel on ati radeon gpus. In ACM SIGGRAPH 2010 Posters, SIGGRAPH '10, pages 54:1–54:1, New York, NY, USA, 2010. ACM. DOI: 10.1145/1836845.1836904

[78] Abtin Rahimian, Ilya Lashuk, Aparna Chandramowlishwaran, Dhairya Malhotra, Logan Moon, Rahul Sampath, Aashay Shringarpure, Shravan Veerapaneni, Jeffrey Vetter, Richard Vuduc, Denis Zorin, and George Biros. Petascale direct numerical simulation of blood flow on 200k cores and heterogeneous architectures. In Proc. ACM/IEEE Conf. Supercomputing (SC), New Orleans, LA, USA, November 2010.

DOI: 10.1109/SC.2010.42

[79] Minsoo Rhu and M. Erez. Capri: Prediction of compaction-adequacy for handling control-divergence in gpgpu architectures. In Computer Architecture (ISCA), 2012 39th Annual International Symposium on, 2012. DOI: 10.1145/2366231.2337167

[80] Michael D. Root and James R. Boer. Directx Complete. McGraw-Hill, Inc., New York, NY, USA, 1st edition, 1998.

[81] R. M. Russell. The CRAY-1 computer system. Communications of the ACM, 21(1):63–72, January 1978. DOI: 10.1145/359327.359336

[82] Shane Ryoo, Christopher I. Rodrigues, Sara S. Baghsorkhi, Sam S. Stone, David B. Kirk, and Wen mei W. Hwu. Optimization principles and application performance evaluation of a multithreaded GPU using CUDA. In Proc. of the 13th ACM SIGPLAN Symp. on Principles and Practice of Parallel Programming, 2008.

[83] Shane Ryoo, Christopher I. Rodrigues, Sam S. Stone, Sara S. Baghsorkhi, Sain-Zee Ueng, and Wen mei W. Who. Program optimization study on a 128-core gpu. In GPGPU-1, 2007.

[84] Shane Ryoo, Christopher I. Rodrigues, Sam S. Stone, Sara S. Baghsorkhi, Sain-Zee Ueng, John A. Stratton, and Wen mei W. Hwu. Program optimization space pruning for a multi-threaded gpu. In CGO-6, pages 195–204, 2008. DOI: 10.1145/1356058.1356084

[85] Jae Woong Sim, Aniruddha Dasgupta, Hyesoon Kim, and Richard Vuduc. ”gpuperf: A performance analysis framework for identifying performance benefits in gpgpu applications”.In Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallal Programming (PPoPP), 2012. DOI: 10.1145/2145816.2145819

[86] Burton J. Smith. Readings in computer architecture. chapter Architecture and applications of the HEP multitrocessor computer system, pages 342–349. Morgan Kaufmann Publishers Inc., 2000.

[87] Allan Snavely, Larry Carter, Jay Boisseau, Amit Majumdar, Kang Su Gatlin, Nick Mitchell, John Feo, and Brian Koblenz. Multi-processor performance on the Tera MTA. In Proceedings of the 1998 ACM/IEEE conference on Supercomputing, pages 1–8, 1998. DOI: 10.1109/SC.1998.10049

[88] David Tarditi, Sidd Puri, and Jose Oglesby. Accelerator: using data parallelism to program gpus for general-purpose uses. In Proceedings of the 12th international conference on Architectural support for programming languages and operating systems, ASPLOS-XII, pages 325–335, New York, NY, USA, 2006. ACM.

[89] James E. Thornton. Design of a Computer: The Control Data 6600. Foresman Press,

1970.

[90] Rafael Ubal, Byunghyun Jang, Perhaad Mistry, Dana Schaa, and David Kaeli. Multi2Sim: A Simulation Framework for CPU-GPU Computing. In Proc. of the 21st International Conference on Parallel Architectures and Compilation Techniques, September 2012. DOI: 10.1145/2370816.2370865

[91] Leslie G. Valiant. A bridging model for parallel computation. Commun. ACM, 33:103–111, August 1990. DOI: 10.1145/79173.79181

[92] Leslie G. Valiant. A bridging model for multi-core computing. In Proceedings of the European Symposium on Algorithms (ESA), volume LNCS 5193, pages 13–28, Universität Karlsruhe, Germany, September 2008. Springer Berlin / Heidelberg. DOI: 10.1016/j.jcss.2010.06.012

[93] Jeffrey Scott Vitter. External memory algorithms and data structures: dealing with massive data. ACM Computing Surveys, 33(2):209–271, June 2001. DOI: 10.1145/384192.384193

[94] Jeffrey Scott Vitter. Algorithms and data structures for external memory. In Foundation and Trends in Theoretical Computer Science, volume 2, pages 305–474. now Publishers Inc., Hanover, MA, USA, 2006. DOI: 10.1561/0400000014

[95] Vasily Volkov. Better performance at lower occupancy. http://www.cs.berkeley.edu/~volkov/volkov10-GTC.pdf.

[96] Vasily Volkov and James W. Demmel. Benchmarking gpus to tune dense linear algebra. In Proceedings of the 2008 ACM/IEEE conference on Supercomputing, SC '08, pages 31:1–31:11, Piscataway, NJ, USA, 2008. IEEE Press. DOI: 10.1109/SC.2008.5214359

[97] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4):65, April 2009. DOI: 10.1145/1498765.1498785

[98] H. Wong, M.-M. Papadopoulou, M. Sadooghi-Alvandi, and A. Moshovos. Demystifying gpu microarchitecture through microbenchmarking. In Performance Analysis of Systems Software (ISPASS), 2010 IEEE International Symposium on, pages 235–246, March 2010. DOI: 10.1109/ISPASS.2010.5452013

[99] Dong Hyuk Woo and Hsien-Hsin S. Lee. Extending Amdahl's Law for energy-efficient computing in the many-core era. IEEE Computer, 41(12):24–31, December 2008. DOI: 10.1109/MC.2008.494

[100] Sven Woop, Jörg Schmittler, and Philipp Slusallek. Rpu: a programmable ray processing unit for realtime ray tracing. In ACM SIGGRAPH 2005 Papers, SIGGRAPH

’05, pages 434–444, New York, NY, USA, 2005. ACM.

[101] Lexing Ying, George Biros, and Denis Zorin. A kernel-independent adaptive fast multipole algorithm in two and three dimensions. Journal of Computational Physics, 196(2):591–626, May 2004. DOI: 10.1016/j.jcp.2003.11.021

[102] Kamen Yotov, Tom Roeder, Keshav Pingali, John Gunnels, and Fred Gustavson. An experimental comparison of cache-oblivious and cache-conscious programs. In Proceedings of the nineteenth annual ACM symposium on Parallel algorithms and architectures – SPAA ’07, page 93, New York, New York, USA, 2007. ACM Press. DOI: 10.1145/1248377.1248394

[103] Georgia L. Yuan, Ali Bakhoda, and Tor M. Aamodt. Complexity effective memory access scheduling for many-core accelerator architectures. In MICRO-42, pages 34–44, New York, NY, USA, 2009. ACM. DOI: 10.1145/1669112.1669119

[104] Vitaly Zakharenko. Fusionsim simulator. www.fusionsim.ca.

[105] Eddy Z. Zhang, Yunlian Jiang, Ziyu Guo, Kai Tian, and Xipeng Shen. On-the-fly elimination of dynamic irregularities for gpu computing. In Proceedings of the sixteenth international conference on Architectural support for programming languages and operating systems, ASPLOS ’11, pages 369–380, New York, NY, USA, 2011. ACM. DOI: 10.1145/1961295.1950408

[106] Yao Zhang and John D. Owens. A quantitative performance analysis model for GPU architectures. In HPCA, 2011. DOI: 10.1109/HPCA.2011.5749745 引用页码 70

作者传记

Hyesoon Kim

Hyesoon Kim 是佐治亚理工学院计算机科学学院的助理教授。她的研究兴趣包括高性能节能异构架构、程序员-编译器-微架构交互以及开发帮助并行编程的工具。她获得了韩国科学技术院（KAIST）机械工程学士学位、首尔国立大学机械工程硕士学位，以及德克萨斯大学奥斯汀分校计算机工程硕士和博士学位。她是 2011 年 NSF 职业奖的获得者。

Richard Vuduc

Richard（Rich）Vuduc 是佐治亚理工学院计算科学与工程学院助理教授。他的研究实验室 The HPC Garage（访问 https://hpcgarage.org/）专注于高性能计算，重点研究并行算法、性能分析和性能调优。其实验室的工作获得了众多最佳论文奖，并且其实验室是获得 2010 年戈登贝尔奖（超级计算最高性能成就奖）的团队的一部分。他是美国国家科学基金会职业奖（2010 年）的获得者，并曾担任美国国防高级研究计划局计算机科学研究组成员（2009 年）。Rich 在加州大学伯克利分校获得博士学位，并曾在劳伦斯利弗莫尔国家实验室做博士后学者。

Sara S. Baghsorkhi

Sara S. Baghsorkhi 是英特尔圣克拉拉编程系统实验室的研究科学家。她在伊利诺伊大学厄巴纳-香槟分校获得计算机科学博士学位。她的主要研究领域包括高性能计算机架构的自动调优和代码生成，专注于宽向量 SIMD 设计。她发表了 10 篇研究论文，拥有 7 项专利。

Jee W. Choi

Jee W. Choi 是佐治亚理工学院电气与计算机工程学院五年级博士生。他的研究兴趣包括多核、加速器和异构系统的性能和功耗建模。
Jee 在佐治亚理工学院获得学士和硕士学位。

Wen-mei W. Hwu

Wen-mei W. Hwu 是伊利诺伊大学厄巴纳-香槟分校电气与计算机工程系的 Sanders-AMD 冠名讲席教授。他的研究兴趣在于高性能、节能计算机系统的架构、编译和编程技术。他因开发用于谓词执行和推测执行的 IMPACT 编译器技术而闻名，该技术如今广泛

应用于 DSP 和 GPU 核心及编译器中。他是并行计算研究所（https://parallel.illinois.edu/）的首席科学家，也是 2.08 亿美元 NSF Blue Waters 超级计算机项目的联合首席研究员。他是 MulticoreWare 的联合创始人兼首席技术官。由于对编译器优化和计算机架构的贡献，他获得了 1993 年 Eta Kappa Nu 杰出青年电气工程师奖、1994 年伊利诺伊大学学者奖、1998 年 ACM SigArch Maurice Wilkes 奖、1999 年 ACM Grace Murray Hopper 奖、ISCA 影响力论文奖，以及加州大学伯克利分校计算机科学杰出校友奖。

Hwu 博士也一直处于计算机工程教育的前沿。他和 David Kirk 在伊利诺伊大学共同创建了一门本科异构并行编程课程（ECE498AL-Programming Parallel Processors），该课程已发展为 ECE408/CS483 - 应用并行编程。该课程已被许多大学采用。Hwu 和 Kirk 在全球范围内提供该课程的暑期学校版本。2010 年，Kirk 和 Hwu 出版了该课程的教科书，题为“Programming Massively Parallel Processors – A Hands-on Approach”，由 Elsevier 出版。截至 2012 年，已售出超过 12，000 册。由于他的教学和对教育的贡献，他获得了 1997 年 Eta Kappa Nu Holmes MacDonald 杰出教学奖和 2002 年 ECE 教学卓越奖。他是 IEEE 和 ACM 的会士。Hwu 博士在加州大学伯克利分校获得计算机科学博士学位。